

Андрей Карпов

ВРЕДНЫЕ
СОВЕТЫ
ДЛЯ
С++
ПРОГРАММИСТОВ

Андрей Карпов

Вредные советы для C++ программистов

2023

Аннотация

В книге читателю предлагается обширный список «советов», которым на самом деле не стоит следовать. Каждый совет сопровождается подробным разбором и рассмотрением неочевидных моментов. Эти пояснения будут полезны новичкам, изучающим программирование. Впрочем, книга развлечёт и профессионалов рассмотрением некоторых нюансов программирования на C++.

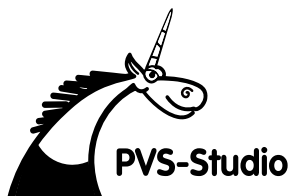
Введение

Изначально я задумал написать книгу о приёмах, помогающих в написании качественного кода. В ней хотелось рассмотреть такие темы, как оформление текста программ, на что стоит обращать внимание при обзоре кода, что из новых возможностей C++ стоит однозначно использовать, а что сомнительно. Но случайно меня увлекла другая тема, и я решил не сопротивляться потоку.

История началась с шуточной публикации на сайте «Хабр», где я сформулировал несколько вредных советов и предложил читателям продолжить этот список. Тема понравилась разработчикам, и они поделились массой своих идей о том, как можно создать ужасный код. Я взял часть советов, добавил свои и опубликовал в виде статьи. Там же я дал пояснение по некоторым советам, вся вредоносность которых может быть непонятна новичкам. Так и зародилась тема, которая в итоге превратилась в эту книгу.

Мне понравилось писать о том, как не надо делать, а не о том, как делать правильно. Хотя бы просто, для разнообразия. Я заскучал от написания правильных обучающих материалов. Захотелось чуть больше юмора и хулиганства.

Дело в том, что я многие годы занимаюсь темой программных ошибок и пишу о том, как их избегать. Это связано с моим участием в разработке и продвижении статического анализатора кода PVS-Studio.



Статический анализатор выдаёт предупреждения о фрагментах кода, которые с большой вероятностью содержат ошибки или не соответствуют используемым стандартам кодирования. PVS-Studio — один из таких инструментов. Его маскот — единорог — ещё не раз промелькнёт в этой книге. Сайт проекта: pvs-studio.ru

На примере открытых проектов я показываю, какие ошибки и потенциальные уязвимости могут быть выявлены с помощью этой методологии. Но, что ещё важнее, я стараюсь рассказывать, как написать код так, чтобы подобным ошибкам просто не нашлось места. В общем, вновь и вновь пишу, как лучше оформить код, какой комментарий написать, говорю о пользе обзоров кода и так далее.

Скучно. Пришло время творческих экспериментов. Я решил развить идею вредных советов и написать эту книгу. Впрочем, кроме идеи, от первоначального списка вредных советов мало что осталось. Я убрал или заменил всё слишком очевидное и неинтересное. Зато добавил много теоретического материала, в котором полезное найдут не только новички, но и опытные разработчики.

Нетрудно догадаться, что опыт, как стоит и не стоит писать код, я приобрёл, разрабатывая анализатор PVS-Studio и изучая обнаруженные с его помощью ошибки. Поэтому время от времени его упоминание будет всплывать в различных главах. Я настолько пропитался статическим анализом, что многие темы вызывают в моих воспоминаниях примеры реальных ошибок, которыми хочется поделиться для убедительности. А раз они убедительны, то почему бы о них и не написать.

Всё, теперь желаю приятного чтения.

Время приготовить чай/кофе, и вперёд.

Вредный совет N1. Только C++

Настоящие программисты пишут код на C++!

Нет ничего плохого в написании кода на C++. На этом языке написано множество прекрасных программ. Взять хотя бы список приложений с домашней страницы Бьёрна Страуструпа.

Вы можете познакомиться с этим списком, отсканировав приведённый ниже QR-код и перейдя по ссылке. Я буду использовать в книге QR-коды, когда нет смысла заимствовать какой-то материал из интернета или это невозможно. Ещё в тексте вы сможете встретить отсылки к терминам и приложениям, приведённым в конце книги. Заглядывайте туда, чтобы лучше разобраться, о чём идёт речь.



[Here is a list of systems, applications, and libraries](#) that are completely or mostly written in C++. Naturally, this is not intended to be a complete list. In fact, I couldn't list a 1000th of all major C++ programs if I tried, and this list holds maybe 1000th of the ones I have heard of. It is a list of systems, applications, and libraries that a reader might have some familiarity with, that might give a novice an idea what is being done with C++, or that I simply thought «cool».

Плохо, когда начинают использовать C++ только потому, что это «круто» или это единственный язык, с которым хорошо знакома команда.

Разнообразие языков программирования отражает многообразие задач, стоящих перед разработчиками приложений. Разные языки помогают элегантно решать различные классы задач.

Язык C++ претендует на звание универсального языка программирования. Однако универсальность не означает быстроту и простоту реализации конкретных приложений. Могут существовать языки, на которых проект будет реализован с меньшим вложением сил и времени.

Нет ничего страшного, если команда разработает небольшую вспомогательную утилиту на C++, хотя эффективнее для этого было бы использовать другой язык. Затраты на изучение нового языка могут превышать пользу от его применения.

Другое дело, когда перед командой стоит задача создания нового крупного проекта. В этот момент стоит остановиться и подумать: эффективно ли использовать для неё хорошо знакомый язык C++? Не лучше ли выбрать для этой задачи другой?

Если ответ — «да, использовать другой язык явно более эффективно», то, возможно, команде рационально потратить время на изучение этого языка. В перспективе это может на порядки сократить затраты на разработку и сопровождение. Или, возможно, стоит поручить этот проект другой команде, которая уже применяет более релевантный в данном случае язык.

Вредный совет N2. Табуляция в строковых литералах

Если в строковом литерале нужен символ табуляции, смело жмите клавишу Tab. Оставьте \t для яйцеголовых. Не парьтесь.

Речь идёт о строковых литералах, в которых требуется табуляцией отделять одни слова от других:

```
const char str[] = "AAA\tBBB\tCCC";
```

Казалось бы, по-другому и сделать нельзя. Тем не менее, нет пределов человеческой безалаберности. Кто-то, не задумываясь, нажимает клавишу Tab вместо того, чтобы использовать '\t'. Такое встречается в самых настоящих коммерческих приложениях.

Такой код компилируется и даже может работать. Однако явное использование символа табуляции плохо сразу по нескольким причинам:

1. На самом деле в литерале могут оказаться не табы, а пробелы. Это зависит от настроек редактора. Но выглядеть это будет так, как будто вставлена табуляция.
2. Человеку, который будет сопровождать код, не будет сразу очевидно, используется в качестве разделителей табуляция или пробелы.
3. Табуляция в процессе рефакторинга или использования утилит автоформатирования кода может превратиться в пробелы, что повлияет на результат работы программы.

Более того, однажды в реальном приложении я вообще видел приблизительно такой код:

```
const char table[] = "\
bla-bla-bla bla-bla-bla bla-bla-bla-bla-bla \n\
    bla-bla-bla        bla-bla-bla\n\
        %s                %d\n\
        %s                %d\n\
        %s                %d\n\
";

printf(table, "X", 1, "Foo", 2, "Time", 3);
```



Строка побита на части с помощью \. Явные символы табуляции использовались вперемешку с пробелами. К сожалению, не знаю, как здесь это показать. Поверьте, это смотрелось экстравагантно. Ещё и выравнивание от начала экрана. Бинго! Чего только не насмотришься, разрабатывая анализатор кода!

Явные табуляции, добавленные кнопкой Tab, очень легко «потерять». Если в процессе рефакторинга или автоформатирования они будут заменены на пробелы, то значения перестанут печататься в столбик:

```
bla-bla-bla bla-bla-bla bla-bla-bla-bla-bla
    bla-bla-bla        bla-bla-bla
        X                1
        Foo              2
        Time             3
```

По-нормальному этот код следовало оформить как-то так:

```
const char table[] =
    "bla-bla-bla bla-bla-bla bla-bla-bla-bla-bla\n"
    "    bla-bla-bla        bla-bla-bla\n"
    "        %s\t          %d\n"
    "        %s\t          %d\n"
    "        %s\t          %d\n";

printf(table, "X", 1, "Foo", 2, "Time", 3);
```

Символы табуляции нужны, чтобы второй столбец выглядел выровненным несмотря на размер названий в первом столбце («X», «Foo», «Time»). Подразумевается, что эти названия всегда короткие, т. е. меньше 8 символов. Исправленный код распечатает:

```
bla-bla-bla bla-bla-bla bla-bla-bla-bla-bla
```

```
    bla-bla-bla    bla-bla-bla
```

```
        X          1
```

```
      Foo          2
```

```
     Time          3
```

Кто-то может удивиться, что я рассказываю про такие простые и очевидные вещи. Более того, даже исправленный код попадает и не тянет на хороший пример для книги. Однако, раз я сам видел явные символы табуляции в строках, про это стоит рассказать. Даже такое исправление лучше, чем если кто-то будет продолжать нажимать где попало Tab, не понимая, почему это плохо.

Вредный совет N3. Вложенные макросы

Всюду используйте вложенные макросы. Так текст программы станет короче, и вы сохраните больше места на жёстком диске. Заодно это развлечёт ваших коллег при отладке.

Язык C++ открывает обширные возможности для того, чтобы обходиться без макросов. Я агитирую использовать макросы как можно реже.

Сразу оговорюсь, что я не являюсь фанатиком и не призываю отказываться от макросов из идеалистических соображений. Например, с макросами можно смириться, когда речь заходит о генерации однотипного кода. Также я спокойно отношусь к макросам в старых программах, написанных с использованием MFC. Нет смысла воевать с чем-то вроде этого:

```
BEGIN_MESSAGE_MAP(efcDialog, EFC_DIALOG_PARENT )
   //{{AFX_MSG_MAP(efcDialog)
    ON_WM_CREATE()
    ON_WM_DESTROY()
   //}}AFX_MSG_MAP
END_MESSAGE_MAP()
```

Существуют такие макросы, да и ладно. Они действительно были созданы для упрощения программирования.

Я говорю о других макросах, с помощью которых пытаются избежать реализации полноценной функции или стараются сократить размер функции. Рассмотрим несколько мотивов избегать таких макросов.

Первое: код с макросами притягивает к себе баги

Я не знаю, как объяснить причины этого явления с философской точки зрения, но это так. Более того, баги, связанные с макросами, часто очень сложно заметить, проводя обзор кода [см. термин «Обзор кода»].

Такие случаи я неоднократно описывал в своих статьях. Например, в проекте Midnight Commander я нашёл подмену функции *isspace* вот таким макросом:

```
#define isspace(c) ((c)==' ' || (c) == '\t')
```

Из-за него условие одного из циклов оказалось неверным. Программист, использовавший *isspace*, полагал, что вызывается настоящая функция, которая считает пробельными символами не только пробелы и табы, но также и

- смену страницы (0x0c, '\f');
- перевод строки LF (0x0a, '\n');
- возврат каретки CR (0x0d, '\r');
- вертикальный таб (0x0b, '\v').

Или как вам вот такое сокращение написания функции *std::printf*?

```
#define sprintf std::printf
```

Кто-то увлёкся макросами в проекте StarEngine, чтобы сокращённо записывать вызовы функции из стандартной библиотеки:

```
....
```

```
#define scout std::cout
```

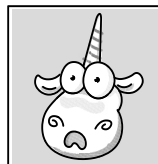
```
#define stringstream std::stringstream
```

```
#define ofstream std::ofstream
```

```
#define ifstream std::ifstream
```

```
#define fstream std::fstream
```

```
....
```



Другой разработчик наступил на эти заботливо разложенные грабли. Он полагал, что вызывает здесь стандартную функцию *sprintf* для печати в буфер:

```
sprintf(buf, "%d", value);
```

Вместо этого вызывалась функция `std::printf`. Вишенка на торте — форматная строка (первый аргумент) — неинициализированный буфер, который предназначался для записи в него, а не для чтения.

Можно возразить, что в этих ошибках виноваты программисты, а не макросы. Это так. Естественно, в ошибках всегда виноваты программисты. Но...

Важно, что макросы провоцируют ошибки. Получается, что макросы нужно использовать с повышенной аккуратностью или вообще не использовать. Рассмотрим ещё пару случаев для убедительности.

Библиотека ATL предоставляет для конвертации строк такие макросы, как `A2W`, `T2W` и так далее. Однако мало кто знает, что эти макросы очень опасно использовать внутри циклов. Внутри макроса происходит вызов функции `alloca`. Получается, что эта функция будет на каждой итерации цикла вновь и вновь выделять память на стеке. Программа может делать вид, что работает корректно. Однако, если она начнёт обрабатывать длинные строки или увеличится количество итераций в цикле, то стек может взять и закончиться в самый неожиданный момент.

Макросы, такие как `A2W`, прячут зло. Они выглядят как функции, но на самом деле имеют побочные эффекты, которые сложно заметить.

Не могу я пройти и мимо подобных попыток сокращать код с помощью макросов:

```
void initialize_sanitizer_builtins (void)
{
    ....
    #define DEF_SANITIZER_BUILTIN(ENUM, NAME, TYPE, ATTRS) \
    decl = add_builtin_function ("__builtin_" NAME, TYPE, ENUM, \
                                BUILT_IN_NORMAL, NAME, NULL_TREE); \
    set_call_expr_flags (decl, ATTRS); \
    set_builtin_decl (ENUM, decl, true);
```

```
#include "sanitizer.def"

if ((flag_sanitize & SANITIZE_OBJECT_SIZE)
    && !builtin_decl_implicit_p (BUILT_IN_OBJECT_SIZE))
    DEF_SANITIZER_BUILTIN (BUILT_IN_OBJECT_SIZE, "object_size",
        BT_FN_SIZE_CONST_PTR_INT,
        ATTR_PURE_NOTHROW_LEAF_LIST)
    ....
}
```

Только первая строка макроса относится к оператору *if*. Остальные строки будут выполняться независимо от условия. Можно сказать, что эта ошибка из мира C, так как она была найдена мною с помощью PVS-Studio внутри компилятора GCC. Код GCC написан в основном на C, а в этом языке без макросов обходиться тяжело. Однако здесь вполне можно было сделать настоящую функцию.

Второе: усложняется чтение кода

Если вы сталкивались с проектом, который весь пестрит макросами, состоящими из других макросов, то вы понимаете, какой это ад — разбираться в подобном коде. Если не сталкивались, то, поверьте на слово, это грустно. В качестве примера тяжёлого для восприятия кода могу привести уже упомянутый ранее компилятор GCC.

По легенде, компания Apple вложилась в развитие проекта LLVM как альтернативного варианта GCC по причине слишком большой сложности кода GCC из-за этих самых макросов. Я не могу вспомнить, где читал про это. Поэтому proof-ов не будет.

Третье: писать макросы сложно

Легко написать плохой макрос. Я нередко встречаю в коде ошибки, возникшие из-за таких неаккуратных макросов. А вот написать хороший и надёжный макрос часто сложнее, чем написать аналогичную функцию.

Написать хороший макрос сложно по той причине, что в отличие от функции его нельзя рассматривать как самостоятельную сущность. Требуется сразу рассматривать макрос в контексте всех возможных вариантов его использования, иначе очень легко получить проблему вида:

```
#define MIN(X, Y) (((X) < (Y)) ? (X) : (Y))
```

```
m = MIN(ArrayA[i++], ArrayB[j++]);
```

Инкремент переменных *i* и *j* будет выполняться не один раз, а дважды.

Конечно, для таких случаев давно придуманы обходные трюки, и макрос можно реализовать безопасно:

```
#define MAX(a,b) \
({ __typeof__ (a) _a = (a); \
  __typeof__ (b) _b = (b); \
  _a > _b ? _a : _b; })
```

Только вопрос, а нужно ли нам всё это в C++? Нет, в C++ есть шаблоны и другие способы построить эффективный код. Так почему я продолжаю встречать подобные макросы в C++ программах?

Четвёртое: усложняется отладка

Есть мнение, что отладка — это для слабаков. Нужно сразу писать код без ошибок, а не искать их отладчиком. Бла-бла... Это, конечно, интересно обсудить, но с практической точки зрения отладка полезна и помогает находить ошибки. Макросы усложняют этот процесс и однозначно замедляют поиск ошибок.

Пятое: ложные срабатывания статических анализаторов

Очень многие макросы в силу специфики своего устройства порождают множественные ложные срабатывания статических анализаторов кода. Могу смело утверждать, что большинство ложных срабатываний при проверке C и C++ кода связано именно с макросами.

Беда с макросами в том, что анализаторы просто не могут отличить корректный хитрый код от ошибочного.

Что делать?

Давайте не использовать макросы в C++ программах без крайней на то необходимости!

C++ предоставляет богатый инструментарий, такой как шаблонные функции, автоматический вывод типов (`auto`, `decltype`), `constexpr functions`.

Почти всегда вместо макроса можно написать функцию. Часто этого не делают по причине обыкновенной лени. Эта лень вредна, и с ней надо бороться. Небольшое дополнительное время, потраченное на написание полноценной функции, окупится с лихвой. Код будет проще читать и сопровождать: уменьшится вероятность отстрелить себе ногу, а компиляторы и статические анализаторы будут выдавать меньше ложных срабатываний.

Кто-то может возразить, что код с функцией менее эффективен. Это тоже только «отмазка».

Компиляторы сейчас отлично инлайнят код, даже если вы не написали ключевое слово *inline*.

Если же речь идёт о вычислении выражений на этапе компиляции, то и здесь макросы не нужны и даже вредны. Для тех же целей намного лучше и безопаснее использовать *constexpr*.

Поясню на примере. Перед вами классическая ошибка в макросе, найденная коллегой в ядре FreeBSD.

```
#define ICB2400_VPOPT_WRITE_SIZE 20

#define ICB2400_VPINFO_PORT_OFF(chan) \
    (ICB2400_VPINFO_OFF + \
     sizeof (isp_icb_2400_vpinfo_t) + \
     (chan * ICB2400_VPOPT_WRITE_SIZE))    // <=

static void
```

```
isp_fibre_init_2400(ispsoftc_t *isp)
{
    ....
    if (ISP_CAP_VP0(isp))
        off += ICB2400_VPINFO_PORT_OFF(chan);
    else
        off += ICB2400_VPINFO_PORT_OFF(chan - 1); // <=
    ....
}
```

Аргумент *chan* используется в макросе без обёртывания в круглые скобки. В результате на константу *ICB2400_VPOPT_WRITE_SIZE* умножается не выражение *(chan - 1)*, а только единица.

Раскрытый макрос:

```
off += (ICB2400_VPINFO_OFF + sizeof(isp_icb_2400_vpinfo_t) +
        (chan - 1 * ICB2400_VPOPT_WRITE_SIZE));
```

Ошибка бы не появилась, если вместо макроса была бы написана обыкновенная функция:

```
size_t ICB2400_VPINFO_PORT_OFF(size_t chan)
{
    return    ICB2400_VPINFO_OFF
        + sizeof(isp_icb_2400_vpinfo_t)
        + chan * ICB2400_VPOPT_WRITE_SIZE;
}
```

С большой вероятностью современный С и С++ компилятор самостоятельно выполнит подстановку (*inlining*) функции, и код будет столь же эффективен, как и в случае макроса.

При этом код стал более читаемым и защищённым от описанной ранее ошибки.

Если известно, что входным значением всегда является константа, то можно добавить *constexpr* и быть уверенным, что все вычисления произойдут на этапе компиляции. Ядро FreeBSD написано на С. Но предположим, что мы имеем дело с языком С++ и что *chan* — некая константа. Тогда функцию *ICB2400_*

VPINFO_PORT_OFF полезно объявить так:

```
constexpr size_t ICB2400_VPINFO_PORT_OFF(size_t chan)
{
    return    ICB2400_VPINFO_OFF
            + sizeof(isp_icb_2400_vpinfo_t)
            + chan * ICB2400_VPOPT_WRITE_SIZE;
}
```

Надеюсь, мне удалось вас убедить. Желаю вам поменьше макросов в коде!

Вредный совет N4. Выключить предупреждения

Отключите предупреждения компилятора. Они отвлекают от работы и мешают писать компактный код.

Программисты понимают, что предупреждения компиляторов — их друзья. Они помогают выявлять ошибки ещё на этапе компиляции кода. Исправить ошибку благодаря предупреждению компилятора намного проще и быстрее, чем отлаживая неработающий код.

Однако я на практике столкнулся с ситуацией, когда в одном большом проекте, часть компонентов компилировалась с полностью отключёнными предупреждениями.

Я написал новый код, запустил приложение и увидел, что оно ведёт себя не так, как планировалось. При повторном чтении своего кода я заметил ошибку и быстро её поправил, но был удивлён, что компилятор не выдаёт предупреждение. Это была какая-то очень грубая ошибка, что-то наподобие использования неинициализированной переменной. Я точно знал, что на такой код должно быть выдано предупреждение, но его не было.

В результате небольшого исследования выяснилось, что в компилируемом DLL-модуле и некоторых других полностью отключены предупреждения. Тогда я подключил старших коллег, чтобы провести расследование, как так вообще вышло.

В итоге выяснилось, что эти модули компилируются с выключенными предупреждениями уже несколько лет. В какой-то момент одному из сотрудников было поручено перевести сборку на новую версию компилятора. И он это сделал. Обновлённый компилятор начал выдавать новые предупреждения. И особенно много как раз на legacy-код этих модулей.

Неизвестно, что двигало человеком, но он просто отключил предупреждения в некоторых модулях. Возможно, он хотел

сделать это временно, чтобы предупреждения не мешали чинить ошибки компиляции.

А затем забыл включить предупреждения обратно. Узнать, к сожалению, как именно всё было, не представлялось возможным, так как к тому моменту этот человек уже не работал в компании.

Включив предупреждения обратно, мне пришлось потратить время на рефакторинг кода, чтобы компилятор не ругался. Но ничего непосильного. Я потратил на это один рабочий день. Зато в ходе разбора предупреждений я исправил ещё несколько ошибок в коде, которые до этого момента оставались незамеченными. Любите предупреждения компиляторов и анализаторов кода!

Полезный совет

Старайтесь, чтобы при компиляции проекта не выдавалось ни одного предупреждения. В противном случае вы столкнётесь с эффектом «разбитых окон» [см. термин «Теория разбитых окон»]. Если при компиляции кода постоянно выдаётся 10 предупреждений, то, когда появится 11-е предупреждение, это не будет казаться недопустимым. Если не вы, так коллеги напишут код, на который будут выдаваться предупреждения. Если считать, что это «ОК», то процесс станет неуправляемым. Чем больше предупреждений выдаётся при компиляции, тем меньше внимания обращается на новые.

Более того, постепенно предупреждения будут терять смысл. Если вы привыкли, что компилятор постоянно выдаёт предупреждения, вы просто не заметите новое, которое будет сообщать о реальной ошибке в новом коде.

-Werror

Ещё один хороший совет: указать компилятору интерпретировать предупреждения как ошибки. Так в вашей команде будет нулевая толерантность к предупреждениям. Вы будете всегда устранять или явно подавлять предупреждения. В про-

тивном случае код просто не скомпилируется. Такой подход очень хорошо сказывается на качестве кода.

Ложка дёгтя. Вам придётся сразу тратить дополнительное время на исправление предупреждений (превратившихся в ошибки компиляции) при обновлении версии компилятора или версии используемых библиотек.

В компиляторе могут появиться новые диагностические правила или быть улучшены старые. Ваш код перестанет компилироваться, и вам придётся сразу заняться его рефакторингом. Аналогично с библиотеками, в которых могут менять интерфейсы, макросы и т. д.

Является ли это причиной отказаться от `-Werror`? Нет, но лучше быть морально готовым к описанным нюансам.

Вредный совет N5. Чем короче имя переменной, тем лучше

Используйте для переменных имена из одной-двух букв. Так в одну строчку, помещающуюся на экране, можно уместить более сложное выражение.

Да, действительно, так можно написать короткий код. Он будет столь же коротким, сколь непонятным впоследствии. Фактически отсутствие нормальных имён переменных делает код `write-only`. Его можно написать и даже сразу отладить, пока ещё помнится, какая переменная что означает. Но по прошествии времени разобраться в таком коде будет крайне сложно.

Ещё один способ испортить код — это использовать аббревиатуры вместо нормального именования переменных. Пример: `ArrayCapacity` -> `AC`.

В первом случае сразу понятно, что речь идёт о «`saracity`» — размере зарезервированной памяти в контейнере [см. термин «`Saracity`»]. Во втором случае потом придётся гадать, что за загадочный `AC`.

Всегда ли следует избегать коротких имён? Нет. Ко всему нужно относиться разумно. Вполне уместно давать счётчикам в циклах такие имена, как i, j, k . Это устоявшаяся общепринятая практика, и любой программист понимает код с такими именами.

Иногда уместны и аббревиатуры. Например, в коде, реализующем численные методы, моделирование процессов и т. д. По факту этот код просто реализует вычисления по определённым формулам, описанным в комментариях или документации. Если в формуле какая-то переменная называется `SC0`, то разумно использовать именно это имя и в коде.

Пример — объявление переменных в проекте COVID-19 CovidSim Model:

```
int n;                /**< number of people in cell */  
int S, L, I, R, D; /**< S, L, I, R, D are numbers of Susceptible,  
                    Latently infected, Infectious,  
                    Recovered and Dead people in cell */
```

Допустимое именование переменных. Что они означают, описано в комментарии. Такое именование позволяет компактно записывать формулы:

```
Cells[i].S = Cells[i].n;  
Cells[i].L = Cells[i].I = Cells[i].R = Cells[i].cumTC = Cells[i].D = 0;  
Cells[i].infected = Cells[i].latent = Cells[i].susceptible + Cells[i].S;
```

Я не хочу сказать, что это хороший подход и стиль. Но иногда рационально давать и короткие имена. К любой рекомендации, правилу, методологии нужно подходить обдуманно и понимать, когда стоит сделать исключение, а когда нет.

Полезные рассуждения о том, как дать хорошие имена переменным, классам и функциям, есть в книге «Совершенный код» С. Макконнелла (ISBN 978-5-7502-0064-1). Всем её очень рекомендую.

Вредный совет N6. Магические числа

Используйте числа в программировании. Так ваша программа будет выглядеть умнее и солиднее. Согласитесь, что такие строки смотрятся хардкорно:
`qw = ty / 65 - 29 * s;`

Числа в коде, назначение которых неочевидно, называются магическими. Их использование является плохой практикой в программировании, так как делает код непонятным для коллег да и для самого автора по прошествии времени.

Магическое число само по себе не несёт никакого смысла и может вызвать недоумение, встретившись в коде программы без соответствующего контекста или комментария. При этом попытка изменить его на другое, даже близкое по значению, может привести к непредсказуемым последствиям. Магические числа вредят кроссплатформенности кода и в целом увеличивают стоимость сопровождения и развития проекта.

Намного лучше чисел использовать именованные константы и перечисления. Впрочем, это не означает, что каждая константа обязательно должна быть как-то названа. Во-первых, есть константы, такие как 0 или 1, суть использования которых очевидна. Во-вторых, программы, где происходят математические вычисления, могут только пострадать от попытки дать название каждой числовой константе. В этом случае лучше использовать комментарии, поясняющие формулы.

К сожалению, невозможно в одной главе описать множество подходов, позволяющих писать понятный красивый код. Поэтому я вновь отсылаю к уже упомянутой в предыдущей главе полезной обстоятельной книге «Совершенный код».

Иногда несложно догадаться, что означает то или иное число в коде, и поэтому его нельзя назвать по-настоящему «магическим». Однако такие числа тоже могут наносить вред. Давайте рассмотрим несколько примеров.

Первый пример я позаимствую из статьи разработчиков игр. Читал её около 10 лет назад, поэтому не могу вспомнить название статьи и где она была опубликована. В ней описывалось, что компания для внутренних нужд разработала специализированный статический анализатор кода, выявляющий проблемы, актуальные для кода игр. Одна из описанных диагностик предназначается для поиска таких констант, как 1024, 1600 и так далее. Это не что иное, как разрешение экрана в пикселях. Разработчик может что-то быстро попробовать, написав код с использованием этих чисел, а затем забыть заменить их на вызовы нужных функций.

Проблема этих чисел не в том, что их сложно идентифицировать. Разработчик быстро сообразит, что число имеет отношение к количеству пикселей на экране. Плохо то, что дефект выявляется только на этапе тестирования, когда проверяется работа игры при разных разрешениях экрана.

Второй пример — число Пи. Однажды статический анализатор PVS-Studio обратил наше внимание на вот этот код в Bullet Physics SDK:

```
float yScale = 1.0 / tan((3.141592538 / 180.0) * fov / 2);
```

Ему не понравилось число 3.141592538. В этом числе вы сразу узнаете математическую константу Пи. Что здесь не так? Неужели анализатор просто ругается на все числа подряд?

Нет. Дело в том, что это число только выглядит как Пи. На самом деле это не Пи. Число записано с ошибкой.

Число в коде: 3.141592 53 8

Настоящее Пи: 3.14159265358

Пропущены 6 и 5. Конечно, это не критично. Ошибка мало повлияет на точность вычислений. Однако было бы лучше, если бы автор кода не пытался блеснуть своими знаниями, беря значение Пи из головы. Ведь существует стандартная именованная константа M_PI .

Третий пример — размеры типов. Я неоднократно описывал в статьях ошибки, которые проявляют себя при портировании приложений, например, с 32-битной на 64-битную архитектуру.

Если говорить про магические числа, то с ними связаны ошибки вычисления размера буфера или неверные битовые операции.

```
size_t values[ARRAY_SIZE];
memset(values, 0, ARRAY_SIZE * 4);
```

Перед нами классическая ошибка, когда программист понадеялся на то, что размер типа *size_t* составляет 4 байта [см. термин «*size_t*»]. Так и было в 32-битной программе. В 64-битной размер типа *size_t* станет равен 8. В результате нулями заполнится только половина буфера.

Про размеры типов мы продолжим разговор в следующей главе, а сейчас ещё пример ошибки работы с битами. Однажды я встретил вот такой код:

```
// constant '1111..110000'
const size_t M = 0xFFFFFFFF0u;
```

Судя по комментарию, программист ожидает, что константа *M* будет представлять собой значение, где все биты, кроме четырёх младших, будут выставлены в единицу. В случае сборки 64-битного приложения результат будет отличаться от ожидаемого.

Константа *0xFFFFFFFF0u* имеет тип *unsigned int*. При расширении до 8-байтового беззнакового типа *size_t* старшие биты окажутся нулевыми.

Т. е. константа будет иметь следующее двоичное значение:

```
00000000 00000000 00000000 00000000 11111111 11111111 11111111 11110000
```

Впрочем, написать исправленный код без использования какой-то константы всё равно не получится:

```
// constant '1111..110000'
const size_t M = ~(static_cast<size_t>(0xF));
```

В любом случае, предупреждён — значит вооружён. Будьте внимательнее с кодом, содержащим явные числовые константы.

P.S. На самом деле можно (но не нужно) написать короче:

```
const size_t M = ~0xF;
```

Константа *0xF* имеет тип *int*. Следовательно, после инвертирования бит значение *0xFFFFFFF0* окажется знаковым. При конвертации в *size_t* оно сначала расширится до знакового значения *0xFFFFFFFFFFFFFFF0* типа *ptrdiff_t* [см. термин «*ptrdiff_t*»] и только затем станет нужным беззнаковым числом *0xFFFFFFFFFFFFFFF0u*. Во всём этом так много неочевидного, что лучше так не делать. Я за первый, более длинный, но зато понятный вариант.

Вредный совет N7. Везде int

Во всех старых книгах для хранения размеров массивов и для счётчиков циклов использовались переменные типа `int`. Так и делайте. Не стоит нарушать традиции.

Долгое время на распространённых платформах, где использовался язык C++, массив не мог на практике содержать более `INT_MAX` элементов. `INT_MAX` — максимальное значение для переменной типа `int`. Если тип `int` является 32-битным, это значение равно 2147483647.

Например, 32-битной программе на Windows доступно 2 GB памяти (на самом деле ещё меньше). Поэтому 32-битного типа `int` было более чем достаточно для хранения размера массивов или для их индексации.

Раньше программисты и авторы книг не заморачивались — смело использовали в циклах счётчики типа `int`. И всё было хорошо.

Однако на самом деле размер таких типов, как `int`, `unsigned` и даже `long`, может быть недостаточен. В этот момент Linux-программисты могут удивиться: почему `long` недостаточно? А дело в том, что, например, компилятор Visual C++ при сборке приложений для платформы Windows x64 использует модель данных LLP64, в которой тип `long` остался 32-битным.

short	int	long	ptr	long long	Label	Examples
	16		16		IP16	PDP-11 Unix (1973)
16	16	32	16		IP16L32	PDP-11 Unix (1977); multiple instructions for long
16	16	32	32		I16LP32	MC68000 (1982); Apple Macintosh 68K; Microsoft operation systems (plus extras for x86 segments)
16	32	32	32		ILP32	IBM 370; VAX Unix; many workstation
16	32	32	32	64	ILP32LL or ILP32LL64	Microsoft Win32; Amdahl; Convex; 1990 Unix systems; Like IP16L32, for same reason; multiple instructions for long long
16	32	32	64	64	LLP64 or IL32LLP64 or P64	64-bit systems Microsoft Win64 (X64/IA64) Most Unix systems (Linux, Solaris, DEC OSF/1 Alpha, SGI Irix, HP UX 11) HAL; logical analog of ILP32 UNICOS
16	32	64	64	64	LP64 or I32LP64	
16	64	64	64	64	ILP64	
64	64	64	64	64	SILP64	

А какие же тогда типы использовать? Безопасными для хранения размеров массивов или индексов являются типы `ptrdiff_t`, `size_t`, `intptr_t`, `uintptr_t`.

Рассмотрим простейший пример, когда использование 32-битного счётчика приведёт к ошибке при обработке большого массива в 64-битной программе:

```
std::vector<char> &bigArray = get();
size_t n = bigArray.size();
for (int i = 0; i < n; i++)
    bigArray[i] = 0;
```



Если контейнер содержит более `INT_MAX` элементов, то произойдёт переполнение знаковой переменной `int`, а это неопределённое поведение. Причём, как оно себя проявит, предсказать не так просто, как может показаться. Вы можете познакомиться с его интересным проявлением, перейдя по ссылке на статью [«Undefined behavior ближе, чем вы думаете»](#).

Правильным вариантом будет написать, например, так:

```
size_t n = bigArray.size();
for (size_t i = 0; i < n; i++)
    bigArray[i] = 0;
```

Ещё более правильным будет такой вариант:

```
std::vector<char>::size_type n = bigArray.size();
for (std::vector<char>::size_type i = 0; i < n; i++)
    bigArray[i] = 0;
```

Согласен, такой вариант длинноват. И может возникнуть соблазн использовать автоматический вывод типа. К сожалению, тогда опять можно получить некорректный код:

```
auto n = bigArray.size();  
for (auto i = 0; i < n; i++)    // опять ошибочный код  
    bigArray[i] = 0;
```

Переменная *n* будет иметь правильный тип, а вот счётчик *i* — нет. Константа 0 имеет тип *int*, а значит, переменная *i* тоже будет иметь тип *int*. И мы возвращаемся к тому, с чего начали.

Так как же правильно перебрать элементы и при этом написать короткий код? Во-первых, можно использовать итераторы:

```
for (auto it = bigArray.begin(); it != bigArray.end(); ++it)  
    *it = 0;
```

Во-вторых, можно использовать range-based for loop:

```
for (auto &a : bigArray)  
    a = 0;
```

Читатель может сказать, что всё правильно, но неприменимо к его программам. Все массивы, которые создаются в его коде, в принципе не могут быть большими, и поэтому можно по-прежнему использовать переменные *int* и *unsigned*. Рассуждение неверно по двум причинам.

Первая причина. Такой подход потенциально опасен для будущего. То, что сейчас программа не работает с большими массивами, не означает, что так будет всегда. Ещё один сценарий — код может быть заимствован в другое приложение, где обработка больших массивов — обычное дело.



В конце концов, одной из причин падения ракеты Ariane 5 стало как раз использование старого кода, не рассчитанного на новые величины «горизонтальной скорости». Подробнее про это можно прочитать в статье [«Космическая ошибка: 370.000.000 \\$ за Integer overflow»](#).

Вторая причина. При использовании смешанной арифметики можно получить проблемы, работая даже с маленькими массивами. Рассмотрим пример кода, который работоспособен в 32-битном варианте и неработоспособен в 64-битном:

```
int A = -2;
unsigned B = 1;
int array[5] = { 1, 2, 3, 4, 5 };
int *ptr = array + 3;
ptr = ptr + (A + B);    // Invalid pointer value on 64-bit platform
printf("%i\n", *ptr);  // Access violation on 64-bit platform
```

Давайте проследим, как происходит вычисление выражения $ptr + (A + B)$:

1. Согласно правилам языка C++, переменная A типа *int* приводится к типу *unsigned*.
2. Происходит сложение A и B . В результате мы получаем значение $0xFFFFFFFF$ типа *unsigned*.
3. Вычисляется выражение $ptr + 0xFFFFFFFFu$.

Что из этого выйдет, будет зависеть от размера указателя на данной архитектуре. Если сложение будет происходить в 32-битной программе, то данное выражение будет эквивалентно $ptr - 1$ и мы успешно распечатаем число «3». В 64-битной программе к указателю честным образом прибавится значение $0xFFFFFFFFu$. Указатель окажется далеко за пределами массива, и при доступе к элементу по данному указателю нас ждут неприятности.

Вредный совет N8. `sizeof(int) == sizeof(void *)`

Размер указателя и `int` — это всегда 4 байта. Смело используйте это число. Число 4 смотрится намного изящнее, чем корявое выражение с оператором `sizeof`.

Это продолжение предыдущей темы, что `int` используется, где надо и не надо. Во-первых, размер `int` может быть очень даже разным. На многих популярных платформах размер `int` действительно 4 байта. Но многие — это не означает все! Существуют системы с различными моделями данных, где `int` может содержать и 8 байт, и 2 байта, и даже 1 байт!

Формально про размер `int` можно сказать только следующее:

```
1 == sizeof(char) <=
    sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)
```

Во-вторых, размер указателя точно так же легко может отличаться от размера типа `int` и значения 4. Например, на большинстве 64-битных систем размер указателя составляет 8 байт, а типа `int` — 4 байта.

С этим связан достаточно распространённый паттерн 64-битной ошибки. В старых 32-битных программах указатели иногда сохраняли в переменные таких типов, как `int/unsigned`. При портировании таких программ на 64-битные системы возникают ошибки, так как при записи значения указателя в 32-битную переменную происходит потеря старших бит.

Какой же тогда целочисленный тип использовать для хранения указателей?

Для начала такая необходимость говорит о плохом дизайне системы. Стоит задуматься, не переписать ли такой код. Если нужно хранить разные данные в одном буфере памяти, в C можно воспользоваться `union`:

```
union MyVariant {  
    char m_c;  
    int m_i;  
    float *m_ptr;  
};
```

Использование *union* не означает красоту в дизайне. Но это лучше, чем использовать множество явных приведений типов для «впихивания одного в другое». В случае C++ помимо *union* программисту доступен класс *std::variant*, который обеспечивает большую безопасность с точки зрения типов.

Если всё-таки требуется сохранить указатель в переменной целочисленного типа, то наилучшим вариантом будут *intptr_t* и *uintptr_t*. Само название типов говорит, что они рассчитаны на хранение указателей.

```
void *ptr = foo();  
uintptr_t a = reinterpret_cast<uintptr_t>(ptr);  
ptr = reinterpret_cast<void *>(a);
```

Некоторые Linux разработчики используют для хранения указателей тип *long*, и это плохой выбор, делающий код непереносимым. Обратите внимание на табличку, приведённую в предыдущей главе. В 64-битных Linux системах тип *long* становится 64-битным, а, например, в 64-битном Windows он по-прежнему 32-битный и не может вместить указатель.

Вредный совет N9. Глобальные переменные

Глобальные переменные очень удобны, так как к ним можно обращаться отовсюду.

Глобальные переменные доставляют проблемы программистам, поскольку они увеличивают степень взаимодействия/взаимозависимости между различными компонентами программы, которые к ним обращаются. Это означает, что по мере увеличения числа функций, использующих глобальную переменную, сложность взаимодействий между ними также возрастает.

Более того, получается, что функции, использующие глобальные переменные, имеют внутреннее состояние. Это затрудняет понимание кода, его сопровождение и тестирование.

Усиление взаимосвязанности функций и наличие у них состояний повышает риск появления в программе ошибок при внесении каких-либо изменений. Затрудняется поиск и исправление этих дефектов.

Противоположностью использования глобальных переменных является принцип инкапсуляции. Целью инкапсуляции является сокращение взаимозависимостей, чтобы упростить понимание исходного кода и внесение в него изменений, а также чтобы сделать код безопаснее и легче в тестировании.



Большое количество глобальных переменных может свидетельствовать о низком качестве программного кода и его ненадёжности. Показательным примером может служить исследование кода, используемого в старых моделях Toyota. После множественных отказов системы торможения было проведено исследование программного обеспечения автомобиля, которое выявило, что в коде используется более 9000 глобальных переменных. [По ссылке вы найдёте подробности этой истории.](#)

Однако глобальные переменные не всегда «абсолютное зло».

Во-первых, глобальные переменные могут быть константными. Собственно, тогда они никакие не «переменные», а просто константы.

Во-вторых, общая область данных может использоваться для хранения глобальных параметров, на которые ссылаются во всём приложении. Этот подход часто применяется для встроенных приложений с небольшим объёмом используемой памяти. При использовании глобальных переменных в подобных приложениях обычно только один компонент может записывать в область данных, а все остальные компоненты рассматривают эту область как *const* и только считывают из неё, никогда и ничего туда не записывая. Такой подход сокращает количество возможных проблем.

Вредный совет N10. `abort` в библиотеках

Совет для разработчиков библиотек: в любой непонятной ситуации сразу завершай программу, используя функцию `abort` или `terminate`.

Иногда в программах можно встретить очень простую обработку ошибок: завершение работы программы. Чуть что-то не получилось, например открыть файл или выделить память, как тут же вызывается функция *`abort`*, *`exit`* или *`terminate`*. Для некоторых утилит и простых программ это вполне приемлемое поведение. Да и вообще автор программы сам вправе решить, что делать в случае сбоя в работе приложения.

Однако такой подход недопустим, если вы разрабатываете библиотечный код. Неизвестно, в каких приложениях он будет использоваться. Библиотечный код должен вернуть статус ошибки или сгенерировать исключение. А уже пользовательскому коду решать, как будет обрабатываться возникшая ошибочная ситуация.

Например, пользователь графического редактора будет не в восторге, если библиотека, предназначенная для печати изображения на принтере, завершит работу приложения, не дав сохранить результаты его работы.

А что, если библиотекой захочет воспользоваться *embedded*-разработчик? Такие руководства для разработчиков встраиваемых систем, как MISRA и AUTOSAR [см. термин «MISRA, AUTOSAR»], вообще запрещают вызывать функции аварийного завершения программы. И это вполне понятно, так как для встраиваемых систем часто нет понятия «просто остановить выполнение программы». Нужно предпринять какие-то действия по отключению питания от двигателей, переводу манипуляторов в безопасное положение и тому подобному.

Вредный совет N11. Во всём виноват компилятор

Если что-то не работает, то, скорее всего, глючит компилятор. Попробуйте поменять местами некоторые переменные и строки кода.

Любой состоявшийся программист понимает, что совет абсурден. Однако на практике не так редка ситуация, когда программист спешит обвинить компилятор в неправильной работе его программы.

Конечно, в компиляторах тоже бывают ошибки, и с ними можно столкнуться. Однако в 99 % случаев, когда кто-то говорит, что «компилятор глючит», он неправ и на самом деле в коде присутствует незамеченный баг.

Чаше всего программист или не понимает какие-то тонкости языка C++, или столкнулся с неопределённым поведением. Давайте рассмотрим пару таких примеров.

Первая история берёт своё начало из обсуждения, происшедшего на форуме linux.org.ru.

Программист жаловался на глюк в компиляторе GCC 8. Как затем выяснилось, виной всему являлся некорректный код, приводящий к неопределённому поведению.

```
int foo(const unsigned char *s)
{
    int r = 0;
    while(*s) {
        r += ((r * 20891 + *s * 200) | *s ^ 4 | *s ^ 3) ^ (r >> 1);
        s++;
    }
    return r & 0x7fffffff;
}
```

Компилятор не генерирует код для оператора побитового И (&). Из-за этого функция возвращает отрицательные значения, хотя, по задумке программиста, этого происходить не должно.

Разработчик считает, что это глюк в компиляторе. Но на самом деле неправ программист, который написал такой код. В функции возникает неопределённое поведение, поэтому она работает не так, как ожидается.

Компилятор видит, что в переменной *r* считается некоторая сумма. Переполнения переменной *r* произойти не должно (с точки зрения компилятора). Иначе это неопределённое поведение, которое компилятор никак не должен рассматривать и учитывать. Итак, компилятор считает, что значение в переменной *r* после окончания цикла не может быть отрицательным. Следовательно, операция *r & 0x7fffffff* для сброса знакового бита является лишней, и компилятор решает просто возвращать из функции значение переменной *r*.

Вот такая интересная ситуация, когда программист поспешил пожаловаться на компилятор. По мотивам этого случая мы даже реализовали в анализаторе PVS-Studio диагностику V1026, которая помогает выявлять подобные дефекты в коде.

Чтобы исправить код, достаточно считать хэш, используя для этого беззнаковую переменную:

```
int foo(const unsigned char *s)
{
    unsigned r = 0;
    while(*s) {
        r += ((r * 20891 + *s * 200) | *s ^ 4 | *s ^ 3) ^ (r >> 1);
        s++;
    }
    return (int)(r & 0x7fffffff);
}
```

Вторая история о конструкторе с интересным комментарием. Однажды при проверке медиадекодера `ffdshow` анализатор PVS-Studio выдал предупреждение на этот код:

```
TprintPrefs::TprintPrefs(IffdshowBase *Ideci,
                        const TfontSettings *IfontSettings)
{
    memset(this, 0, sizeof(this)); // This doesn't seem to
                                   // help after optimization.

    dx = dy = 0;
    isOSD = false;
    xpos = ypos = 0;
    align = 0;
    linespacing = 0;
    sizeDx = 0;
    sizeDy = 0;
    ....
}
```

Анализатор прав. Автор кода – нет.

Комментарий говорит нам: компилятор глючит при включении оптимизации и не обнуляет поля структуры.

Поругав компилятор, программист ниже пишет код, который по отдельности обнуляет каждый член класса. Печально, но, скорее всего, программист остался уверенным в своей правоте. А ведь на самом деле перед нами обыкновенная ошибка из-за невнимательности.

Обратите внимание на третий аргумент функции *memset*. Оператор *sizeof* вычисляет вовсе не размер класса, а размер указателя. В результате обнуляется только часть объекта. В режиме без оптимизаций, видимо, все поля всегда были обнулены и казалось, что функция *memset* работала правильно.

Правильное вычисление размера класса должно выглядеть

так:

```
memset(this, 0, sizeof(*this));
```

Впрочем, даже исправленный вариант кода нельзя назвать правильным и безопасным. Он остаётся таким до тех пор, пока класс тривиально копируемый. Всё может сломаться, стоит, например, добавить в класс виртуальную функцию или поле нетривиально копируемого типа.

Не надо так писать. Я привёл этот пример только потому, что эти нюансы меркнут перед ошибкой вычисления размера класса.

Вот так и рождаются легенды о глючных компиляторах и от-важных программистах, которые с ними сражаются.

Вывод. Не торопитесь обвинять компилятор в неработоспособности вашего кода. И уж тем более не пытайтесь добиться работоспособности вашей программы различными модификациями кода в надежде «обойти баг компилятора».

Прежде чем подозревать компилятор, полезно:

1. Попросить опытных коллег провести обзор кода.
2. Внимательно посмотреть, не выдаёт ли компилятор на код предупреждения, и попробовать такие ключи, как *-Wall*, *-pedantic*.
3. Проверить код статическим анализатором, таким как PVS-Studio.
4. Проверить код динамическим анализатором.
5. Изучить ассемблерный листинг, сгенерированный компилятором для кода, и подумать, почему он мог таким получиться.
6. Воспроизвести ошибку в как можно более коротком коде и задать вопрос на сайте Stack Overflow (stackoverflow.com).

Вредный совет N12. `double == double`

Смело сравнивайте числа с плавающей запятой с помощью оператора `==`. Раз есть такой оператор, значит им нужно пользоваться.

Сравнивать-то можно... Вот только у такого сравнения есть нюансы, которые нужно знать и учитывать. Рассмотрим пример:

```
double a = 0.5;
if (a == 0.5)           // True
    foo();
```

```
double b = sin(M_PI / 6.0);
if (b == 0.5)           // ???
    foo();
```

Первое сравнение `a == 0.5` истинно. Второе сравнение `b == 0.5` может быть как истинно, так и ложно. Результат выражения `b == 0.5` зависит от используемого процессора, версии и настроек компилятора. Например, когда я пишу эту статью, мой компилятор создал код, который вычисляет значение переменной `b`, равное 0.49999999999999994.

Более корректно этот код можно написать следующим образом:

```
double b = sin(M_PI / 6.0);
if (std::abs(b - 0.5) < DBL_EPSILON)
    foo();
```

В данном случае сравнение с погрешностью `DBL_EPSILON` верно, так как результат функции `sin` лежит в диапазоне `[-1, 1]`. `DBL_EPSILON` — это минимальное значение, отделяющее число 1.0 от другого ближайшего к нему значения: `1.0 + DBL_EPSILON != 1.0`.

Если мы работаем со значениями больше нескольких единиц, то такие погрешности, как *FLT_EPSILON*, *DBL_EPSILON*, могут оказаться слишком малы. И наоборот, при работе со значениями типа 0.00001 эти погрешности слишком велики. Каждый раз следует выбирать погрешность, адекватную диапазону возможных значений.

Почему так? В отличие от чисел с фиксированной запятой, сетка чисел, которые способна отобразить арифметика с плавающей запятой, неравномерна. Она более густая для чисел с малыми порядками и более редкая для чисел с большими порядками.

Возникает вопрос: как же все-таки сравнить две переменные типа *double*?

```
double a = ...;
double b = ...;
if (a == b) // how?
{
}
```

Одного единственного правильного ответа нет. В большинстве случаев можно сравнить две переменных типа *double*, написав код следующего вида:

```
if (std::abs(a - b) <= DBL_EPSILON * std::max(std::abs(a),
                                              std::abs(b)))
{
}
```

Только осторожней с этой формулой, она работает лишь для чисел с одинаковым знаком. Также при большом количестве вычислений постоянно набегают ошибки и константа *DBL_EPSILON* может оказаться слишком маленькой.

А можно ли все-таки точно сравнивать значения в формате с плавающей точкой?

В некоторых случаях — да. Но эти ситуации весьма ограничены. Сравнивать можно в том случае, если это и есть, по сути, одно и то же значение.

Пример, где допустимо точное сравнение:

```
// -1 - признак, что значение переменной не было установлено
```

```
double val = -1.0;
```

```
if (Foo1())
```

```
    val = 123.0;
```

```
if (val == -1.0) // OK
```

```
{
```

```
}
```

В данном случае сравнение со значением `-1` допустимо, так как именно точно таким же значением мы инициализировали переменную ранее.

Такое сравнение будет работать даже в случае, если число не может быть представлено конечной дробью. Следующий код распечатает «`V == 1.0/3.0`»:

```
double V = 1.0/3.0;
```

```
if (V == 1.0/3.0)
```

```
{
```

```
    std::cout << "V == 1.0/3.0" << std::endl;
```

```
} else {
```

```
    std::cout << "V != 1.0/3.0" << std::endl;
```

```
}
```

Однако надо быть очень бдительным. Достаточно заменить тип переменной `V` на `float`, и условие станет ложным:

```
float V = 1.0/3.0;
```

```
if (V == 1.0/3.0)
```

```
{
```

```
    std::cout << "V == 1.0/3.0" << std::endl;
```

```
} else {
```

```
    std::cout << "V != 1.0/3.0" << std::endl;
```

```
}
```

Этот код уже печатает « $V \neq 1.0/3.0$ ». Почему? Значение переменной V равно 0.333333 , а значение $1.0/3.0$ равно 0.3333333333333333 . Перед сравнением переменная V , имеющая тип *float*, расширяется до типа *double*. Происходит сравнение:

```
if (0.3333330000000000 == 0.3333333333333333)
```

Эти числа, естественно, неравны. В общем, будьте аккуратны.

Вредный совет N13. Undefined behavior — просто страшилка

Undefined behavior – это страшилка на ночь для детей. На самом деле всегда понятно, как работает правильный и неправильный код. Всё очень даже «defined», просто нужно хорошо знать C++.

Основная беда с undefined behavior (неопределённым поведением) не в том, что оно существует, а в том, что программисты думают, что могут совладать с ним. Из-за этого они пишут код, который, на их взгляд, будет работать именно так, как задумано, даже зная, что формально он содержит неопределённое поведение. Они полагают, что учтут, как неопределённое поведение проявит себя, и напишут работоспособный код. *Очень хорошо про это написано в статье [«Ложные представления программистов о неопределённом поведении»](#).*

В ней перечислены:



- заблуждения о том, когда «происходит» UB;
- заблуждения о выполнении UB;
- заблуждения о возможных последствиях неопределённого поведения;
- заблуждения из серии «но раньше же прекрасно работало»;
- заблуждения об автономном поведении UB;
- заблуждения о UB в целом.

Можно сто раз повторить, что нельзя угадать, что такое неопределённое поведение. Однако это всё слова. Бла-бла... «Настоящим» программистам эти страшилки не помеха, они ведь «знают».

Значит, сейчас передо мной стоит задача показать на примерах, насколько тщетны такие попытки угадать проявление неопределённого поведения. Возможно, после этих примеров кто-то скажет: «Чёрт, что-то я промахнулся с предположением».

Он получит прививку от излишней программисткой самоуверенности и поспешных выводов. Моя цель будет достигнута: кто-то станет аккуратней писать код.

Пример N1: отсутствие `return`

Отсутствие оператора `return` в функции, возвращающей значение, приводит к неопределённому поведению.

The C++ Standard, [stmt.return], paragraph 2 [ISO/IEC 14882-2014], states the following: Flowing off the end of a function is equivalent to a return with no value; this results in undefined behavior in a value-returning function.

Рассмотрим некорректный код:

```
bool foobar(const std::string &s)
{
    std::string sx = s;
    std::cout << sx << std::endl;
}

int main(int argc, char **argv)
{
    bool b = foobar(argv[1]);
    std::cout << b << std::endl;
    return 0;
}
```

Как ведёт себя код? Правильный ответ: неизвестно.

Однако программист может начать размышлять следующим образом. Да, функция ничего не возвращает. Скорее всего, для возврата значения типа `bool` используется один из регистров процессора. Он будет хранить какое-то случайное значение. Значит, в переменную `b` будет записано случайное число. Программа распечатает первый аргумент командной строки и затем 0 или 1 в зависимости от случайного значения переменной `b`.

Можно ожидать, что программа, запущенная с аргументом «Test», напечатает:

```
Test
0
```

или

```
Test
1
```

Звучит убедительно?

Возможно, убедительно, но реальность другая. Например, мой компилятор с моими настройками построил программу, которая выводит:

```
Test
208
```

На этом всё? Нет, это один из вариантов проявления неопределённого поведения. Могут быть другие: например, программа может упасть. Именно такой случай я описывал в статье «Пример проявления неопределённого поведения из-за отсутствия return». Если хотите, можете загуглить её, а здесь я кратко перескажу суть.

```
#include <string>
#include <iostream>

bool foobar(const std::string &s)
{
    std::string sx = s;
    std::cout << sx << std::endl;
}

int main(int argc, char **argv)
{
    foobar(argv[0]);
    return 0;
}
```

Сборка: Linux, libc-2.33, GCC 11.1.0, оптимизация -O2.

Эта программа падает с SIGSEGV: Segmentation fault (core dumped).

Компилятор не генерирует инструкцию возврата из функции (*ret*). Её просто нет.

Пример N2: разыменование нулевого указателя

Разыменовывание нулевого указателя приводит к неопределённому поведению. Но иногда может казаться, что не приводит. Рассмотрим следующий код:

```
bool init(usb_line6_podhd *usb)
{
    usb_line6 *line6 = &usb->m_line6;
    if (usb == nullptr)
        return false;
    line6->card = card;
    ....
}
```

Как неопределённое поведение проявит себя здесь? Можно дать неправильный ответ: никак. Рассмотрим ход неверных рассуждений.

Указатель *usb* на самом деле не разыменовывается. Если *usb* будет нулевым, то указатель *line6* окажется невалидным, но это нестрашно. В нём будет храниться смещение члена *m_line6* от начала структуры *usb_line6_podhd*. Чтобы вычислить это смещение, указатель *usb* «физически» не разыменовывается. Используется адресная арифметика без обращения к памяти по указателям.

Далее указатель *usb* проверяется на равенство нулю. Если указатель нулевой, то функция завершает работу. Невалидный указатель *line6* нигде до момента проверки переменной *usb* не используется, так что всё хорошо.

Если указатель *usb* нулевой, то функция завершит работу, не произведя никакого некорректного доступа к памяти. Если указатель *usb* не нулевой, то далее указатели можно смело использовать.

Более того, есть даже доказательство, что адресная арифметика над нулевыми указателями (без доступа к данным) безопасна. В некоторых компиляторах макрос *offsetof* раскрывается следующим образом:

```
#define offsetof(st, m) ((size_t)(&((st *)0)->m))
```

Звучит логично и обоснованно? Да. Это возможный сценарий? Да. Неопределённое поведение никак себя здесь не проявляет, и код будет корректно работать.

Казалось бы, на этом можно поставить точку, ведь сомнительно, что возможны какие-то другие варианты. Но в том-то и дело, что они возможны, так как мы говорим о неопределённом поведении. Сейчас я опишу другой столь же реальный сценарий.

Сразу скажу, что приведённая реализация *offsetof* не является аргументом в защиту рассуждений. Конкретные реализации некоторых компиляторов могут корректно работать с таким макросом, а другие — нет.

Макрос *offsetof* не может быть реализован в рамках стандартных языковых конструкций C++ и требует поддержки со стороны компилятора. Например, GCC и Clang раскрывают этот макрос в вызов intrinsic-функции *__builtin_offsetof*.

Ещё раз взглянем на код.

```
usb_line6 *line6 = &usb->m_line6;

if (usb == nullptr)
    return false;

line6->card = card;
```

Компилятор вправе интерпретировать этот код так:

1. В выражении *usb->m_line6* указатель *usb* разыменовывается. Значит, он ненулевой.. Если он будет нулевым, то это неопределённое поведение и компилятор считает такой сценарий невозможным.
2. Проверка *usb == nullptr* избыточна. Выше уже стало известно, что указатель ненулевой. Следовательно, в целях оптимизации лишнюю проверку следует удалить.

В результате код, скомпилированный оптимизирующим компилятором, будет эквивалентен следующему:

```
usb_line6 *line6 = &usb->m_line6;  
line6->card = card;
```

При таком сценарии компиляции неопределённое поведение очень даже ярко проявит себя. Никогда не разыменовывайте указатели, которые могут быть нулевыми.

Пример N3: переполнение int

Рассмотрим некорректный пример кода.

```
size_t Count = 1024*1024*1024; // 1 Gb  
if (is64bit)  
    Count *= 5; // 5 Gb  
  
char *array = (char *)calloc(Count, sizeof(char));  
  
int index = 0;  
for (size_t i = 0; i != Count; i++)  
    array[index++] = char(i) | 1;  
  
if (array[Count - 1] == 0)  
    printf("Ошибка: последний элемент остался нулевым!\n");  
  
free(array);
```

Этот код работает, если собрать 32-битную версию программы, так обрабатывается только 1 Gb данных. А вот если собрать 64-битный вариант программы, всё становится намного интересней.

64-битная программа выделяет массив байт размером в 5 гигабайт и заполняет его нулями. Затем в цикле массив заполняется разными числами, не равными нулю. Чтобы числа не были равны 0, используется «| 1». Код выглядит немного странным, так как приходится подбирать содержимое примеров, чтобы неопределённое поведение чётко себя демонстрировало.

Попробуйте угадать, как поведёт себя эта программа, собранная в режиме x64. Заготовили ответ? Если да, то продолжим.

Можно предположить, что программа упадёт из-за выхода за границу массива. В какой-то момент переменная *index* переполнится и её значение станет равно -2147483648 (*INT_MIN*). Впрочем, думаю, вы уже подозреваете, что не всё так просто.

В своё время я компилировал этот код с помощью Visual C++ 14.0. Если скомпилировать и запустить отладочную версию этой программы, то она действительно аварийно завершится из-за выхода за границу массива.

Магия начинается, если собрать приведённый пример кода в режиме с оптимизацией. Программа перестанет падать, корректно выполнится, и сообщение «Ошибка: последний элемент остался нулевым!» выдано не будет.

Подождите, но это же невозможно! Как удалось записать ненулевое значение в последний элемент массива? Ведь 32-битной переменной типа *int* недостаточно, чтобы пройти по всем ячейкам массива, содержащего более 5 миллиардов элементов. Ха! Неопределённое поведение позволило случиться невозможному.

Хотя на самом деле всё просто. Взглянем на сгенерированный ассемблерный код:

```
int index = 0;
for (size_t i = 0; i != Count; i++)
0000000013F6D102D xor      ecx,ecx
0000000013F6D102F nop
    array[index++] = char(i) | 1;
0000000013F6D1030 movzx    edx,cl
0000000013F6D1033 or      dl,1
0000000013F6D1036 mov     byte ptr [rcx+rbx],dl
0000000013F6D1039 inc     rcx
0000000013F6D103C cmp     rcx,rdi
0000000013F6D103F jne     main+30h (013F6D1030h)
```

Вот что произошло. Переменная *index* типа *int* увеличивается в цикле. С точки зрения компилятора она никогда не может переполниться, так как это неопределённое поведение. Значит, переменной типа *int* достаточно для индексации всего массива, её можно увеличивать без каких-либо дополнительных действий и проверок.

Переменная *index* используется для индексации массива, поэтому в целях оптимизации компилятор использует для этой переменной не 32-битный, а 64-битный регистр *rcx*. С точки зрения компилятора нет никакой разницы. В результате компилятор «чинит» неработоспособный код.

Условно можно сказать, что изменился тип переменной:

```
int64_t index = 0;
for (size_t i = 0; i != Count; i++)
    array[index++] = char(i) | 1;
```

Красивое проявление неопределённого поведения.

Если заменить *int* на *unsigned*, неопределённое поведение исчезнет. Массив будет заполнен только частично, и в конце будет выдано сообщение: «Ошибка: последний элемент остался нулевым!». Ассемблерный код при использовании типа *unsigned*:

```
    unsigned index = 0;
0000000013F07102D xor      r9d,r9d
    for (size_t i = 0; i != Count; i++)
0000000013F071030 mov      ecx,r9d
0000000013F071033 nop      dword ptr [rax]
0000000013F071037 nop      word ptr [rax+rax]
        array[index++] = char(i) | 1;
0000000013F071040 movzx    r8d,cl
0000000013F071044 mov      edx,r9d
0000000013F071047 or       r8b,1
0000000013F07104B inc      r9d
0000000013F07104E inc      rcx
0000000013F071051 mov      byte ptr [rdx+rbx],r8b
0000000013F071055 cmp      rcx,rdi
0000000013F071058 jne      main+40h (013F071040h)
```

Теперь для переменной *index* используется 32-битный регистр *r9d*. Неопределённого поведения нет, так как переполнение беззнаковых переменных определено. Если переменная типа *unsigned* равна *UINT_MAX*, то её инкремент сделает значение равным нулю. В результате единицами заполнятся первые *UINT_MAX* элементов массива, а последний гигабайт по-прежнему будет содержать нули.

Вывод

Можно долго приводить неожиданные примеры того, как может проявиться неопределённое поведение. Про неопределённое поведение вообще можно отдельно целую книгу написать. Впрочем, это не имеет практического смысла. Неопределённое поведение на то и неопределённое, что его нельзя обстоятельно описать и систематизировать. Это всё что угодно, когда вы нарушаете правила программирования на языке C++.

Самое главное — не думайте, что можете предсказать, как именно будет вести себя код с неопределённым поведением, и обуздать его. Никогда не допускайте возникновения неопределённого поведения в вашей программе.

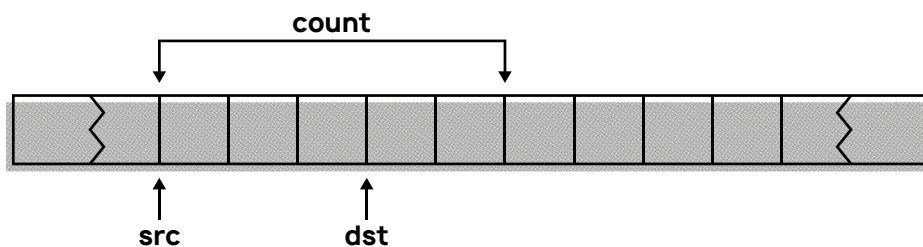
Вредный совет N14. memmove — лишняя функция

memmove — лишняя функция. Всегда и везде используйте memcpy.

```
void* memcpy ( void* dest, const void* src, std::size_t count );  
void* memmove( void* dest, const void* src, std::size_t count );
```

Функции копируют байты из буфера *src* в буфер *dest*. Аргумент *count* задаёт количество копируемых байт.

Назначение функций одинаково. Но есть важное отличие: когда области памяти, переданные через первые два параметра, частично перекрываются, *memmove* гарантирует, что результат копирования — правильный. В случае с *memcpy*, согласно документации, возникнет неопределённое поведение.



Предположим, что нужно сдвинуть пять байт памяти на три байта, как показано на картинке. Тогда:

- **memmove** — проблем с копированием перекрывающихся областей не возникнет, содержимое будет скопировано правильно;
- **memcpy** — возникнет проблема. Исходные значения двух байтов будут перезаписаны и не сохранены. Поэтому последние два байта последовательности будут точно такими же, что и два первых. Впрочем, это всего лишь один из возможных вариантов проявления неопределённого поведения.

Зачем нужны две функции для выполнения одинакового действия? Функция *memcpy* работает быстрее, по крайней мере, теоретически. Ей не нужно учитывать перекрывающиеся области памяти, выполняя дополнительные проверки и используя различные способы копирования данных.

Раз функции ведут себя по-разному, что стало поводом шутить на эту тему? Оказывается, авторы многих проектов невнимательно читали документацию. Дополнительно невнимательных программистов спасало и спасает то, что иногда *memcpy* реализована как *memmove*. Например, в старых версиях glibc функция *memcpy* была псевдонимом *memmove*.

В 2010 активно обсуждалась история, что выпуск новой версии библиотеки glibc 2.13 вызвал сбои в различных приложениях. Пользователи Fedora столкнулись с искажениями звука и другими странными багами. Причина оказалась в том, что разработчики glibc изменили реализацию *memcpy* на ту, которая в теории более оптимизирована для имеющихся процессоров (в том числе x86-64).

Это изменение спровоцировало появление ошибок в коде, где разработчики проигнорировали требование о том, что исходный массив и массив назначения, переданные *memcpy*, не должны перекрываться. Для решения предлагались различные «костыли». В glibc 2.14 был добавлен версионный символ, чтобы старые двоичные файлы (те, которые связаны с glibc более ранних версий, чем 2.14) использовали реализацию *memcpy*, которая безопасно обрабатывает случай перекрытия буферов.

Была большая дискуссия, является ли подобное изменение поведения функции допустимым с точки зрения совместимости и не является ли оно само ошибкой. Разработчики glibc не признали такое изменение поведения функции багом, и ответственность справедливо легла на неаккуратных разработчиков прикладных приложений.

Эта история канула в Лету. Однако по-прежнему не все программисты понимают разницу между *memcpy* и *memmove*, так

как на их системах функция *memcpy* может быть в данный момент алиасом для *memmove*. Как результат, они пишут ошибочный код, работоспособность которого может быть легко нарушена в любой момент при смене версии библиотек или при портировании кода на другую систему.

Вредный совет N15. Не проверяй, что вернула функция `malloc`

Нет смысла проверять, удалось ли выделить память. На современных компьютерах её много. А если не хватило, то и незачем дальше работать. Пусть программа упадёт. Всё равно уже больше ничего сделать нельзя.

Упасть, если кончится память, допустимо игре. Это неприятно, но некритично. Если, конечно, в этот момент вы не участвуете в игровом чемпионате.

Некоторые разработчики пренебрежительно относятся к проверкам: удалось ли выделить память с помощью функции `malloc` или нет. Их логика проста — памяти всегда должно хватать. Если не хватает, всё равно уже ничего не сделаешь, и пусть программа упадёт при разыменовании нулевого указателя. Очень плохой подход по целому ряду причин.

Примечание. Далее под функцией `malloc` будет подразумеваться не только эта функция, но и `calloc`, `realloc`, `_aligned_malloc`, `_recalloc`, `strdup` и так далее. Не хочется загромождать текст, постоянно повторяя названия всех этих функций. Общее у них то, что они могут вернуть нулевой указатель.

Я помню, что в названии книги написано «C++», а `malloc` — это функция из мира C. Однако по ряду причин эта функция живее всех живых в мире C++. Так что будет интересно и полезно разобрать эту тему.

malloc

Если функция `malloc` не смогла выделить буфер памяти, то она возвращает `NULL`. Программа должна проверять указатели, которые возвращает функция `malloc`, и соответствующим образом обрабатывать ситуацию, когда память выделить не получилось.

К сожалению, многие программисты небрежно относятся к проверке указателей, а иногда сознательно не проверяют, удалось выделить память или нет. Их логика следующая.

Если функция *malloc* не смогла выделить память, то вряд ли программа продолжит функционировать должным образом. Скорее всего, памяти будет не хватать и для других операций, поэтому можно вообще не заморачиваться об ошибках выделения памяти. Первое же обращение к памяти по нулевому указателю приведёт к генерации Structured Exception в Windows, или процесс получит сигнал *SIGSEGV*, если речь идёт о Unix-подобных системах. В результате программа упадёт, что вполне приемлемо. Раз нет памяти, то и нечего мучиться. Как вариант, можно перехватить структурное исключение/сигнал и обрабатывать разыменовывания нулевого указателя более централизованно. Это удобнее, чем писать тысячи проверок.

Я не раз общался с людьми, которые считают такой подход уместным и сознательно никогда не проверяют результат, который возвращает функция *malloc*.

Кстати, существует ещё одно оправдание разработчиков в отсутствии проверок. Функция *malloc* только резервирует память, но вовсе нет гарантии, что хватит физической памяти, когда мы начнём использовать выделенный буфер. Поэтому раз всё равно гарантии нет, то и проверять не надо. Carsten Haitzler, являющийся одним из разработчиков библиотеки EFL Core, объяснял этим, почему я насчитал более 500 мест в коде библиотеки, где отсутствуют проверки. Вот его комментарий к моей статье о проверке этого проекта:

Общепринятое мнение — как минимум на Linux (на котором мы фокусируемся уже долгое время), что нельзя доверять тому, что вернули функции *malloc/calloc/realloc*. Linux по умолчанию осуществляет *overcommit* [см. термин «Overcommit»]. То есть вы получаете новую память,

но ядро ещё не присвоило ей реальных физических страниц памяти. Только присвоило виртуальное пространство, и так будет до тех пор, пока вы не начнёте взаимодействовать с памятью. Если ядро не может обслужить этот запрос, ваша программа упадёт, пытаясь получить доступ к памяти по, казалось бы, надёжному указателю.

Так что, в общем, ценность проверки возвратов небольших аллоков невелика, по крайней мере, в Linux. Иногда мы проверяем... иногда нет. Но возвратам в принципе доверять нельзя, за исключением случаев, когда у вас огромные объёмы памяти, а ваш аллок никогда обработан не будет — т. е. ваш аллок не сможет поместиться в виртуальное адресное пространство в принципе (иногда такое случается на 32-битных системах). Да, `overcommit` можно настроить, но у этого есть своя цена, которую многие программисты платить отказываются. А некоторые и не знают, что `overcommit` можно настроить.

Также если аллокацию для небольшого участка памяти (к примеру, для узла связанного списка) выполнить не удаётся... если вам при аллокации возвращается `null`... то падение, возможно, будет лучшим из всего, что может произойти. Если памяти у вас настолько мало, что программа может упасть, зовите `abort()` (как `g_malloc` в `glib`). Если вы не можете выделить 20—40 байтов, ваша система упадёт в любом случае — просто потому, что у вас не останется рабочей памяти. Здесь я говорю не о маленьких `embedded` системах, а о больших машинах с виртуальной памятью и несколькими гигабайтами памяти, и т. д. — они были и являются нашей целью.

Подобные рассуждения программистов являются неправильными, и ниже я подробно объясню почему.

Есть пять причин, чтобы обязательно делать проверки после вызова функции `malloc`. Если кто-то в команде не пишет проверки, то обязательно заставьте его прочитать эту главу.

Защищённые страницы памяти

Прежде чем начать, небольшая теоретическая справка, почему возникают структурные исключения (SEH в Windows) или сигналы (в UNIX-подобных системах), если происходит разыменовывание нулевого указателя. Это важно для дальнейшего повествования.

В начале адресного пространства одна или несколько страниц памяти защищены операционной системой от записи. Это позволяет выявлять ошибки обращения к памяти по нулевому указателю или указателю, значение которого близко к 0.

В разных операционных системах для этих целей резервируется разное количество памяти. При этом в некоторых ОС это значение можно настраивать. Поэтому нет смысла называть какое-то конкретное число зарезервированных байт памяти. Но, чтобы как-то сориентировать читателя, скажу, что в Linux-системах типовым значением является 64 Кбайта.

Важно, что, прибавив к нулевому указателю достаточно большое число, можно «промазать» мимо контрольных страниц памяти и случайно попасть в какие-то незащищённые от записи страницы. Таким образом можно испортить данные, но операционная система этого не заметит, и никакого сигнала/исключения она не сгенерирует.

Примечание. Если говорить про встраиваемые системы, то там вообще может не быть никакой защиты от записи по нулевому адресу. В некоторых системах мало памяти, и вся она используется для хранения данных. Впрочем, в системах с малым количеством RAM, скорее всего, не будет динамического управления памятью и, соответственно, функции `malloc`.

Причина 1: разыменовывание нулевого указателя — это неопределённое поведение

С точки зрения языков C и C++ разыменовывание нулевого указателя приводит к неопределённому поведению. Неопределённое поведение — это что угодно. Не думайте, что вы зна-

ете, как будет вести себя программа, если произойдёт разыменовывание `nullptr`. В этом вы уже могли убедиться в главе N13.

Не надейтесь, что сможете совладать с разыменовыванием нулевого указателя, используя обработчики структурных исключений или сигналы. Раз разыменовывание нулевого указателя было, то работа программы уже нарушена, и может произойти что угодно. Давайте рассмотрим пример, почему нельзя полагаться на SEH-обработчики и т. п.

```
size_t *ptr = (size_t *)malloc(sizeof(size_t) * N * 2);
for (size_t i = 0; i != N; ++i)
{
    ptr[i] = i;
    ptr[N * 2 - i - 1] = i;
}
```

Этот код заполняет массив от краёв к центру. К центру значения элементов увеличиваются. Это придуманный за 1 минуту пример, поэтому не гадайте, зачем такой массив кому-то нужен. Я и сам не знаю. Мне было важно, чтобы в соседних строках программы происходила запись в начало массива и куда-то в его конец. Такое иногда бывает нужно и в практических задачах, и мы рассмотрим реальный код, когда доберёмся до 4-й причины.

Ещё раз внимательно посмотрим на эти две строки:

```
ptr[i] = i;
ptr[N * 2 - i - 1] = i;
```

С точки зрения программиста, в начале цикла произойдёт запись в элемент `ptr[0]` и возникнет структурное исключение/сигнал. Оно будет обработано, и всё будет хорошо.

Однако компилятор в целях оптимизации может переставить присваивания местами. Он имеет на это полное право. С точки зрения компилятора, если указатель разыменовывается, то он не может быть равен `nullptr`. Если указатель нулевой, то это

неопределённое поведение, и компилятор не обязан думать о последствиях оптимизации.

Так вот, компилятор может решить, что в целях оптимизации выгоднее выполнить присваивания так:

```
ptr[N * 2 - i - 1] = i;
```

```
ptr[i] = i;
```

В результате вначале произойдёт запись по адресу $((size_t *) nullptr)[N * 2 - 0 - 1]$. Если значение N достаточно велико, то страница защиты в начале памяти будет «перепрыгнута» и значение переменной i может быть записано в какую-то ячейку, доступную для записи. В общем, произойдёт порча каких-то данных.

И только после этого будет выполнено присваивание по адресу $((size_t *) nullptr)[0]$. Операционная система заметит попытку записи в контролируемую ею область и сгенерирует сигнал/исключение.

Программа может обработать это структурное исключение/сигнал. Но уже поздно. Где-то в памяти появились испорченные данные. Причём непонятно, какие данные испорчены и к каким последствиям это может привести!

Виноват ли компилятор, что поменял операции присваивания местами? Нет. Программист допустил разыменовывание нулевого указателя и тем самым ввёл программу в состояние неопределённого поведения. В данном конкретном случае неопределённое поведение программы будет заключаться в том, что где-то в памяти испорчены данные.

Вывод

Исходите из аксиомы: любое разыменовывание нулевого указателя — это неопределённое поведение программы. Не бывает «безобидного» неопределённого поведения. Любое неопределённое поведение недопустимо. Не допускайте разыменовывания указателей, которые верну-

ла функция *malloc* и её аналоги, без их предварительной проверки. Не полагайтесь на какие-то другие способы перехвата разыменовывания нулевого указателя. Следует использовать только старый добрый оператор *if*.

Причина 2: разыменовывание нулевого указателя — это уязвимость

То, что некоторые разработчики вообще не считают за ошибку, другие воспринимают как уязвимость. Именно так обстоит дело с разыменовыванием нулевого указателя.

В ряде проектов считается допустимым, если программа из-за разыменовывания нулевого указателя упадёт или если ошибка будет обработана каким-то общим способом с помощью перехвата сигнала/структурного исключения.

В других приложениях разыменовывание нулевого указателя интерпретируется как разновидность потенциальной уязвимости, которую можно использовать для DoS-атаки уровня приложения [см. термин «DoS-атака»]. Вместо того чтобы штатно обработать нехватку памяти, программа или один из потоков исполнения завершает свою работу. Это может приводить к потере данных, нарушению их целостности и так далее.

Приведу пример. Программа *Ytnef* предназначена для декодирования TNEF потоков, например, созданных в Outlook. Отсутствие проверки после вызова *calloc* в ней было идентифицировано как уязвимость CVE-2017-6298.

Все исправленные места, в которых могло произойти разыменовывание нулевого указателя, имели приблизительно один и тот же вид:

```
vl->data = calloc(vl->size, sizeof(WORD));
temp_word = SwapWord((BYTE*)d, sizeof(WORD));
memcpy(vl->data, &temp_word, vl->size);
```

Выводы

Если вы разрабатываете безответственное приложение, для которого упасть в процессе работы не является бедой, то да, писать проверки необязательно.

Однако если вы разрабатываете настоящий программный продукт или библиотеку, то отсутствие проверок недопустимо!

Поэтому я идеологически не согласен, например, с аргументацией Carsten Haitzler о допустимости отсутствия проверок в библиотеке EFL Core. Данный подход не позволяет построить на основе таких библиотек надёжные приложения. Если вы создаёте библиотеку, то помните, что в некоторых приложениях разыменовывание нулевого указателя — это уязвимость. Необходимо обрабатывать ошибки выделения памяти и штатно возвращать информацию о неудаче.

Причина 3: нет гарантии, что будет разыменовыван именно нулевой указатель

Те, кто ленится писать проверки, почему-то думают, что разыменовывание затрагивает именно нулевые указатели. Да, часто именно так и бывает. Но может ли поручиться программист за код всего приложения? Уверен, что нет.

Сейчас я на практических примерах покажу, что я имею в виду. Возьмём, например, код из библиотеки LLVM-subzero, которая используется в Chromium:

```
void StringMapImpl::init(unsigned InitSize) {
    assert((InitSize & (InitSize-1)) == 0 &&
           "Init Size must be a power of 2 or zero!");
    NumBuckets = InitSize ? InitSize : 16;
    NumItems = 0;
    NumTombstones = 0;

    TheTable = (StringMapEntryBase **)
        calloc(NumBuckets+1,
              sizeof(StringMapEntryBase **)) +
```

```
        sizeof(unsigned));

    // Allocate one extra bucket, set it to look filled
    // so the iterators stop at end.
    TheTable[NumBuckets] = (StringMapEntryBase*)2;
}
```

Предупреждение PVS-Studio: V522 [CWE-690] There might be dereferencing of a potential null pointer 'TheTable'. Check lines: 65, 59. stringmap.cpp 65

Сразу после выделения буфера памяти происходит запись в ячейку *TheTable[NumBuckets]*. Если значение переменной *NumBuckets* достаточно большое, то мы испортим какие-то данные с непредсказуемыми последствиями. После такой порчи вообще нет смысла рассуждать, как будет работать программа. Могут последовать самые неожиданные последствия.

Пример аналогичного дефекта можно наблюдать и в уже упомянутой библиотеке EFL:

```
static void
st_collections_group_parts_part_description_filter_data(void)
{
    ....
    filter->data_count++;
    array = realloc(filter->data,
        sizeof(Edge_Part_Description_Spec_Filter_Data) *
        filter->data_count);
    array[filter->data_count - 1].name = name;
    array[filter->data_count - 1].value = value;
    filter->data = array;
}
```

Предупреждение PVS-Studio:
V522 [CWE-690] There might be dereferencing of a potential

null pointer 'array'. `edje_cc_handlers.c` 14249

Перед нами типовая ситуация: в буфере не хватает свободного места для хранения данных, и его следует увеличить. Для увеличения размера буфера используется функция *realloc*, которая может вернуть *NULL*. При этом вовсе не обязательно возникнет структурное исключение/сигнал из-за разыменовывания нулевого указателя. Взглянем вот на эти строчки:

```
array[filter->data_count - 1].name = name;  
array[filter->data_count - 1].value = value;
```

Если значение переменной *filter->data_count* достаточно большое, то данные будут записаны по какому-то непонятному адресу.

В памяти будут испорчены какие-то данные, а программа продолжит своё выполнение. Последствия непредсказуемы, но ничего хорошего точно не получится.

Вывод

Вопрос: «Где гарантии, что будет разыменовывание именно нулевого указателя?». Нет таких гарантий. Невозможно, разрабатывая или модифицируя код, помнить про только что рассмотренный нюанс. Запросто можно что-то испортить в памяти, при этом программа продолжит выполняться как ни в чём не бывало.

Единственный способ написать надёжный и правильный код — это всегда проверять результат, который вернула функция *malloc*. Проверьте и живите спокойно.

Причина 4: нет гарантии, что *memset* заполняет память в прямом порядке

Найдётся кто-то, кто скажет что-то подобное:

Я отлично понимаю про *realloc* и всё остальное, что написано в статье. Но я профессионал и, выделяя память, сразу заполняю её нулями с помощью *memset*. Там, где

| действительно необходимо, я использую проверки. Но лишние проверки после каждого *malloc* я писать не буду.

Вообще заполнять память сразу после выделения буфера достаточно странная идея. Странная потому, что есть функция *calloc*. Тем не менее так поступают очень часто. Далеко за примером ходить не надо, вот код из библиотеки WebRTC:

```
int Resampler::Reset(int inFreq, int outFreq, size_t num_channels) {  
    ....  
    state1_ = malloc(8 * sizeof(int32_t));  
    memset(state1_, 0, 8 * sizeof(int32_t));  
    ....  
}
```

Выделяется память, затем буфер заполняется нулями. Частая практика, хотя две строчки можно сократить до одной, используя *calloc*. Но всё это неважно.

Главное, что даже подобный код небезопасен! Функция *memset* не обязана начинать заполнять память с начала и тем самым вызывать разыменовывание нулевого указателя.

Функция *memset* имеет право начать заполнять буфер с конца. И если выделялся большой буфер, то могут быть затёрты какие-то полезные данные. Да, заполняя память, функция *memset* рано или поздно достигнет страницы, защищённой от записи, и операционная система сгенерирует структурное исключение/сигнал. Однако обрабатывать их уже не имеет смысла. К этому моменту будет испорчен большой фрагмент памяти, и дальнейшая работа программы будет непредсказуема.

Читатель может возразить, что всё это носит исключительно теоретический характер. Да, функция *memset* чисто теоретически может заполнять буфер, начиная с конца, но на практике никто не будет так реализовывать эту функцию.

Соглашусь, что подобная реализация *memset* – действительно экзотика, и я даже задавал вопрос на Stack Overflow на эту

тему. В ответе говорится:

The Linux kernel's memset for the SuperH architecture has this property:

```
/*
 * void *memset(void *s, int c, size_t n);
 */

#include <linux/linkage.h>

ENTRY(memset)
    tst        r6,r6
    bt/s       5f          ! if n=0, do nothing
    add        r6,r4
    mov        #12,r0
    cmp/gt     r6,r0
    bt/s       4f          ! if it's too small,
                          ! set a byte at once

    mov        r4,r0
    and        #3,r0
    cmp/eq     #0,r0
    bt/s       2f          ! It's aligned
    sub        r0,r6

1:    dt        r0
    bf/s       1b
    mov.b      r5,@-r4

2:    ! make VVVV
    extu.b     r5,r5
    swap.b     r5,r0      !  V0
    or         r0,r5      !  VV
    swap.w     r5,r0      ! VV00
    or         r0,r5      ! VVVV
    !
    mov        r6,r0
    shlr2      r0
    shlr       r0        ! r0 = r6 >> 3

3:    dt        r0
    mov.l      r5,@-r4    ! set 8-byte at once
    bf/s       3b
    mov.l      r5,@-r4
    !
    mov        #7,r0
    and        r0,r6
    tst        r6,r6
    bt         5f
    ! fill bytes

4:    dt        r6
    bf/s       4b
    mov.b      r5,@-r4

5:    rts
```

```
mov      r4,r0
```

К сожалению, это код на незнакомой мне разновидности ассемблера, поэтому я не берусь рассуждать о нём. Зато есть ещё одна интересная реализация на языке Си. Приведу начало этой функции:

```
void *memset(void *dest, int c, size_t n)
{
    unsigned char *s = dest;
    size_t k;
    if (!n) return dest;
    s[0] = c;
    s[n-1] = c;
    ....
}
```

Обратите внимание на эти строки кода:

```
s[0] = c;
s[n-1] = c;
```

Здесь мы возвращаемся к причине N1 «Разыменовывание нулевого указателя — это неопределённое поведение». Нет гарантии, что компилятор в целях оптимизации не поменяет присваивания местами. Если компилятор это сделает и аргумент *n* будет иметь большое значение, то вначале будет испорчен какой-то байт памяти. И только потом произойдёт разыменовывание нулевого указателя.

Опять неубедительно? Хорошо, а как вам вот такая реализация?

```
void *memset(void *dest, int c, size_t n)
{
    size_t k;
    if (!n) return dest;
    s[0] = s[n-1] = c;
```

```
    if (n <= 2) return dest;  
    ....  
}
```

Вывод

Нельзя доверять даже функции *memset*. Да, это во многом искусственная и надуманная проблема. Я просто хотел показать, как много существует нюансов, связанных с разыменованием нулевых указателей. Просто невозможно всё это учесть. Следует аккуратно проверять каждый указатель, который вернула функция *malloc* и аналогичные ей. Именно тогда вы станете профессионалом и будете писать надёжный код.

Причина 5: функция *malloc* действительно возвращает *NULL*

Допускаю, что кто-то может посчитать, что перечисленные аргументы не совсем относятся к делу. Мне говорят «функция всё равно на практике не вернёт *NULL*», а я объясняю, что если функция вернула *NULL*, то его нельзя разыменовывать, и поэтому требуется писать проверки.

Ранее мы рассматривали термин *overcommit* и сейчас вновь вернёмся к нему. Использование *overcommit* не означает, что операционная система всегда возвращает ненулевой указатель. Это частный случай поведения, и не стоит писать код исходя из него.

Код, рассчитанный на то, что *malloc* в любом случае не вернёт *NULL*, ненадёжен по причинам:

1. Поведение *overcommit* можно настраивать. Пользователь может запускать приложение вовсе не в том режиме, на который рассчитывал программист.
2. Код может быть заимствован для другого приложения, работающего в другой операционной системе, где другие правила.
3. Если вы разрабатываете код для встраиваемых систем

или код впоследствии может быть использован в таких системах, то там нехватка памяти — совершенно естественная ситуация.

4. Память может быть не выделена по причине её сильной фрагментации. Это вполне реальный сценарий при длительной работе приложения в 32-битной системе. Т. е. не будет найдено ни одного непрерывного участка памяти достаточной длины, и функция `malloc` вернёт `NULL`, хотя суммарно свободной памяти может быть ещё много.

Закончу этот раздел комментарием к одной из моих статей про `malloc`, который мне понравился и который хорошо суммирует мои мысли.

По опыту обсуждения подобной темы складывается ощущение, что в «Интернетах» есть две секты. Приверженцы первой свято уверены в том, что под Linux-ом `malloc` никогда не возвращает `NULL`. Приверженцы второй свято уверены в том, что если память в программе выделить не удалось, то ничего уже в принципе сделать нельзя, нужно только падать. Переубедить их никак нельзя. Особенно когда эти две секты пересекаются. Можно только принять это как данность. Причём не суть важно, на каком профильном ресурсе идёт обсуждение. [Евгений Охотников, @eao197 на сайте habr.com]

Будем надеяться, что эти группы разработчиков пишут только не критичные программы. Если, например, какие-то данные испортятся в игре или игра упадёт, это не страшно. Важно, чтобы так не делали разработчики библиотек, баз данных и т. д.

Вывод

Функция `malloc` возвращает `NULL`, и это не экзотика, а вполне естественная ситуация. Учитывайте её при разработке кода.

Примечания

По мотивам обсуждений моих статей на тему `malloc` я знаю,

какие недопонимания могут возникнуть, поэтому ещё несколько комментариев по этой теме.

1. Если `malloc` вернула `NULL`, то лучше сразу завершить работу программы, чем писать кучу `if`-ов и пытаться как-то обработать нехватку памяти, из-за которой часто выполнение программы всё равно невозможно.

Я вовсе не призывал до последнего бороться с последствиями нехватки памяти, пробрасывая ошибку всё выше и выше. Если для приложения допустимо завершить работу без предупреждения, то пусть так и будет. Для этого достаточно всего одной проверки сразу после `malloc` или использования `xmalloc` (см. следующий пункт).

Я возражал и предупреждал об отсутствии проверок, из-за которых программа продолжает работать «как ни в чём не бывало». Это другое. Это опасно, так как приводит к неопределённому поведению, порче данных и так далее.

2. Не рассказано про решение, которое заключается в написании функций-вращивов для выделения памяти с последующей проверкой, или использование уже существующих функций, таких как `xmalloc`.

Мне было важнее донести до читателя, в чём опасность отсутствия проверки. Как исправить код, это уже вопрос вкуса и деталей реализации.

Функция `xmalloc` не является частью стандартной библиотеки C. Однако она может быть объявлена в других библиотеках, например в GNU utils library.

Суть функции в том, что программа завершает работу, если не удалось выделить память. Реализация этой функции может выглядеть, например, так:

```
void* xmalloc(size_t s)
{
    void* p = malloc(s);
    if (!p) {
```

```
    fprintf(stderr, "fatal: out of memory (xmalloc(%zu)).\n", s);  
    exit(EXIT_FAILURE);  
}  
return p;  
}
```

Соответственно, вызывая везде функцию *xmalloc* вместо *malloc*, можно быть уверенным, что в программе не возникнет неопределённого поведения из-за какого-либо использования нулевого указателя.

К сожалению, *xmalloc* тоже не является панацеей от всех бед. Надо помнить, что использование *xmalloc* недопустимо, если речь идёт о написании кода библиотек. Про это я писал в главе N10.

Воззвание к разработчикам библиотек и ответственного кода

Если вы разрабатываете библиотеку или иной ответственный код, то всегда проверяйте значение указателя, который вернула функция *malloc*, и возвращайте ввне код ошибки, если память выделить не удалось.

В библиотеках нельзя вызвать функцию *exit*, если не удалось выделить память. По этой же причине нельзя использовать *xmalloc*. Для многих приложений недопустимо просто взять и аварийно завершить работу. Из-за этого, например, может быть испорчена база данных или проект, над которым человек работал многие часы. Могут быть потеряны данные, которые считались много часов. Из-за этого программа может быть подвержена уязвимостям «отказ в обслуживании», когда вместо того, чтобы как-то корректно обработать возрастающую нагрузку, работа многопоточного приложения просто завершается.

Нельзя предположить, как и в каких проектах будет использована библиотека. Поэтому следует исходить из того, что приложение может решать очень ответственные задачи. И просто

«убить» его, вызвав `exit`, — никуда не годится. Скорее всего, такая программа написана с учётом возможности нехватки памяти и может что-то предпринять в этом случае. Например, некая CAD-система из-за сильной фрагментации памяти не может выделить достаточный буфер для очередной операции. Но это не повод ей завершиться в аварийном режиме с потерей данных. Программа может дать возможность сохранить проект и перезапустить себя в штатном режиме.

Ни в коем случае нельзя полагаться на то, что `malloc` всегда может выделить память. Неизвестно, на какой платформе и как будет использоваться библиотека. Если на одной платформе нехватка памяти — это экзотика, то на другой — это может быть весьма частой ситуацией.

Нельзя надеяться, что программа упадёт, если `malloc` вернёт `NULL`. Произойти может всё что угодно. Программа может писать данные вовсе не по нулевому адресу. В результате могут быть испорчены некие данные, что приведёт к непредсказуемым последствиям. Даже `memset` опасен. Если заполнение данных идёт в обратном порядке, то вначале испортятся некие данные, и только потом программа упадёт. Но падение может произойти слишком поздно. Если в момент работы функции `memset` в параллельных потоках будут использованы испорченные данные, то последствия могут быть фатальны. Можно получить испорченную транзакцию в базе данных или отправку команды на удаление случайных файлов. Предлагаю читателю пофантазировать самостоятельно, к чему может привести использование мусора в памяти.

Таким образом, у библиотеки есть только один правильный вариант работы с функциями `malloc`. Нужно сразу проверить, что вернула функция, и если это `NULL`, то вернуть статус ошибки.

Подытожим

Всегда сразу проверяйте указатель, который вернула функция `malloc` или аналогичная ей.

Надеюсь, теперь вы по-новому взглянете на функцию *malloc*. А также с большим пониманием отнесётесь к предупреждениям PVS-Studio или другого статического анализатора, который предупредит вас об отсутствии проверки указателя. Такая проверка не прихоть и не педантизм, а реальная необходимость.

Вредный совет N16. Расширяй пространство `std`

Добавляйте разные вспомогательные функции и классы в пространство имён `std`. Ведь для вас эти функции и классы стандартные, а значит, им самое место в `std`.

Несмотря на то, что код успешно компилируется и исполняется, модификация пространства имён `std` может привести к неопределённому поведению программы. То же самое касается и пространства `posix`.

Содержимое пространства имён `std` определяется исключительно комитетом стандартизации. Стандарт запрещает добавлять в него:

- декларации переменных;
- декларации функций;
- декларации классов/структур/объединений;
- декларации перечислений;
- декларации шаблонов функций, классов и переменных (C++14).

Стандарт разрешает добавлять следующие специализации шаблонов, определённых в пространстве имен `std`, если они зависят хотя бы от одного определённого в программе типа (program-defined type):

- полная или частичная специализация шаблона класса;
- полная специализация шаблона функции (до C++20);
- полная или частичная специализация шаблона переменной, не лежащей в заголовочном файле `<type_traits>` (до C++20).

Однако специализации шаблонов, лежащих внутри классов или шаблонов классов, запрещены.

Наиболее частым сценарием, когда пользователь расширяет пространство имен `std`, является добавление своей перегруз-

ки функции `std::swap` и полной или частичной специализации шаблона класса `std::hash`.

Рассмотрим неправильный фрагмент кода с добавлением перегрузки `std::swap`:

```
template <typename T>
class MyTemplateClass
{
    ....
};

class MyClass
{
    ....
};

namespace std
{
    template <typename T>
    void
    swap(MyTemplateClass<T> &a, MyTemplateClass<T> &b) noexcept // UB
    {
        ....
    }

    template <>
    void
    swap(MyClass &a, MyClass &b) noexcept // UB since C++20
    {
        ....
    };
}
```

Первый шаблон функции не является специализацией *std::swap*, и такая декларация ведёт к неопределённому поведению. Второй шаблон функции является специализацией, и до C++20 поведение программы определено. Однако в данном случае можно поступить иначе: можно вынести обе функции из пространства имен *std* и поместить их в то пространство имен, где определены классы:

```
template <typename T>
class MyTemplateClass
{
    ....
};

class MyClass
{
    ....
};

template <typename T>
void swap(MyTemplateClass<T> &a, MyTemplateClass<T> &b) noexcept
{
    ....
}

void swap(MyClass &a, MyClass &b) noexcept
{
    ....
};
```

Теперь, когда необходимо написать шаблон функции, который применяет функцию *swap* для двух объектов типа *T*, можно написать следующий код:

```
template <typename T>
void MyFunction(T& obj1, T& obj2)
{
    using std::swap; // make std::swap visible for overload resolution
    ....
    swap(obj1, obj2); // best match of 'swap' for objects of type T
    ....
}
```

Компилятор выберет нужную перегрузку функции на основе поиска с учётом аргументов (argument-dependent lookup, ADL): пользовательские функции *swap* для класса *MyClass* и для шаблона класса *MyTemplateClass* и стандартную версию *std::swap* для остальных типов.

Разберём следующий пример со специализацией шаблона класса *std::hash*:

```
namespace Foo
{
    class Bar
    {
        ....
    };
}

namespace std
{
    template <>
    struct hash<Foo::Bar>
    {
        size_t operator()(const Foo::Bar &) const noexcept;
    };
}
```

С точки зрения стандарта этот код является валидным. Однако, начиная с C++11, можно и в этом случае поступить иначе, написав специализацию шаблона класса за пределами пространства имен *std*:

```
template <>
struct std::hash<Foo::Bar>
{
    size_t operator()(const Foo::Bar &) const noexcept;
};
```

В отличие от пространства имён *std*, стандарт C++ запрещает абсолютно любую модификацию пространства имён *posix*.

Вредный совет N17. Старая школа

Коллеги должны знать о вашем богатом опыте с языком С. Не стесняйтесь демонстрировать им в вашем С++ проекте свои умелые навыки ручного управления памятью и `longjmp`.

Другая вариация этого вредного совета: **умные указатели и прочее RAII от лукавого, всеми ресурсами надо управлять вручную, это делает код простым и понятным.**

Нет смысла отказываться от умных указателей и городить сложные конструкции при работе с памятью. Умные указатели в С++ не требуют дополнительного процессорного времени, это не сборка мусора. При этом код с использованием умных указателей становится короче и проще, что дополнительно снижает вероятность допустить ошибку.

Давайте рассмотрим, почему ручное управление памятью — это муторно и ненадёжно. Начнём с простейшего кода на С, где выделяется и освобождается память.

Примечание. Я рассматриваю в примерах выделение и освобождение памяти. На самом деле, это более широкая тема ручного управления ресурсами. Вместо *malloc* вполне можно подставить, например, *foopen*.

```
int Foo()
{
    float *buf = (float *)malloc(ARRAY_SIZE * sizeof(float));
    if (buf == NULL)
        return STATUS_ERROR_ALLOCATE;
    int status = Go(buf);
    free(buf);
    return status;
}
```

Этот код прост и понятен. Функция выделяет память для каких-то нужд, использует её и затем освобождает. Дополнительно приходится проверять, смогла ли функция *malloc* выделить память. Почему эта проверка обязательно необходима, мы разбирали в главе N15.

Теперь представим, что нам требуется выполнить операции с двумя разными буферами. Код сразу начинает пухнуть, так как при невозможности в очередной раз выделить память нужно позаботиться о предыдущем буфере. Плюс теперь нужно учитывать, что вернула функция *Go_1*.

```
int Foo()
{
    float *buf_1 = (float *)malloc(ARRAY_SIZE_1 * sizeof(float));
    if (buf_1 == NULL)
        return STATUS_ERROR_ALLOCATE;
    int status = Go_1(buf_1);
    if (status != STATUS_OK)
    {
        free(buf_1);
        return status;
    }
    float *buf_2 = (float *)malloc(ARRAY_SIZE_2 * sizeof(float));
    if (buf_2 == NULL)
    {
        free(buf_1);
        return STATUS_ERROR_ALLOCATE;
    }
    status = Go_2(buf_1, buf_2);
    free(buf_1);
    free(buf_2);
    return status;
}
```

Дальше — хуже. Размер кода растёт нелинейно. При трёх буферах:

```
int Foo()
{
    float *buf_1 = (float *)malloc(ARRAY_SIZE_1 * sizeof(float));
    if (buf_1 == NULL)
        return STATUS_ERROR_ALLOCATE;
    int status = Go_1(buf_1);
    if (status != STATUS_OK)
    {
        free(buf_1);
        return status;
    }
    float *buf_2 = (float *)malloc(ARRAY_SIZE_2 * sizeof(float));
    if (buf_2 == NULL)
    {
        free(buf_1);
        return STATUS_ERROR_ALLOCATE;
    }
    status = Go_2(buf_1, buf_2);
    if (status != STATUS_OK)
    {
        free(buf_1);
        free(buf_2);
        return status;
    }
    float *buf_3 = (float *)malloc(ARRAY_SIZE_3 * sizeof(float));
    if (buf_3 == NULL)
    {
        free(buf_1);
        free(buf_2);
        return STATUS_ERROR_ALLOCATE;
    }
    status = Go_3(buf_1, buf_2, buf_3);
    free(buf_1);
    free(buf_2);
    free(buf_3);
    return status;
}
```

Что интересно, сложность кода по-прежнему низкая. Его легко писать и читать. Но вместе с тем чувствуется, что это какой-то неправильный путь.

Больше половины кода в функции относится к проверке статусов и выделению/освобождению памяти. Полезного кода меньше, чем второстепенного. Вот этим и плохо ручное управление памятью. Создаётся много, хоть и нужного, но не относящегося к делу кода.

И хотя код, как я сказал, несложен, с ростом его размера всё проще допустить ошибку. Например, можно при досрочном выходе из функции забыть освободить какой-то указатель и получить утечку памяти. И такое я действительно встречал в коде различных проектов, когда проверяю их с помощью PVS-Studio. Вот, например, фрагмент кода из Persistent Memory Development Kit (PMDK):

```
static enum pocli_ret
pocli_args_obj_root(struct pocli_ctx *ctx, char *in, PMEMoid **oidp)
{
    char *input = strdup(in);
    if (!input)
        return POCLI_ERR_MALLOC;

    if (!oidp)
        return POCLI_ERR_PARS;
    ....
}
```

Функция *strdup* создаёт копию строки в буфере, который затем должен быть где-то освобождён с помощью функции *free*. Здесь же в случае, если аргумент *oidp* является нулевым указателем, произойдёт утечка памяти. Корректный код должен быть таким:

```
char *input = strdup(in);
if (!input)
```

```
    return POCLI_ERR_MALLOC;

if (!oidp)
{
    free(input);
    return POCLI_ERR_PARS;
}
```

Или нужно перенести проверку аргумента в начало функции:

```
if (!oidp)
    return POCLI_ERR_PARS;

char *input = strdup(in);
if (!input)
    return POCLI_ERR_MALLOC;
```

В любом случае перед нами классическая ошибка в коде с ручным управлением памятью.

Вернёмся к нашему синтетическому коду с тремя буферами. Можно как-то упростить код? Да, для этого используется паттерн с одной точкой выхода и операторами *goto*.

```
int Foo()
{
    float *buf_1 = NULL;
    float *buf_2 = NULL;
    float *buf_3 = NULL;
    int status;

    buf_1 = (float *)malloc(ARRAY_SIZE_1 * sizeof(float));
    if (buf_1 == NULL)
    {
        status = STATUS_ERROR_ALLOCATE;
        goto end;
    }

    status = Go_1(buf_1);
    if (status != STATUS_OK)
        goto end;
    buf_2 = (float *)malloc(ARRAY_SIZE_2 * sizeof(float));
```

```
if (buf_2 == NULL)
{
    status = STATUS_ERROR_ALLOCATE;
    goto end;
}
status = Go_2(buf_1, buf_2);
if (status != STATUS_OK)
{
    status = STATUS_ERROR_ALLOCATE;
    goto end;
}
buf_3 = (float *)malloc(ARRAY_SIZE_3 * sizeof(float));
if (buf_3 == NULL)
{
    status = STATUS_ERROR_ALLOCATE;
    goto end;
}
status = Go_3(buf_1, buf_2, buf_3);
end:
free(buf_1);
free(buf_2);
free(buf_3);
return status;
}
```

Так часто пишут программисты на С. Код стал немного проще и короче. Я не могу назвать такой код хорошим и красивым, но что поделать. Ручное управление ресурсами в любом случае страшненькое...

Кстати, некоторые компиляторы поддерживают специальное расширение языка С, которое упрощает жизнь. Например, если для сборки кода вы используете GCC, то вам доступны конструкции следующего вида:

```
void free_buf(int **p) {
    free(*p);
}

int main(void) {
    __attribute__((cleanup (free_buf))) int *buf = malloc(sizeof(int));
```

```
*buf = 123;  
} // Когда закончится область видимости переменной buf,  
    // вызовется функция free_buf и освободит память.
```

Вернёмся к вредному совету. Беда в том, что некоторые программисты продолжают использовать этот стиль ручного управления ресурсами в C++ коде, хотя в этом нет никакого смысла! Не делайте так. C++ позволяет сделать код простым и коротким.

Можно использовать контейнеры, такие как `std::vector`. Но, даже если нам нужен именно массив байт, выделенный с помощью оператора `new[]`, всё равно код можно сделать на порядок лучше.

```
int Foo()  
{  
    std::unique_ptr<float[]> buf_1 (new float[ARRAY_SIZE_1]);  
    if (int status = Go_1(buf_1); status != STATUS_OK)  
        return status;  
  
    std::unique_ptr<float[]> buf_2(new float[ARRAY_SIZE_2]);  
    if (int status = Go_2(buf_1, buf_2); status != STATUS_OK)  
        return status;  
  
    std::unique_ptr<float[]> buf_3(new float[ARRAY_SIZE_3]);  
    return Go_3(buf_1, buf_2, buf_3);  
}
```

Красота! Проверять результат вызова оператора `new[]` не нужно, так как в случае ошибки создания буфера будет сгенерировано исключение. Буферы сами освобождаются при штатном завершении функции или возникновении исключения.

Так какой же смысл писать по старинке в C++? Никакого.

Тогда почему можно встретить такой код? Я думаю, это может происходить вследствие следующих причин.

Первая причина. Человек делает это по привычке. Он не хочет изучать что-то новое, переучиваться. Фактически он пишет код на С с использованием каких-то возможностей C++. Грустный вариант, и непонятно, что тут посоветовать.

Вторая причина. Перед вами C++ код, который когда-то был кодом на С. Его немного изменили, но не переписали и не отрефакторили. Т. е. просто заменили *malloc* на *new*, а *free* — на *delete*. Такой код можно легко распознать по двум артефактам.

Во-первых, в нём присутствуют вот такие проверки-атавизмы:

```
in_audio_ = new int16_t[AUDIO_BUFFER_SIZE_W16];
if (in_audio_ == NULL) {
    return -1;
}
```

Нет смысла проверять указатель на равенство *NULL*, так как в случае ошибки выделения памяти будет сгенерировано исключение типа *std::bad_alloc*. Конечно, существует *new(std::nothrow)*, но это не наш случай.

Во-вторых, часто можно заметить ошибку, что память выделяется с помощью оператора *new[]*, а освобождается с помощью *delete*, хотя правильно использовать *delete[]*. Про это мы, кстати, ещё поговорим в главе N20. Пример:

```
char *poke_data = new char[length + 2*sizeof(int)];
....
delete poke_data;
```

Третья причина. Боязнь дополнительных накладных расходов. Необоснованный страх. Да, у умных указателей иногда могут быть незначительные накладные расходы по сравнению с простыми указателями. Однако следует принять в расчёт:

1. Возможные накладные расходы от умных указателей пренебрежимо малы по сравнению с относительно медленными операциями выделения и освобождения памяти. Если нужна максимальная скорость, то стоит ду-

мать, как уменьшить количество операций выделения/освобождения памяти, а не над тем, использовать умный указатель или контролировать указатели вручную. Ещё вариант — написать собственный аллокатор.

2. Простота, надёжность и безопасность кода, где используются умные указатели, на мой взгляд, однозначно перевешивает дополнительные накладные расходы (которых, кстати, может и не быть).

Четвёртая причина. Программисты просто не осведомлены, как можно использовать тот же `std::unique_ptr`. Условно, они рассуждают так:

«Хорошо, есть `std::unique_ptr`. Он умеет контролировать указатель на объект. Но мне-то ещё нужно работать с массивами объектов. А ещё есть дескрипторы файлов. Местами я вообще вынужден по-прежнему использовать `malloc/realloc`. Для всего этого `unique_ptr` не подходит. Так что проще для единообразия продолжать везде управлять ресурсами вручную».

Всё описанное очень даже можно контролировать с помощью `std::unique_ptr`.

// Работа с массивами:

```
std::unique_ptr<T[]> ptr(new T[count]);
```

// Работа с файлами:

```
std::unique_ptr<FILE, int(*)(FILE*)> f(fopen("a.txt", "r"), &fclose);
```

// Работа с malloc:

```
struct free_delete
```

```
{
```

```
    void operator()(void* x) { free(x); }
```

```
};
```

```
....
```

```
std::unique_ptr<int, free_delete> up((int*)malloc(sizeof(int)));
```

На этом всё. Надеюсь, если у вас были сомнения в умных указателях, я их развеял.

P.S. Я ничего не написал про `longjmp`. И не вижу смысла. В C++ для тех же целей следует использовать исключения.

Вредный совет N18. Компактный код

Используйте как можно меньше фигурных скобок и переносов строк, старайтесь писать условные конструкции в одну строку. Так код будет быстрее компилироваться и занимать меньше места.

Код будет короче — бесспорно. Бесспорно и то, что в нём будет больше ошибок.

«Сжатый код» сложнее читать, а значит, с большей вероятностью опечатки не будут замечены ни автором кода, ни коллегами при его обзоре. Хотите какой-нибудь proof? Легко!

Однажды пользователь написал нам о том, что анализатор PVS-Studio выдаёт странные ложные срабатывания на условие, и прикрепил вот такую картинку:

WORD ch;

```

523 if (ch >= 0xFF00)
524 {
525     if (!((ch >= 0xFF10) && (ch <= 0xFF19)) || ((ch >= 0xFF21) && (ch <= 0xFF3A)) || ((ch >= 0xFF41) && (ch <= 0xFF5A)))
526     {
527         if (j == 0)
528             continue;
529         ch = chx;
530     }
531 }

```

V560	A part of conditional expression is always false: (ch >= 0xFF21).	decodew.cpp	525
☆ V560	A part of conditional expression is always true: (ch <= 0xFF3A).	decodew.cpp	525
☆ V560	A part of conditional expression is always false: (ch >= 0xFF41).	decodew.cpp	525
☆ V560	A part of conditional expression is always true: (ch <= 0xFF5A).	decodew.cpp	525

Видите ошибку в коде? Скорее всего, нет. А почему? А потому, что перед нами большое сложное выражение, написанное в одну строку. Человеку сложно прочесть и осознать этот код. Скорее всего, вы и не стали пробовать разобраться, а сразу продолжили читать книгу.

А вот анализатор не поленился и совершенно справедливо указал на аномалию: часть подвыражений всегда истинны или ложны. Давайте проведём рефакторинг кода:

```

if (!((ch >= 0xFF10) && (ch <= 0xFF19)) ||
    ((ch >= 0xFF21) && (ch <= 0xFF3A)) ||
    ((ch >= 0xFF41) && (ch <= 0xFF5A)))

```

Теперь намного легче заметить, что логический оператор «не» (!) применяется только к первому подвыражению. В общем, здесь не хватает ещё одних скобочек.

Однако ещё больше скобок — это путь не туда. Код станет хоть и правильным, но ещё более страшным. Поэтому я бы написал этот код так:

```
const bool isLetterOrDigit =    (ch >= 0xFF10 && ch <= 0xFF19) // 0..9
                                || (ch >= 0xFF21 && ch <= 0xFF3A) // A..Z
                                || (ch >= 0xFF41 && ch <= 0xFF5A); // a..z

if (!isLetterOrDigit)
```

Обратите внимание, что я убрал часть скобок. Как мы только что видели, большое количество скобок вовсе не помогло избежать ошибки. Скобки должны облегчать чтение кода, а не усложнять. Программисты хорошо помнят, что приоритет сравнений <=, >= выше, чем у оператора &&, поэтому здесь скобки не нужны. А вот если спросить, что приоритетней — && или ||, многие растеряются. Поэтому для задания последовательности вычислений && и || скобки лучше поставить.

Форматирование кода таблицей

Я рекомендую форматировать сложный код «таблицей». Как раз пример такого «табличного» форматирования был показан выше. Такое оформление кода не гарантирует отсутствие опечаток, но позволяет их легче и быстрее замечать.

Давайте разберём эту тему подробнее на другом примере. Возьмём фрагмент кода из проекта ReactOS, в котором я нашёл ошибку благодаря предупреждению PVS-Studio: V560 A part of conditional expression is always true: 10035L.

```
void adns__querysend_tcp(adns_query qu, struct timeval now) {
    ....
    if (!(errno == EAGAIN || EWOULDBLOCK ||
          errno == EINTR || errno == ENOSPC ||
          errno == ENOBUFS || errno == ENOMEM)) {
        ....
    }
}
```

Здесь приведён маленький фрагмент кода — найти в нём ошибку несложно, но в реальном коде заметить ошибку весьма проблематично. Взгляд просто пропускает блок однотипных сравнений и идёт дальше.

Почему мы пропускаем такие ошибки? Условия плохо отформатированы, и не хочется внимательно их читать, это требует усилий. Мы надеемся, что раз проверки однотипные, то всё хорошо, и автор кода не допустил ошибок в условии.

Одним из способов борьбы с опечатками является «табличное» оформление кода.

Для читателей, поленившихся найти ошибку, скажу, что в одном месте пропущено «*errno* ==». В результате условие всегда истинно, так как константа *EWOULDBLOCK* равна 10035. Корректный код:

```
if (!(errno == EAGAIN || errno == EWOULDBLOCK ||
    errno == EINTR || errno == ENOSPC ||
    errno == ENOBUFS || errno == ENOMEM)) {
```

Теперь рассмотрим, как лучше провести рефакторинг этого фрагмента. Для начала я приведу код, оформленный самым простым «табличным» способом. Мне он не нравится.

```
if (!(errno == EAGAIN || EWOULDBLOCK ||
    errno == EINTR || errno == ENOSPC ||
    errno == ENOBUFS || errno == ENOMEM)) {
```

Стало лучше, но ненамного. Такой стиль оформления неудовлетворителен по двум причинам:

1. Ошибка по-прежнему не очень заметна.
2. Приходится вставлять большое количество пробелов для выравнивания.

Поэтому надо сделать два усовершенствования в оформлении кода. Первое — не больше одного сравнения на строку, тогда ошибку легко заметить. Смотрите, ошибка стала больше бросаться в глаза:

```
a == 1 &&  
b == 2 &&  
c      &&  
d == 3 &&
```

Второе — рационально писать операторы `&&`, `||` и т. д., не справа, а слева.

Обратите внимание, как много работы для написания пробелов:

```
x == a      &&  
y == bbbbbb &&  
z == ccccccccc &&
```

А вот так работы намного меньше:

```
  x == a  
&& y == bbbbbb  
&& z == ccccccccc
```

Выглядит код немного необычно, но к этому быстро привыкаешь.

Объединим это всё вместе и напишем в новом стиле код, приведённый в начале:

```
if (!(  errno == EAGAIN  
      || EWOULDBLOCK  
      || errno == EINTR  
      || errno == ENOSPC  
      || errno == ENOBUFS  
      || errno == ENOMEM)) {
```

Да, код стал занимать больше строк кода, зато ошибка стала намного заметнее.

Согласен, смотрится код непривычно. Тем не менее, я рекомендую именно этот подход. Я пользуюсь им последние годы и весьма доволен, поэтому с уверенностью рекомендую его всем читателям.

То, что код стал длиннее, я вообще не считаю проблемой. Я даже написал бы как-то так:

```
const bool error =      errno == EAGAIN
                      || errno == EWOULDBLOCK
                      || errno == EINTR
                      || errno == ENOSPC
                      || errno == ENOBUFS
                      || errno == ENOMEM;

if (!error) {
```

Кто-то ворчит, что это длинно и загромождает код? Согласен. Так давайте вынесем это в функцию!

```
static bool IsInterestingError(int errno)
{
    return      errno == EAGAIN
              || errno == EWOULDBLOCK
              || errno == EINTR
              || errno == ENOSPC
              || errno == ENOBUFS
              || errno == ENOMEM;
}

....

if (!IsInterestingError(errno)) {
```

Может показаться, что я сгущаю краски и я слишком перфекционист, однако ошибки в сложных выражениях очень распространены. Я бы не вспомнил о них, если бы они мне постоянно не попадались: эти ошибки повсюду и плохо заметны.

Вот ещё один пример из проекта WinDjView:

```
inline bool IsValidChar(int c)
{
    return c == 0x9 || 0xA || c == 0xD ||
           c >= 0x20 && c <= 0xD7FF ||
           c >= 0xE000 && c <= 0xFFFFD ||
           c >= 0x10000 && c <= 0x10FFFF;
}
```

В функции всего несколько строк, и всё равно в неё закралась ошибка. Функция всегда возвращает *true*. Вся беда в том, что она плохо оформлена, и многие годы её ленятся читать и не замечают там ошибку.

Давайте отрефакторим код в «табличном» стиле. Дополнительно, я бы ещё скобочки добавил:

```
inline bool IsValidChar(int c)
{
    return
        c == 0x9
        || 0xA
        || c == 0xD
        || (c >= 0x20    && c <= 0xD7FF)
        || (c >= 0xE000  && c <= 0xFFFFD)
        || (c >= 0x10000 && c <= 0x10FFFF);
}
```

Необязательно форматировать код именно так, как я предлагаю. Смысл этой главы — привлечь ваше внимание к опечаткам в «хаотичном коде». Придавая коду «табличный» вид, можно избежать множества глупых опечаток, и это замечательно. Надеюсь, кому-то эта заметка принесёт пользу.

Ложка дёгтя

Я честный программист, и поэтому должен упомянуть, что иногда форматирование «таблицей» может пойти во вред. Вот один из примеров:

```
inline
void elxLuminocity(const PixelRGBi& iPixel,
                   LuminanceCell< PixelRGBi >& oCell)
{
    oCell._luminance = 2220*iPixel._red +
                      7067*iPixel._blue +
                      0713*iPixel._green;
    oCell._pixel = iPixel;
}
```

Это проект eLynx SDK. Программист хотел выровнять код, поэтому перед 713 дописал 0. К сожалению, программист не учёл, что 0 в начале числа означает, что число будет представлено в восьмеричном формате.

Массив строк

Надеюсь, концепция форматирования «таблицей» понятна, но давайте закрепим тему. С этой целью рассмотрим пример, которым я продемонстрирую, что табличное форматирование можно применять не только к условиям, но и к совершенно разным конструкциям языка.

Фрагмент взят из проекта Asterisk. Ошибка выявляется диагностикой PVS-Studio: V653 A suspicious string consisting of two parts is used for array initialization. It is possible that a comma is missing. Consider inspecting this literal: "KW_INCLUDES" "KW_JUMP".

```
static char *token_equivs1[] =
{
    ....
    "KW_IF",
    "KW_IGNOREPAT",
    "KW_INCLUDES"
    "KW_JUMP",
    "KW_MACRO",
    "KW_PATTERN",
    ....
};
```

Опечатка — забыта одна запятая. В результате две различные по смыслу строки соединяются в одну, т. е. на самом деле здесь написано:

```
....
"KW_INCLUDESKW_JUMP",
....
```

Ошибки можно было бы избежать, выравнивая код таблицей. Тогда, если запятая будет пропущена, это будет легко заметить.

```
static char *token_equivs1[] =
{
    ....
    "KW_IF"          ,
    "KW_IGNOREPAT"   ,
    "KW_INCLUDES"    ,
    "KW_JUMP"        ,
    "KW_MACRO"       ,
    "KW_PATTERN"     ,
    ....
};
```

Как и в прошлый раз, обращаю внимание, что если мы ставим разделитель справа (в данном случае это запятая), то приходится добавлять массу пробелов, что неудобно. Особенно неудобно, если появляется новая длинная строка/выражение: придётся перереформатировать всю таблицу.

Поэтому я вновь рекомендую оформлять код так:

```
static char *token_equivs1[] =
{
    ....
    , "KW_IF"
    , "KW_IGNOREPAT"
    , "KW_INCLUDES"
    , "KW_JUMP"
    , "KW_MACRO"
    , "KW_PATTERN"
    ....
};
```

Теперь очень легко заметить недостающую запятую и нет необходимости писать много пробелов — код красивый и понятный. Оформление по началу кажется странным, но к нему быстро привыкаешь, попробуйте.

И под конец короткий лозунг. Как правило, **красивый код — это правильный код.**

Вредный совет N19. Анализаторы нужны только студентам

Профессиональные разработчики не используют статические анализаторы кода. Это инструменты для студентов и неудачников.

Как раз наоборот. В первую очередь статические анализаторы используют профессиональные программисты с целью повысить качество своих программных проектов. Они ценят статический анализ за то, что он помогает выявить многие ошибки на раннем этапе. Чем раньше дефект в коде обнаружен, тем дешевле его устранение. Также статический анализ оказывает незаменимую помощь в поисках уязвимостей нулевого дня [см. термин «Уязвимость нулевого дня»].

Что интересно, у студента есть шанс написать без ошибок программу в рамках курсового проекта. Он вполне может обойтись без статического анализа. А вот написать без ошибок проект уровня игрового движка невозможно. Дело в том, что с ростом кодовой базы растёт плотность ошибок. И для поддержания высокого качества кода приходится затрачивать большие усилия и использовать различные методологии, в том числе и инструменты анализа кода.

Давайте разберём, что значит **рост плотности ошибок**. Чем больше размер кодовой базы, тем легче допустить ошибку. Количество ошибок растёт с ростом размера проекта не линейно, а экспоненциально.

Человеку уже невозможно удержать в голове весь проект. Каждый работает только с частью проекта и кодовой базы. В результате программист не может предвидеть абсолютно все последствия, которые могут возникнуть в процессе изменения им какого-то кода. По-простому: в одном месте что-то изменили, в другом месте что-то сломалось.

Да и в целом, чем сложнее становится система, тем легче допустить ошибку. Это подтверждается числами. Взгляните на таблицу, взятую из книги Стивена Макконнелла «Совершенный код».

Размер проекта (число строк кода)	Типичная плотность ошибок
Менее 2K	0 – 25 ошибок на 1000 строк кода
2K – 16K	0 – 40 ошибок на 1000 строк кода
16K – 64K	0.5 – 50 ошибок на 1000 строк кода
64K – 512K	2 – 70 ошибок на 1000 строк кода
512K и более	4 – 100 ошибок на 1000 строк кода

Статический анализ кода — хороший помощник программистов и менеджеров, заботящихся о качестве проекта и скорости его разработки. Регулярное использование инструментов анализа снижает плотность ошибок, а это в целом положительно сказывается на продуктивности работы. Вот что пишет Дэвид Андерсон в книге «Канбан. Альтернативный путь в Agile»:



Кейперс Джонс сообщает, что в 2000 году во время пузыря доткомов он оценивал качество программ для североамериканских команд, и оно колебалось от шести ошибок на одну функциональную точку до менее чем трех ошибок на 100 функциональных точек — 200 к одному. Серединой будет примерно одна ошибка на 0,6–1,0 функциональной точки. Таким образом, для команд вполне типично тратить более 90 % **своих усилий на устранение ошибок**. Есть и прямое тому свидетельство: в конце 2007 года Аарон Сандерс, один из первых последователей Канбана, написал на листе рассылки Kanbandev, что команда, с которой он работал, тратила 90 % доступной производительности на исправление ошибок.

Стремление к изначально высокому качеству оказывает серьёзное влияние на производительность и пропускную способность команд, имеющих большую долю ошибок. Можно ожидать увеличения пропускной способности в два-четыре раза. Если команда изначально отстающая, то концентрация на качестве позволяет увеличить этот показатель вдесятеро.

Используйте статические анализаторы кода, и ваша команда будет больше заниматься интересным полезным программированием, а не искать, из-за какой опечатки что не работает.

Написанное выше вовсе не означает, что студентам нет смысла использовать статические анализаторы кода. Во-первых, статический анализатор, указывая на ошибки и некачественный код, помогает быстрее осваивать тонкости языка программирования. Во-вторых, умение работать с анализаторами кода пригодится в будущем при работе с большими проектами.

Вредный совет N20. Нет разницы между `delete` и `delete[]`

Если вы выделили память с помощью оператора `new[]`, то для её освобождения формально нужно вызывать `delete[]`. Однако на практике нет ничего страшного, если напишете просто `delete` без квадратных скобок. Всё равно всё сводится к вызову функции `free`.

Грубо говоря, вызов операторов `delete/delete[]` действительно завершается вызовом функции `free` или её аналогом для освобождения памяти в куче. Но есть нюанс! Следует не забывать про конструкторы и деструкторы! Давайте разберёмся, какие последствия из этого вытекают.

Сразу хочу предупредить читателя, что все следующие описания того, во что компилятор превращает операторы, являются условными. Я описываю одну из возможных реализаций компилятора. На самом деле, компиляторы могут работать по-другому. Моя цель — показать, почему `delete` и `delete[]` нельзя путать, хотя иногда кажется, что разницы нет и всё хорошо работает.

Работа с `trivial types`

Начнём с простого:

```
int *p = new int(123);
```

Этот код компилятор может превратить в выделение памяти с последующей инициализацией:

```
int *p = static_cast<int *>(malloc(sizeof(int)));  
*p = 123;
```

Уничтожение объекта:

```
delete p;
```

Скорее всего, сведётся к простому освобождению памяти:

```
free(p);
```

Никаких дополнительных действий оператору *delete* для объекта типа *int* выполнять не требуется.

Теперь рассмотрим работу с простыми массивами:

```
int *p = new int[10] {};
```

Компилятор выделит память под массив и инициализирует его:

```
int *p = static_cast<int *>(malloc(sizeof(int) * 10));  
for (size_t i = 0; i < 10; i++)  
    p[i] = 0;
```

Уничтожение объекта:

```
delete[] p;
```

Вновь достаточно просто освободить буфер памяти. Следовательно, всё опять сводится к:

```
free(p);
```

Смотрите, как интересно получается. Описанная нами реализация компилятора генерирует одинаковый код для *delete* и *delete[]*, если речь идёт о *trivial types*.

Возможно, отсюда берутся ложные представления, что эти операторы взаимозаменяемы и, даже если ошибиться, всё будет работать корректно.

Ещё раз вернёмся к некорректным вариантам:

```
int *a = new int(123);  
delete[] a;                // undefined behavior
```

```
int *b = new int[10];  
delete b;                  // undefined behavior
```

Формально такой код приводит к неопределённому поведению. Но это неопределённое поведение может выражаться и в том, что всё работает, как ожидает программист. Такое возможно, если *delete* и *delete[]* превращаются в вызов функции *free*. Важно понимать, что это просто везение.

Может ли компилятор по-разному обрабатывать вызов *delete* и *delete[]*, если речь идёт о *trivial types*? Да! И тогда всё сломается.

Почему код может быть разный? Например, в целях оптимизации компилятор может использовать разные пулы памяти для выделения маленьких и больших буферов. И при выделении/освобождении памяти будут использованы разные *intrinsic*-функции. В этом случае псевдокод может выглядеть так:

```
// int *p = new int;
// delete p;

int *p = static_cast<int *>(__intrinsic_small_alloc(sizeof(int)));
__intrinsic_small_free(p);
```

И тогда, если перепутать операторы, всё сломается. Неопределённое поведение проявит себя, например, в том, что мы не сможем больше выделять память в испорченном пуле:

```
// int *p = new int[10];
// delete p;

int *p = static_cast<int *>(__intrinsic_big_alloc(sizeof(int)));
__intrinsic_small_free(p);
```

Работа с классами

Примеры будут использовать вот такой класс:

```
class C
{
    size_t size = 100;
    std::unique_ptr<char[]> buf;
public:
    C() : buf(new char[size])
    {
        std::cout << "C()" << std::endl;
    }
    ~C()
    {
        std::cout << "~C()" << std::endl;
    }
};
```

Начнём с создания и уничтожения одного экземпляра класса:

```
C* p = new C;  
delete p;
```

Компилятор может превратить это в следующий псевдокод:

```
C* p = (C *)malloc(sizeof(C));  
new (p) C();  
p->~C();  
free(p);
```

Здесь уже при разрушении объекта одного вызова функции *free* недостаточно — нужно ещё вызвать деструктор класса.

Перейдём к созданию и уничтожению массива объектов. Сейчас начнётся самое интересное. Создадим массив объектов:

```
C* p = new C[3];
```

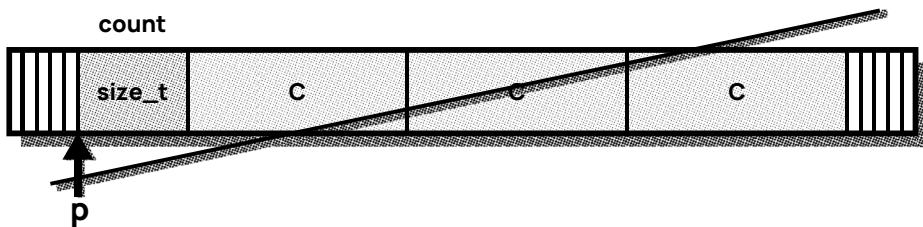
Если поспешить, то можно предположить, что этот код будет выглядеть так:

```
C* p = static_cast<int *>(malloc(sizeof(C) * 3));  
for (size_t i = 0; i < 3; i++)  
    new (p[i]) C();
```

Нет, этого недостаточно. Далее программа будет просто оперировать с указателем *p*, и теряется информация о том, сколько элементов в массиве. Эта информация очень важна, так как компилятору впоследствии надо знать, сколько деструкторов вызывать. Эту информацию нужно хранить, но у нас есть только один указатель. Что же делать?

Одним из способов выйти из затруднения, является выделение памяти чуть больше, чтобы хранить в ней не только сами объекты, но и их количество. Естественно, количество можно разместить только в начале буфера. Если разместить в конце, то мы приходим к той же проблеме: неизвестно, сколько объектов в буфере нужно пропустить, чтобы добраться до их количества.

Хорошо, можно записать в начало буфера количество элементов. Но непонятно, как потом работать с таким указателем. Он ведь содержит не адрес первого экземпляра класса в массиве, а не пойми что с точки зрения остальной программы. Т. е. вот такое расположение данных в памяти нас не устраивает:

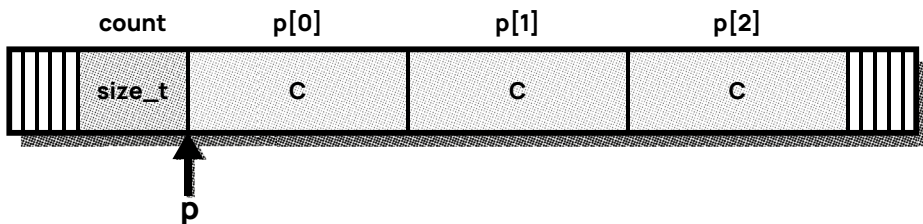


Ничего страшного. Можно подвинуть указатель так, чтобы он ссылался на первый элемент. Но компилятор понимает, что это особый указатель и его надо альтернативно обрабатывать при разрушении массива. Откуда он это знает? У него будет подсказка: вызов *delete[]*.

Псевдокод:

```
// C *p = new C[3];
size_t bufSize = sizeof(size_t) + sizeof(C) * 3;
byte *tmp = static_cast<byte *>(malloc(bufSize));
*(reinterpret_cast<size_t *>(tmp)) = 3;
tmp += sizeof(size_t);
C* p = reinterpret_cast<C *>(tmp);
for (size_t i = 0; i < 3; i++)
    new (p[i]) C();
```

Вот как теперь схематично выглядят данные в памяти:



При вызове оператора `delete[]` компилятор понимает, что перед ним особенный указатель и нужно не просто вызвать один деструктор и функцию `free`, а проделать более сложные действия:

```
// delete[] p;
byte *rawPtr = reinterpret_cast<byte *>(p) - sizeof(size_t);
size_t count = *reinterpret_cast<size_t *>(rawPtr);
for (size_t i = 0; i < count; i++)
    p[i].~C();
free(rawPtr);
```

Теперь, я думаю, вы уже догадываетесь, что путаница между `delete` и `delete[]` приведёт к фатальным последствиям.

Давайте рассмотрим первый вариант некорректного кода:

```
C* p = new C[3];
delete p;
```

В общем случае такая ошибка приводит к неопределённому поведению. Для рассматриваемой нами реализации компилятора это неопределённое поведение проявит себя двумя способами.

Во-первых, будет вызван деструктор только у первого элемента массива. Если запустить такую программу, то она распечатает:

```
C()
C()
C()
~C()
```

Дополнительным следствием будут утечки памяти, так как класс `C` выделяет в конструкторе для своих внутренних нужд память. Имеется в виду `new char[size]`. У двух экземпляров класса не будут вызваны деструкторы, где эта память освобождается.

Во-вторых, вызов `free(p)` закончится неудачей, так как это «не-настоящий указатель». Будет нарушена работа с кучей, и последствия предсказать невозможно.

Второй вариант некорректного кода:

```
C* p = new C;  
delete[] p;
```

Ооо! Тут всё ещё хуже! Чтобы получить количество элементов в массиве, будут прочитаны данные, лежащие перед указателем `p`. Но там нет никакого количества элементов! Там может быть что угодно!

В результате случайные данные будут интерпретироваться как данные, хранящиеся в экземплярах класса. Скорее всего попытка вызовов деструкторов у таких «ненастоящих классов» приведёт к `access violation`. Даже если каким-то чудом программа продолжит выполнение и доберётся до вызова `free`, то в неё тоже будет передан невалидный указатель.

Промежуточный вывод

Как видите, устройство `delete` и `delete[]` может весьма сильно отличаться. Использование одного оператора вместо другого является серьёзной ошибкой, приводящей к неопределённому поведению.

Другие реализации

А существуют другие варианты реализаций операторов `delete`/`delete[]`? Да. Например, количество элементов может храниться не перед массивом объектов, а записываться в какой-то внутренний ассоциативный контейнер наподобие `std::map`.

Тогда создание и удаление массивов может быть представлено таким псевдокодом:

```
std::map<void *, size> globalMap;

// C* p = new C[3];
C* p = static_cast<int *>(malloc(sizeof(C) * 3));
for (size_t i = 0; i < 3; i++)
    new (p[i]) C();
globalMap.insert({p, 3});

// delete[] p;
auto iterator = globalMap.find(p);
if (iterator == globalMap.end())
    RunDoom(); // Неопределённое поведение - это что угодно.
               // В том числе запуск игры DOOM :)
size_t count = iterator->second;
globalMap.erase(iterator);
for (size_t i = 0; i < count; i++)
    p[i].~C();
```

Примечание. На самом деле я везде привожу очень упрощенный псевдокод. Например, он не учитывает возможность генерации исключения в одном из конструкторов, что должно повлечь разрушение элементов массива, которые уже были созданы к этому моменту.

Операторы *new/delete*, предназначенные для работы с одиночными объектами, могут не задействовать *globalMap*. Тогда путаница между *delete* и *delete[]* может стать причиной запуска игры DOOM (проявление неопределённого поведения).

Могут ли быть какие-то ещё реализации операторов? Могут. Просто я знаю только про те две, которые здесь описал.

Кстати, о `void *`

```
void *p = foo();
delete p; // undefined behavior
```

Согласно стандарту языка C++, вызов `delete/delete[]` для указателя типа `void *` приводит к неопределённому поведению. Если вам когда-то было интересно почему, то, думаю, вы получили ответ.

Тип `void *` не несёт никакой информации о типе, а следовательно, неизвестно, нужно вызывать деструктор или нет. И даже если нужно, то неизвестно, что это за класс.

Подведём итог

Откуда я знаю все эти моменты, связанные с работой `new/delete`? Чего только не узнаешь, разрабатывая статический анализатор кода. Приходится разбираться в таких деталях, чтобы потом реализовать в PVS-Studio соответствующие диагностики. Мы познакомились со всем этим, разрабатывая диагностики, выявляющие аномалии, связанные с путаницей между `delete` и `delete[]`, а также с вызовом `delete` для `void *`.

Всё это знать прикладному программисту необязательно. Стандарт говорит: неопределённое поведение — всё. Этого достаточно. Так писать нельзя. Точка.

Однако интересно заглянуть под капот и понять, как оно всё работает. Хотя бы просто из любопытства. Дополнительно вы в очередной раз убедились, что, когда говорят про неопределённое поведение, это не пустые слова. В каком-то смысле это продолжение главы N13, так как мы в очередной раз увидели, что неопределённое поведение — это реальная, серьёзная штука.

Вредный совет N21. Просьба помощь на форуме, делай так, чтобы из тебя вытягивали информацию. Так всем будет интереснее

Все только и мечтают тебе помочь. Поэтому стоит спросить на Stack Overflow/Reddit «почему мой код не работает?», и все готовы преодолеть любые препятствия, чтобы ответить.

Вам готовы помочь, дайте шанс это сделать

Этот вредный совет предназначается тем, кто пришёл за помощью на сайт Stack Overflow, Reddit или ещё какой-то форум. Сообщество достаточно лояльно относится к помощи новичкам, но иногда создаётся впечатление, что задающие вопросы делают всё возможное, чтобы его проигнорировали. Рассмотрим подобный вопрос, который я видел на Reddit.

Тема: Кто-то сможет мне объяснить, почему у меня возникает segmentation fault?

Тело вопроса: Ссылка на документ, лежащий в облачном хранилище OneDrive. И всё.

Это хороший пример, как не надо задавать вопросы.

Выбран крайне неудобный способ для выкладывания кода. Вместо того чтобы прочитать необходимый код сразу в теле вопроса, нужно переходить по ссылке. Сайт OneDrive требовал, чтобы я залогинился, но я поленился это делать. Не настолько мне было интересно, что там за код. Судя по отсутствию ответов на вопрос, лень было не только мне. Более того, многие вообще там не зарегистрированы.

Непонятно даже, о каком языке программирования идёт речь. Стоит ли идти смотреть код, если может оказаться, что это незнакомый тебе язык...

Вопрос задан неправильно. Я так и не узнал, что там за код был, но подозреваю, что его могло быть недостаточно, чтобы дать ответ.

Никакой дополнительной информации вопрос не содержит. Не надо так делать.

Попробую сформулировать, как следует написать вопрос, чтобы вам помогли.

Основной принцип

Пусть тем, кого вы просите дать ответ, будет удобно!

Вопрос должен быть самодостаточным и содержать всю необходимую информацию, чтобы не требовалось задавать уточняющие вопросы. Часто спрашивающие делают неосознанную ошибку, считая, что и так всем всё будет понятно. Помните: помогающие вам люди не знают, что делают ваши функции и классы. Покажите, как они устроены или хотя бы опишите словами.

В идеале следует предоставить минимальный фрагмент кода, на котором воспроизводится проблема. Да, не всегда это получается, но крайне полезно по трём причинам:

1. Пока вы будете составлять минимальный пример для воспроизведения, высока вероятность, что вы сами поймёте, в чём состоит ошибка.
2. С воспроизводимой ошибкой легко и приятно работать. Скорее всего, вы получите нужную помощь от сообщества.
3. Оформив воспроизводимый пример, вы можете попробовать обнаружить в нём ошибку, воспользовавшись другим компилятором — или статическим анализатором кода. Для этого удобно использовать сайт `Compiler Explorer` (godbolt.org). Про него я расскажу ниже. На сайте можно в online-режиме скомпилировать код разными C++ компиляторами с разными ключами. Возможно, один из инструментов выдаст полезное предупреждение.

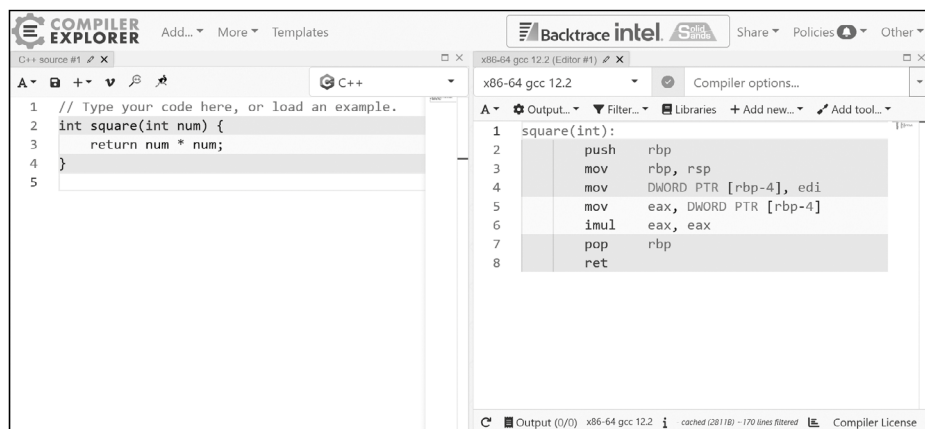
Compiler Explorer

`Compiler Explorer` позволяет проводить быстрые эксперименты с кодом. Код можно скомпилировать и запустить. Можно

посмотреть предупреждения/ошибки, выдаваемые разными компиляторами, изучить ассемблерный листинг при разных ключах оптимизации и так далее.

Крутая фишка — сохранить ссылку на код и все выставленные настройки и использовать её, чтобы публиковать вопросы на форуме или показывать код коллегам.

Открываем сайт Compiler Explorer по ссылке **godbolt.org**. Сайт так называется в честь его создателя Мэтта Годболта (Matt Godbolt). По умолчанию в редакторе введена функция возведения в квадрат, а в правом окне показан сгенерированный ассемблерный код.



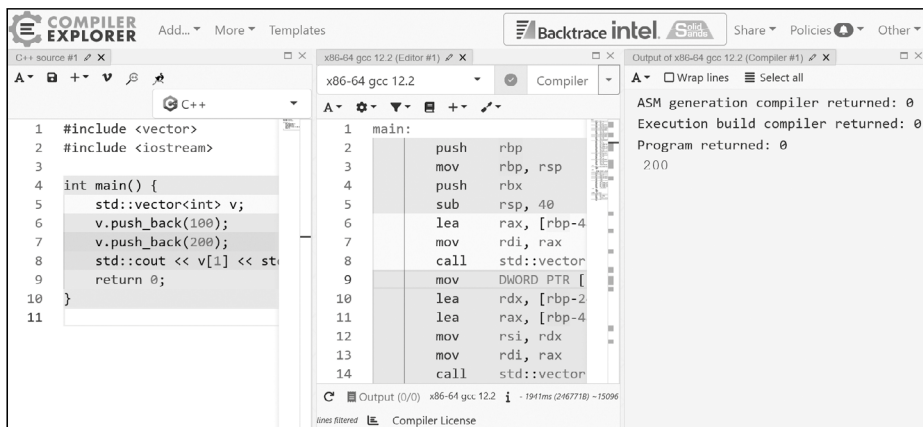
Напишем в редакторе собственную программу. Для начала корректную.

```
#include <vector>
```

```
#include <iostream>
```

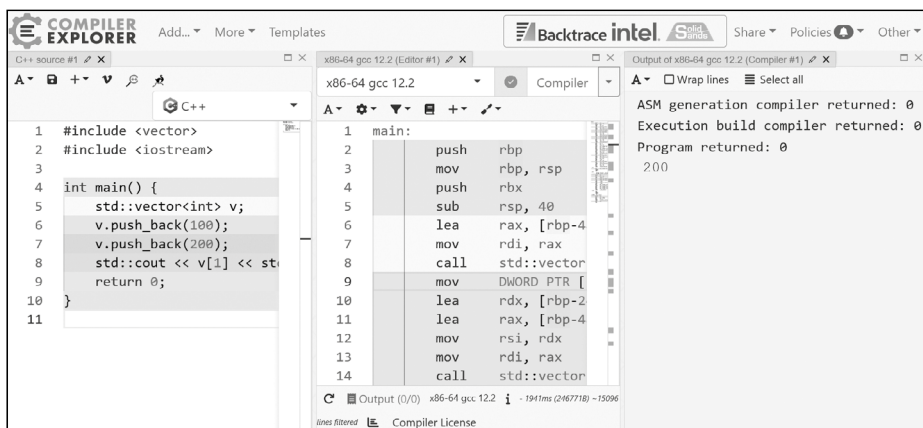
```
int main() {
    std::vector<int> v;
    v.push_back(100);
    v.push_back(200);
    std::cout << v[1] << std::endl;
    return 0;
}
```

Откроем окно вывода результатов работы: нажмём на «Output» в левом нижнем углу ассемблерного листинга. Далее нажимаем на шестерёнку и выбираем опцию «Execute the code».



Описание возможностей Compiler Explorer тянет на отдельную книгу. Поэкспериментируйте сами с интерфейсом и опциями. Можно гибко настраивать, что и как отображается.

Перенесём вкладку с ассемблерным листингом влево и сделаем активной вкладку с кодом на C++: теперь мы можем писать код и смотреть, как он работает. Думаю, такой способ использования сайта может приглянуться студентам для быстрого выполнения лабораторных заданий по программированию.

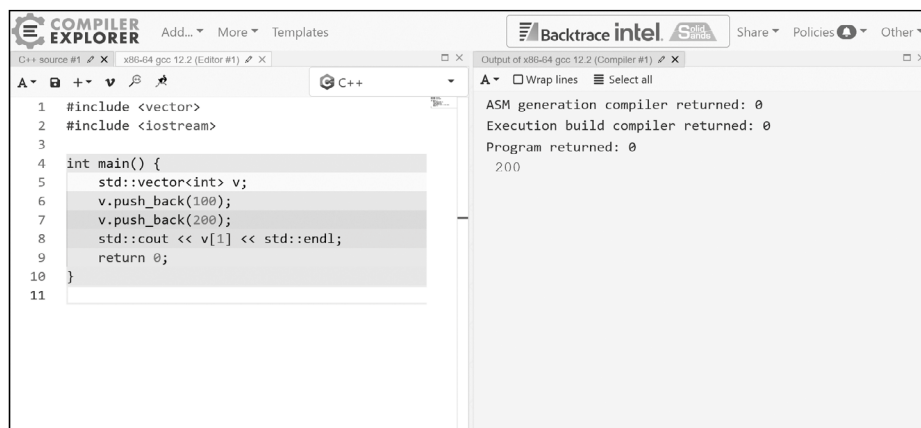


Сейчас мы видим, что код успешно компилируется: программа запускается и печатает ожидаемое значение 200.

Сломаем код. Предположим, мы только начали изучать C++ и забыли, что элементы нумеруются с 0. Поэтому мы используем для взятия последнего элемента массива индекс 2.

```
std::cout << v[2] << std::endl;
```

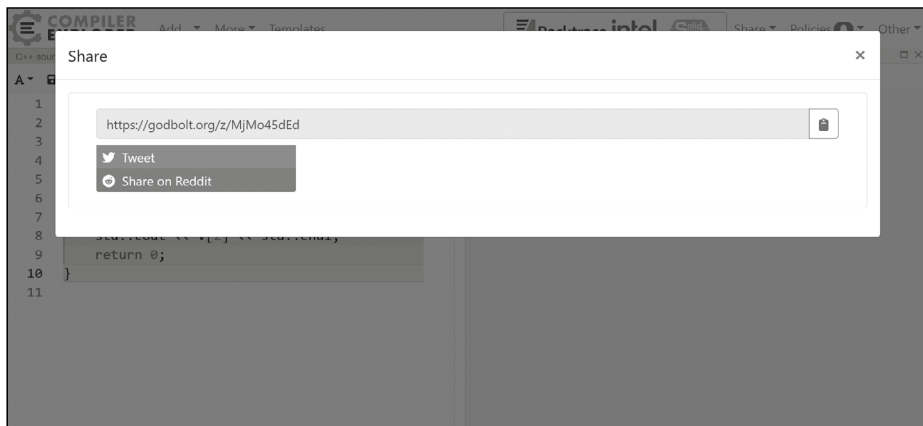
Программа по-прежнему компилируется, но выводит 0.



Думаю, понятно, что перед нами выход за границу массива, который приводит к неопределённому поведению программы. Конкретно в данном случае это поведение выражается не в падении программы, а в распечатке мусора из памяти. Сейчас в памяти оказались нули, поэтому и распечатано значение 0.

Допустим, мы по-прежнему пребываем в неведении, что не так с нашим кодом и хотим разобраться с проблемой. Compiler Explorer станет нашим хорошим помощником.

Во-первых, можно сгенерировать ссылку на ваш код со всеми настройками. Другими словами, имеется возможность легко поделиться примером с воспроизводящейся ошибкой. Для этого выберите в меню пункт «Share» -> «Short Link»:

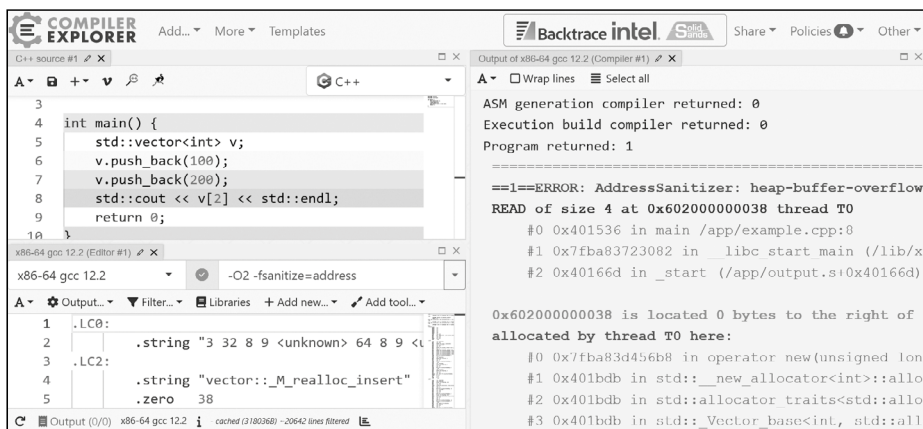


Во-вторых, можно попробовать найти ошибку с помощью предупреждений компилятора или статического анализатора кода.

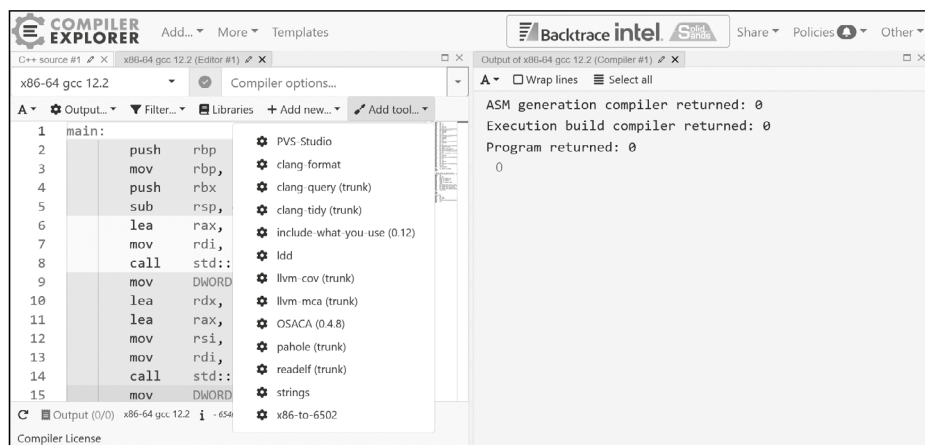
Сразу скажу, что я не смог подобрать ключи для компиляторов GCC и Clang, чтобы они выдали предупреждение о выходе за границу массива. Компиляторы могут предупредить о выходе за границы обыкновенных массивов, таких как `int arr[100]`, но молчат при работе с контейнерами `std::vector`. По крайней мере так было на момент написания книги.

Ладно, хорошо, признаюсь. Я специально составил такой сложный для компиляторов пример. Иначе было бы не так интересно.

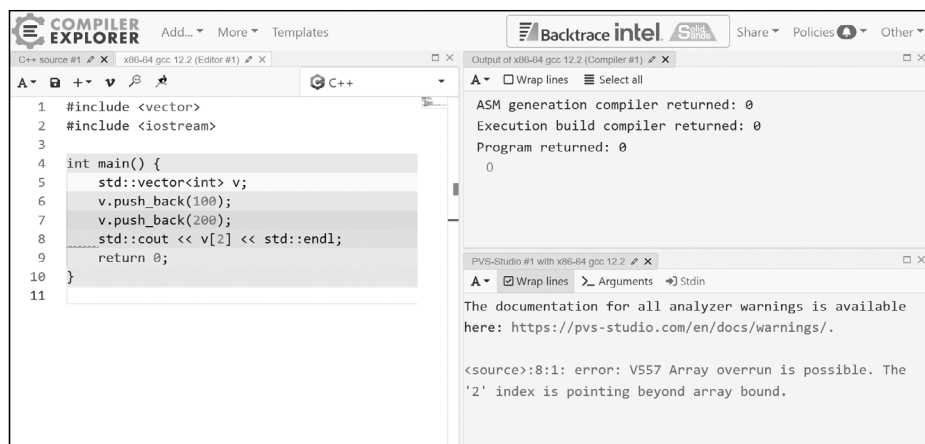
Зато ошибку вполне можно найти с помощью AddressSanitizer, который, кстати, тоже можно запустить прямо на сайте:



Также ошибка может быть найдена ещё на этапе компиляции статическим анализатором PVS-Studio. Нажмите на кнопку «Add tool...» и выберите в списке пункт «PVS-Studio»:



После этого откроется окно с отчётом анализатора:



Запуск статического анализатора в данном случае более предпочтителен, так как вы получите простое понятное предупреждение: V557 Array overrun is possible. The '2' index is pointing beyond array bound. Если непонятно, почему анализатор выдал то или иное предупреждение, загляните в документацию на сайте PVS-Studio. Каждое предупреждение сопровождается подробным описанием с примерами кода.

Воспользовавшись Compiler Explorer, вы можете сэкономить своё собственное время, проводя различные эксперименты и изыскания. Это может оказаться намного быстрее, чем задавать вопросы на форуме и ждать ответов. Если же задаёте вопрос, очень полезно, помимо всего прочего, добавить ссылку на сайт Compiler Explorer.

Вредный совет N22. Ромбовидное наследование

Язык C++ разрешает выполнять виртуальное наследование и реализовывать с его помощью ромбовидное наследование. Так почему бы не воспользоваться такой прикольной штукой?

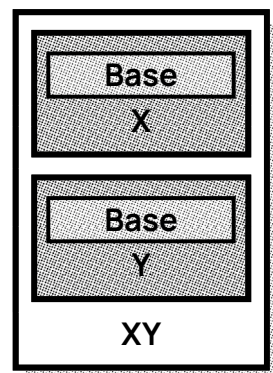
Использовать можно. Только надо знать про некоторые тонкости, и сейчас мы их рассмотрим.

Об инициализации виртуальных базовых классов

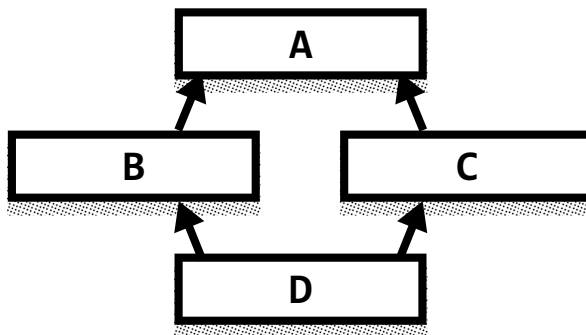
Вначале поговорим, как размещаются в памяти экземпляры классов, если нет виртуального наследования. Рассмотрим код:

```
class Base { .... };  
class X : public Base { .... };  
class Y : public Base { .... };  
class XY : public X, public Y { .... };
```

Здесь всё просто. Члены не виртуального базового класса *Base* размещаются как простые данные-члены производного класса. В результате внутри объекта *XY* мы имеем два независимых подобъекта *Base*. Справа схематическое изображение.



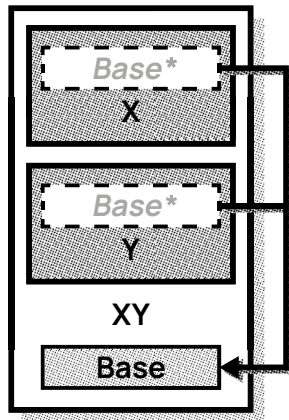
Теперь рассмотрим ситуацию с виртуальными базовыми классами. Объект виртуального базового класса входит в объект производного класса только один раз. Это ещё называется ромбовидным наследованием:



Рассмотрим устройство класса *XY*, в котором применяется ромбовидное наследование.

```
class Base { ... };
class X : public virtual Base { ... };
class Y : public virtual Base { ... };
class XY : public X, public Y { ... };
```

Память для разделяемого подобъекта *Base*, скорее всего, будет выделена в конце объекта *XY*. Как именно будет устроен класс, зависит от компилятора. Например, в классах *X* и *Y* могут храниться указатели на общий объект *Base*. Но, как я понимаю, такой метод вышел из обихода. Чаще ссылка на разделяемый подобъект реализуется в виде смещения или информации, которая хранится в таблице виртуальных функций.



Только «самый производный» класс *XY* точно знает, где должна находиться память для подобъекта виртуального базового класса *Base*. Поэтому инициализировать все подобъекты виртуальных базовых классов поручается «самому производному» классу.

Конструкторы *XY* инициализируют подобъект *Base* и указатели на этот объект в *X* и *Y*. Затем инициализируются остальные члены классов *X*, *Y*, *XY*.

После того как подобъект *Base* инициализируется в конструкторе *XY*, он не будет ещё раз инициализироваться конструктором *X* или *Y*. Как это будет сделано, зависит от компилятора. Например, компилятор может передавать специальный дополнительный аргумент в конструкторы *X* и *Y*, который будет указывать, что инициализировать класс *Base* не нужно.

А теперь самое интересное и являющееся источником ошибок. Рассмотрим вот такие конструкторы:

```
X::X(int A) : Base(A) {}
Y::Y(int A) : Base(A) {}
XY::XY() : X(3), Y(6) {}
```

Какое число примет конструктор базового класса в качестве

аргумента? Число 3 или 6? Ни одно из них.

Конструктор *XY* инициализирует виртуальный подобъект *Base*, но делает это неявно: вызывается конструктор *Base* по умолчанию. Когда конструктор *XY* вызывает конструктор *X* или *Y*, он не инициализирует *Base* заново. Поэтому явного обращения к *Base* с каким-то аргументом не происходит.

На этом приключения с виртуальными базовыми классами не заканчиваются. Помимо конструкторов существуют операторы присваивания. Если я не ошибаюсь, стандарт говорит, что генерируемый компилятором оператор присваивания может многократно выполнять присваивание подобъекту виртуального базового класса. Однако это необязательно, и он может сделать это только один раз. Так что неизвестно, сколько раз будет происходить копирование объекта *Base*.

Если вы реализуете свой оператор присваивания, то должны самостоятельно позаботиться об однократном копировании объекта *Base*. Рассмотрим неправильный код:

```
XY &XY::operator =(const XY &src)
{
    if (this != &src)
    {
        X::operator =(*this);
        Y::operator =(*this);
        ....
    }
    return *this;
}
```

Этот код приведёт к двойному копированию объекта *Base*. Чтобы этого избежать, в классах *X* и *Y* необходимо реализовать функции, которые не будут копировать члены класса *Base*.

Содержимое класса *Base* копируется однократно здесь же. Исправленный код:

```
XY &XY::operator =(const XY &src)
{
    if (this != &src)
    {
        Base::operator =(*this);
        X::PartialAssign(*this);
        Y::PartialAssign(*this);
        ....
    }
    return *this;
}
```

Такой код будет работать, но всё это некрасиво и запутанно. Поэтому и говорят, что лучше избегать множественного виртуального наследования.

Виртуальные базовые классы и приведение типов

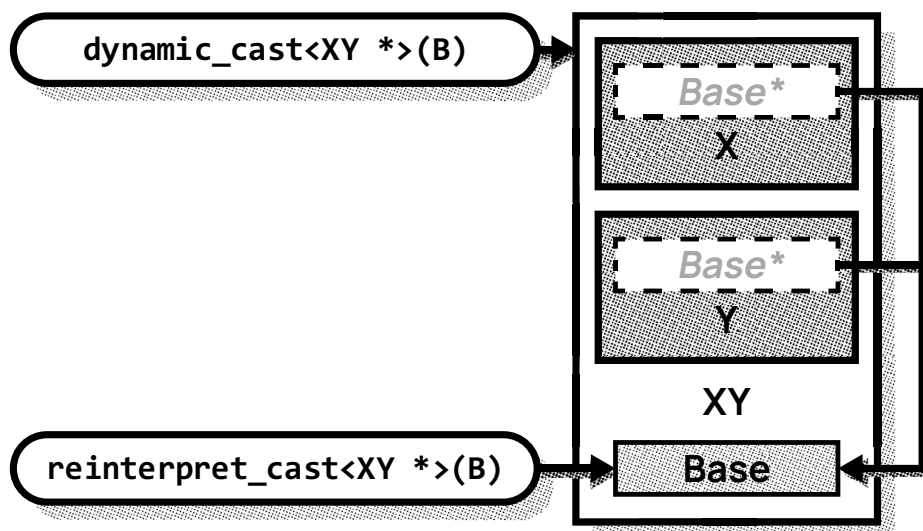
Из-за особенностей размещения виртуальных базовых классов в памяти нельзя выполнить такие приведения типов:

```
Base *B = Get();
XY *Q = static_cast<XY *>(B); // Ошибка компиляции
XY *W = (XY *) (B);          // Ошибка компиляции
```

Однако настойчивый программист может всё-таки привести тип, воспользовавшись оператором 'reinterpret_cast':

```
XY *E = reinterpret_cast<XY *>(B);
```

Однако, скорее всего, это даст непригодный для использования результат. Адрес начала объекта *Base* будет интерпретирован как начало объекта *XY*. А это совсем не то, что надо.



Единственный способ выполнить приведение типа — воспользоваться оператором *dynamic_cast*. Однако код, где регулярно используется *dynamic_cast*, плохо пахнет.

Отказываться ли от виртуального наследования?

Я согласен с мнением многих авторов, что следует всячески избегать виртуального наследования. И от простого множественного наследования тоже лучше уходить.

Виртуальное наследование порождает проблемы при инициализации и копировании объектов. Инициализацией и копированием должен заниматься «самый производный» класс. А это значит, он должен знать интимные подробности об устройстве базовых классов. Образуется лишняя связанность между классами, которая усложняет структуру проекта и заставляет делать дополнительные правки в разных классах при рефакторинге. Всё это способствует ошибкам и усложняет понимание проекта новыми разработчиками.

Сложности приведений типов также способствуют возникновению ошибок. Отчасти проблемы решаются при использовании оператора *dynamic_cast*. Однако это медленный оператор: если он начинает массово появляться в программе, то, скорее всего, это

свидетельствует о плохой архитектуре проекта. Почти всегда можно реализовать структуру проекта, не прибегая к множественному наследованию. Собственно, во многих языках вообще нет таких изысков, и это не мешает реализовывать большие проекты.

Глупо настаивать отказаться от виртуального наследования. Иногда оно полезно и удобно. Однако стоит хорошо подумать, прежде чем им воспользоваться.

Полезьа от множественного наследования

Хорошо, критика множественного виртуального наследования и просто множественного наследования понятна. А есть ли места, где такое наследование безопасно и удобно?

Да, я могу назвать как минимум одно: подмешивание интерфейсов. Если вам не знакома эта методология, предлагаю обратиться к книге Алена И. Голуба «Верёвка достаточной длины, чтобы... выстрелить себе в ногу». Легко ищется в интернете. Следует смотреть раздел 101 и далее.

В интерфейсном классе нет никаких данных. Все функции, как правило, чисто виртуальные. Конструктора в нём нет, или он ничего не делает. Это значит, что нет проблем с созданием или копированием таких классов.

Если базовый класс является интерфейсом, то присваивание — это пустая операция. Так что, даже если объект будет копироваться множество раз, это нестрашно. В скомпилированном коде программы это копирование будет просто отсутствовать.

Вредный совет N23. Напишу всё сам

Не пользуйтесь стандартной библиотекой языка. Что может быть интереснее, чем написать свои строки и списки с уникальным синтаксисом и семантикой?

Может быть, это действительно интересно. Вот только это отнимает много времени, причём результат, скорее всего, окажется более низкого качества, чем у существующих стандартных решений. На практике выходит, что непросто написать без ошибок даже аналоги таких простых функций, как *strdup* или *memcpy*. А теперь представьте, сколько недостатков будет в более сложных функциях и классах.

Не верите про *strdup* и *memcpy*? Дело в том, что я уже даже начал коллекционировать ошибки в самодельных функциях копирования данных. Возможно, когда-то я сделаю про это отдельную статью. А сейчас пара proof-ов.

В статье про проверку Zephyr RTOS я описал вот такую неудачную попытку реализации аналога функции *strdup*:

```
static char *mntpt_prepare(char *mntpt)
{
    char *cpy_mntpt;

    cpy_mntpt = k_malloc(strlen(mntpt) + 1);
    if (cpy_mntpt) {
        ((u8_t *)mntpt)[strlen(mntpt)] = '\0';
        memcpy(cpy_mntpt, mntpt, strlen(mntpt));
    }
    return cpy_mntpt;
}
```

Предупреждение PVS-Studio: V575 [CWE-628] The 'memcpy' function doesn't copy the whole string. Use 'strcpy / strcpy_s' function to preserve terminal null. shell.c 427

Анализатор сообщает, что функция *тетсру* копирует строку, но не копирует терминальный ноль, и это очень подозрительно. Кажется, что этот терминальный 0 копируется здесь:

```
((u8_t *)mntpt)[strlen(mntpt)] = '\0';
```

Нет, здесь опечатка, из-за которой терминальный ноль копируется сам в себя. Обратите внимание, что запись происходит в массив *mntpt*, а не в *cpy_mntpt*. В итоге функция *mntpt_prepare* возвращает строку, не завершённую терминальным нулём.

На самом деле программист хотел написать так:

```
((u8_t *)cpy_mntpt)[strlen(mntpt)] = '\0';
```

Непонятно только, зачем код написан так запутанно и нестандартно. Как результат, в небольшой и несложной функции допущена серьёзная ошибка. Этот код можно упростить до следующего варианта:

```
static char *mntpt_prepare(char *mntpt)
{
    char *cpy_mntpt;

    cpy_mntpt = k_malloc(strlen(mntpt) + 1);
    if (cpy_mntpt) {
        strcpy(cpy_mntpt, mntpt);
    }
    return cpy_mntpt;
}
```

Другой пример. Программист интересуется на сайте Stack Overflow, на правильном ли он пути.

```
void myMemCpy(void *dest, void *src, size_t n)
{
    char *csrc = (char *)src;
    char *cdest = (char *)dest;
    for (int i=0; i<n; i++)
        cdest[i] = csrc[i];
}
```

Этот код содержит недостаток: нельзя использовать в качестве индекса переменную типа *int*. В 64-битной программе почти наверняка (экзотические архитектуры не рассматриваем) переменная *int* будет 32-битной, и функция сможет скопировать не более *INT_MAX* байт. Т. е. не более 2 Гигабайт.

При большем размере копируемого буфера произойдёт переполнение знаковой переменной, что с точки зрения языка C и C++ является неопределённым поведением.

Особенно забавно, что этот код появился как попытка убрать какое-то предупреждение анализатора Checkmarx, возникавшее при вызове функции *memcpy*. Программист не придумал ничего лучше, кроме как сделать свой собственный велосипед. И, несмотря на простоту функции копирования, она всё равно получилась неправильной. То есть по факту человек, скорее всего, сделал ещё хуже, чем было. Вместо того чтобы разобраться в причине предупреждения, он замаскировал проблему написанием своей собственной функции (запутал анализатор). Плюс добавил ошибку, используя для счётчика *int*. Ах да, такой код ещё может помешать оптимизации. Неэффективно использовать свой собственный код вместо оптимизированной функции *memcpy*. Не делайте так.

Вредный совет N24. Удалите stdafx.h

Удалите отовсюду этот дурацкий stdafx.h. Из-за него постоянно возникают какие-то странные ошибки компиляции.

«Вы просто не умеете их готовить» (с). Давайте разберёмся, как в среде Visual Studio работают предкомпилируемые заголовки и как их правильно использовать.

Для чего нужны precompiled headers

Precompiled headers предназначены для ускорения сборки проектов. Обычно программисты начинают знакомиться с Visual C++, используя крошечные проекты. На них сложно заметить выигрыш от precompiled headers. Что с ними, что без них — на глаз программа компилируется одинаковое время. Это добавляет путаницы. Человек не видит для себя пользы от этого механизма и решает, что он для специфичных задач и никогда не понадобится. И иногда считает так многие годы.

На самом деле precompiled headers — весьма полезная технология. Пользу можно заметить, даже если в проекте всего несколько десятков файлов. Особенно выигрыш становится заметен, если используются такие тяжёлые библиотеки, как boost.

Если посмотреть *.cpp файлы в проекте, то можно заметить, что во многие включаются одни и те же наборы заголовочных файлов. Например, `<vector>`, `<string>`, `<algorithm>`. В свою очередь, эти файлы включают другие заголовочные файлы и так далее.

Всё это приводит к тому, что препроцессор в компиляторе вновь и вновь выполняет идентичную работу. Он должен читать одни и те же файлы, вставлять их друг в друга, выбирать `#ifdef` ветки и подставлять значения макросов. Происходит колоссальное дублирование одних и тех же операций.

Можно существенно сократить объём работы, которую дол-

жен проделать препроцессор при компиляции проекта. Идея в том, чтобы заранее препроцессировать группу файлов и затем просто подставлять готовый фрагмент текста.

На самом деле делается ещё ряд шагов. Можно хранить не просто текст, а более обработанную информацию. Я не знаю, как именно устроено в Visual C++. Но, например, можно хранить текст, уже разбитый на лексемы. Это ещё больше ускорит процесс компиляции.

Как работают **precompiled headers**

Файл, который содержит **precompiled headers**, имеет расширение «.pch». Имя файла обычно совпадает с названием проекта. Естественно, это и другие используемые имена можно изменить в настройках. Файл может быть весьма большим, и его размер зависит от того, как много заголовочных файлов в нём раскрыто.

Файл *.pch создаётся в результате компиляции stdafx.cpp с ключом «/Yc». Этот ключ как раз и говорит компилятору, что нужно создать **precompiled header**. Файл stdafx.cpp может содержать одну строчку: *#include «stdafx.h»*.

В файле «stdafx.h» находится самое интересное. Сюда нужно включить заголовочные файлы, которые будут заранее препроцессироваться. Пример того, как может выглядеть такой файл:

```
#pragma warning(push)
#pragma warning(disable : 4820)
#pragma warning(disable : 4619)
#pragma warning(disable : 4548)
#pragma warning(disable : 4668)
#pragma warning(disable : 4365)
#pragma warning(disable : 4710)
#pragma warning(disable : 4371)
#pragma warning(disable : 4826)
```

```
#pragma warning(disable : 4061)
#pragma warning(disable : 4640)
#include <stdio.h>
#include <string>
#include <vector>
#include <iostream>
#include <fstream>
#include <algorithm>
#include <set>
#include <map>
#include <list>
#include <deque>
#include <memory>
#pragma warning(pop)
```

Директивы «`#pragma warning`» нужны, чтобы избавиться от предупреждений, выдаваемых на стандартные библиотеки, если стоит высокий уровень предупреждений в настройках компилятора. Это фрагмент кода из старого проекта. Возможно, сейчас все эти *pragma* не нужны. Я просто хотел показать, как можно подавить лишние предупреждения, если такие возникнут.

Теперь во все файлы `*.c/*.cpp` следует включить «`stdafx.h`». Заодно стоит удалить из этих файлов заголовки, которые уже включаются с помощью «`stdafx.h`».

А что делать, если используются хотя и похожие, но разные наборы заголовочных файлов? Например, такие:

- файл A: `<vector>`, `<string>`
- файл B: `<vector>`, `<algorithm>`
- файл C: `<string>`, `<algorithm>`

Нужно делать отдельные `precompiled headers`? Так сделать можно, но не нужно.

Достаточно сделать один precompiled header, в котором будут раскрыты `<vector>`, `<string>` и `<algorithm>`. Выигрыш от того, что при препроцессировании не надо читать множество файлов и вставлять их друг в друга, намного больше, чем потери от синтаксического анализа лишних фрагментов кода.

Как использовать precompiled headers

Wizard в Visual Studio при создании нового проекта создаёт два файла: `stdafx.h` и `stdafx.cpp`. Именно с помощью них и реализуется механизм precompiled headers.

На самом деле эти файлы могут называться как угодно. Важно не название, а параметры компиляции в настройках проекта.

В `*.c/*.cpp` файле можно использовать только один precompiled header. Однако в одном проекте может присутствовать несколько разных precompiled headers. Пока будем считать, что он у нас только один.

Итак, если вы воспользовались wizard-ом, то у вас уже есть файлы `stdafx.h` и `stdafx.cpp`. Плюс выставлены все необходимые ключи компиляции.

Если в проекте не использовался механизм precompiled headers, то давайте рассмотрим, как его включить. Предлагаю следующую последовательность действий:

1. Во всех конфигурациях для всех `*.c/*.cpp` файлов включаем использование precompiled headers. Это делается на вкладке «Precompiled Header»:
 - выставляем для параметра «Precompiled Header» значение «Use (/Yu)»;
 - для параметра «Precompiled Header File» указываем «`stdafx.h`»;
 - для параметра «Precompiled Header Output File» указываем «`$(IntDir)$(TargetName).pch`».
2. Создаём и добавляем в проект файл `stdafx.h`. В дальнейшем в него мы будем включать те заголовочные файлы, которые хотим заранее препроцессировать.

3. Создаём и добавляем в проект файл stdafx.cpp. В нём одна единственная строка: `#include «stdafx.h»`.
4. Во всех конфигурациях меняем настройки для файла stdafx.cpp. Выставляем для параметра «Precompiled Header» значение «Create (/Yc)».

Вот мы и включили механизм precompiled headers. Теперь если мы запустим компиляцию, то будет создан *.pch файл. Однако затем компиляция остановится из-за ошибок.

Для всех *.c/*.cpp файлов мы указали, что они должны использовать precompiled headers. Этого мало. Теперь в каждый из файлов нужно добавить `#include «stdafx.h»`.

Заголовочный файл «stdafx.h» должен включаться в *.c/*.cpp файл самым первым. Обязательно! Иначе всё равно возникнут ошибки компиляции.

Если подумать, в этом есть логика. Когда файл «stdafx.h» находится в самом начале, то можно подставить уже препроцессированный текст. Этот текст всегда одинаков и ни от чего не зависит.

Представьте ситуацию, если бы мы могли включить до «stdafx.h» ещё какой-то файл. А в этом файле взяли бы и написали: `#define bool char`. Возникла бы неоднозначность. Мы меняем содержимое всех файлов, в которых встречается «bool». Теперь просто так нельзя взять и подставить заранее препроцессированный текст. Ломается весь механизм «precompiled headers». Думаю, это одна из причин, почему «stdafx.h» должен быть расположен в начале. Возможно, есть и другие.

Life hack!

Прописывать `#include «stdafx.h»` во все *.c/*.cpp файлы достаточно утомительно и неинтересно. Дополнительно получится ревизия в системе контроля версий, где будет изменено огромное количество файлов. Нехорошо.

Ещё одно неудобство вызывают сторонние библиотеки, вклю-

чаемые в проект в виде файлов с кодом. Править эти файлы нет смысла. По правильному для них нужно отключить использование «precompiled headers». Однако, если используется несколько мелких сторонних библиотек, это неудобно. Программист постоянно спотыкается о precompiled headers.

Есть вариант, как использовать precompiled headers легко и просто. Способ подойдёт не везде и не всегда, но мне он часто помогал.

Можно не прописывать во все файлы `#include «stdafx.h»`, а воспользоваться механизмом «Forced Included File».

Идём на вкладку настроек «Advanced». Выбираем все конфигурации. В поле «Forced Included File» пишем:

```
StdAfx.h;%(ForcedIncludeFiles)
```

Теперь «stdafx.h» автоматически будет включаться в начало ВСЕХ компилируемых файлов. PROFIT!

Больше не потребуется писать `#include «stdafx.h»` в начале всех *.c/*.cpp файлов. Компилятор сделает это сам.

Что включать в stdafx.h

Это очень важный момент. Бездумное включение в «stdafx.h» всего подряд не только не ускорит компиляцию, но и наоборот — замедлит её.

Все файлы, включающие «stdafx.h», зависят от его содержимого. Пусть в «stdafx.h» включён файл «X.h». Если вы поменяете хоть что-то в «X.h», это может повлечь полную перекомпиляцию всего проекта.

Правило. Включайте в «stdafx.h» только те файлы, которые никогда не изменяются или меняются ОЧЕНЬ редко. Хорошими кандидатами являются заголовочные файлы системных и сторонних библиотек.

Если включаете в «stdafx.h» собственные файлы из проекта, соблюдайте двойную бдительность. Включайте только те фай-

лы, которые меняются очень-очень редко.

Если какой-то *.h файл меняется раз в месяц, это уже слишком часто. Как правило, редко удаётся сделать все правки в h-файле с первого раза. Обычно требуется 2-3 итерации. Согласитесь, 2-3 раза полностью перекомпилировать весь проект — занятие неприятное. Плюс полная перекомпиляция потребуется всем вашим коллегам.

Не увлекайтесь и с неизменяемыми файлами. Включайте только то, что действительно часто используется. Нет смысла включать <set>, если это нужно только в двух местах. Где нужно, там и подключите этот заголовочный файл.

Несколько precompiled headers

Зачем в одном проекте может понадобиться несколько precompiled headers? Действительно, это нужно нечасто, но приведу пару примеров.

В проекте используются одновременно *.c и *.cpp файлы. Для них нельзя использовать единый *.pch файл. Компилятор выдаст ошибку.

Нужно создать два *.pch файла. Один должен получаться при компировании C-файла (xx.c), а другой — при компировании C++-файла (yy.cpp). Соответственно в настройках надо указать, чтобы в C-файлах использовался один precompiled header, а в C++-файлах — другой.

Примечание. Не забудьте указать разные имена для *.pch файлов. Иначе один файл будет перетирать другой.

Другая ситуация. Одна часть проекта использует одну большую библиотеку, а другая часть — другую большую библиотеку.

Естественно, не стоит всем участкам кода знать про обе библиотеки. Плюс в (неудачных) библиотеках могут пересекаться имена каких-то сущностей.

Логично сделать два precompiled headers и использовать их в разных участках программы. Как уже отмечалось, можно задать произвольные имена файлов, из которых генерируются *.pch файлы. Да и имя *.pch файла тоже можно изменить. Всё это, конечно, требуется делать аккуратно, но ничего сложного в использовании двух precompiled headers нет.

Типовые ошибки при использовании precompiled headers

Прочитав внимательно материал выше, вы сможете понять и устранить ошибки, связанные с stdafx.h. Но давайте ещё раз пройдёмся по типовым ошибкам компиляции и разберём их причины.

Fatal error C1083: Cannot open precompiled header file: 'Debug\project.pch': No such file or directory

Вы пытаетесь скомпилировать файл, который использует precompiled header. Но соответствующий *.pch файл отсутствует. Возможные причины:

1. Файл stdafx.cpp не компилировался и, как следствие, *.pch файл ещё не создан. Такое может быть, если, например, вначале очистить проект (Clean Solution), а потом попробовать скомпилировать один *.cpp файл. Решение: скомпилируйте проект целиком или как минимум файл stdafx.cpp.
2. В настройках не указано ни одного файла, из которого должен генерироваться *.pch файл. Речь идёт о ключе компиляции «/Yc». Как правило, такая ситуация возникает у начинающих, которые захотели использовать precompiled headers для своего проекта. Как сгенерировать *.pch файл, описано выше в разделе «Как использовать precompiled headers».

Fatal error C1010: unexpected end of file while looking for precompiled header. Did you forget to add '#include "stdafx.h"' to your source?

Сообщение говорит само за себя. Файл компилируется с ключом /Yu. Это значит, что следует использовать precompiled header. Но в файл не включён «stdafx.h».

Нужно вписать в файл #include «stdafx.h».

Если это невозможно, то следует не использовать precompiled header для этого *.c/*.cpp файла. Уберите ключ /Yu.

Fatal error C1853: 'project.pch' precompiled header file is from a previous version of the compiler, or the precompiled header is C++ and you are using it from C (or vice versa)

В проекте присутствуют как C (*.c), так и C++ (*.cpp) файлы. Для них нельзя использовать единый precompiled header (*.pch файл).

Возможные решения:

1. Отключить для всех C-файлов использование precompiled headers. Как показывает практика, *.c файлы препроцессируются в несколько раз быстрее, чем *.cpp файлы. Если *.c файлов не очень много, то, отключив precompiled headers для них, вы ничего не потеряете.
2. Завести два precompiled headers. Первый должен создаваться из stdafx_cpp.cpp, stdafx_cpp.h. Второй из stdafx_c.c, stdafx_c.h. Соответственно, в *.c и *.cpp файлах следует использовать разные precompiled headers. Имена *.pch файлов, естественно, тоже должны различаться.

Из-за precompiled header компилятор глючит

Скорее всего, что-то сделано не так. Например, #include «stdafx.h» расположен не в самом начале.

Рассмотрим пример:

```
int A = 10;
#include "stdafx.h"
int _tmain(int argc, _TCHAR* argv[]) {
    return A;
}
```

```
}
```

Этот код не скомпилируется. Компилятор выдаст, на первый взгляд, странное сообщение об ошибке:

```
error C2065: 'A' : undeclared identifier
```

Компилятор считает, что всё указанное до строчки `#include «stdafx.h»` (включительно) является `precompiled header`. При компиляции файла компилятор заменит всё, что до `#include «stdafx.h»`, на текст из `*.pch` файла. В результате теряется строчка `«int A = 10»`.

Правильный вариант:

```
#include "stdafx.h"

int A = 10;

int _tmain(int argc, _TCHAR* argv[]) {
    return A;
}
```

Ещё пример:

```
#include "my.h"
#include "stdafx.h"
```

Содержимое файла `«my.h»` не будет использоваться. В результате нельзя будет использовать функции, объявленные в этом файле. Такое поведение очень сбивает программистов с толку. Они «лечат» его полным отключением `precompiled headers` и потом рассказывают байки о глючности Visual C++. Помните, компилятор — это один из наиболее редко глючащих инструментов. В 99.99 % случаев надо не злиться на компилятор, а искать ошибку у себя (см. главу N11).

Чтобы таких ситуаций не было, ВСЕГДА пишите `#include «stdafx.h»` в самом начале файла. Комментарии перед `#include «stdafx.h»` можно оставить. Они всё равно никак не участвуют в компиляции.

Ещё один вариант — используйте `Forced Included File`. См.

выше раздел «Life hack».

Из-за precompiled headers проект постоянно перекомпилируется целиком

В stdafx.h включён файл, который часто редактируется или генерируется автоматически.

Внимательно проверьте содержимое файла «stdafx.h». В него должны входить только заголовочные файлы, которые не изменяются или изменяются крайне редко. Учтите, что включённые файлы могут не меняться, но внутри они могут ссылаться на другие изменяющиеся *.h файлы.

Творится что-то непонятное

Иногда может возникнуть ситуация, что вы поправили код, а ошибка не исчезает. Отладчик показывает непонятные вещи.

Причиной может быть *.pch файл. Как-то так получилось, что компилятор не замечает изменения в одном из заголовочных файлов и не перестраивает *.pch файл. В результате подставляется старый код. Возможно, это происходило из-за каких-то сбоев, связанных с временем модификации файлов.

Это ОЧЕНЬ редкая ситуация. Но она возможна, и про неё надо знать. Я за многие годы программирования сталкивался с ней только 2-3 раза. Помогает полная перекомпиляция проекта.

Вредный совет N25. Массив на стеке — лучшее решение

И вообще, выделение памяти — зло. *char c[256]* хватит всем, а если не хватит, то потом поменяем на 512. В крайнем случае – на 1024.

Создание массива на стеке действительно имеет ряд преимуществ по сравнению с выделением памяти в куче с помощью функции *malloc* или оператора *new []*:

1. Выделение и освобождение памяти происходит моментально. Если посмотреть ассемблерный код, то всё сводится просто к уменьшению/увеличению указателя стека.
2. Не нужно заботиться об освобождении памяти. Она автоматически будет освобождена при выходе из функции.

Так чем же тогда плоха конструкция *char c[1024]*?

Причина N1. Магическое число 1024 намекает, что автор кода на самом деле точно не знает, сколько памяти понадобится для обработки данных. Он предполагает, что одного килобайта всегда будет достаточно. Так себе надёжность.

Если известно, что буфер определённого размера вместит все необходимые данные, то, скорее всего, мы увидим в коде именованную константу, например *char path[MAX_PATH]*.

Подход с созданием «буфер с запасом» ненадёжен. Во-первых, есть вероятность, что «запаса» при определённом стечении обстоятельств не хватит, как ни в чём не бывало к переполнению буфера. Во-вторых, такой код является потенциальной уязвимостью. Есть вероятность, что на вход программы можно специально подать такие данные, которые не просто переполнят буфер, но и изменят поведение программы нужным для злоумышленника образом.

Причина N2. Как мы обсудили, фиксированный размер буфера плох тем, что его может не хватить. При этом в большинстве случаев этот буфер избыточен. Почти всегда резервируется больше памяти, чем потом используется. Если увлекаться созданием таких массивов на стеке, он неожиданно может кончиться. Возникнет переполнение стека.

Настройки компилятора позволяют задавать, сколько стековой памяти доступно приложению. Например, в Visual C++ для этого используется ключ «/F».

/F (Set Stack Size). Задаёт размер стека в байтах. Синтаксис: */F number*. Аргумент *number* может быть десятичным числом или числовым литералом в нотации языка C. Значение округляется в большую сторону для кратности 4 байтам. Без этой опции размер резервируемого стека равен 1 мегабайту.

Можно и здесь указать значение «с запасом». Но это вновь весьма ненадёжное решение, так как невозможно предсказать, как много стековой памяти понадобится приложению (особенно если на стеке создаётся множество массивов).

Что интересно, некоторые компиляторы для микроконтроллеров выдадут ошибку компиляции, если вычислят, что стека недостаточно для одной из веток выполнения программы. В некоторых контроллерах стек очень маленький (считанные байты), и такая проверка поможет выявить ошибку переполнения стека ещё на этапе компиляции. Компиляторы могут это сделать, потому что и сами программы очень маленькие и простые. В случае классических прикладных приложений невозможно на этапе компиляции вычислить, сколько максимально стека понадобится. Собственно, глубина использования стека может зависеть от входных данных, о которых компилятор ничего не знает.

Как лучше поступать?

Создание массива на стеке не является ни хорошим, ни плохим решением. Плюсы: максимально быстрый способ выделения памяти, не требуется заботиться о её освобождении. Минус: риск переполнения стека, особенно если создавать буферы «с запасом». Как и многие другие возможности языка C++, создание массива на стеке — острый полезный нож, которым можно порезаться.

Можно как-то улучшить ситуацию? Да, можно создавать на стеке массивы не фиксированного размера, а ровно того, который требуется. Их называют «Variable Length Arrays» (VLA) [см. термин «VLA»].

Во-первых, это позволит экономить стековую память, не выделяя лишнего. Во-вторых, это снизит риск переполнения буфера, поскольку можно выделить памяти ровно столько, сколько нужно для обработки данных. Я написал «снизит риск», а не «гарантирует отсутствие» по следующей причине: даже если буфера достаточно, ошибка может содержаться в алгоритме обработки данных.

В языке C создать на стеке массив произвольного размера крайне просто. Достаточно написать:

```
void foo(size_t n)
{
    float array[n];
    // ....
}
```

Это возможно, начиная с C99.

Как я написал выше, это позволяет экономить стек, выделяя память без запаса. С другой стороны, можно увлечься и начать выделять на стеке слишком большие массивы и в результате получить переполнение стека. Так что не расслабляйтесь и думайте, не надёжнее ли выделять память в куче.

В C++ другая ситуация. В C++ нет возможности создавать *variable-length* массивы.

Впрочем, некоторые компиляторы, такие как GCC, позволяют вам использовать VLA в C++ программе. Но это расширение, на которое не стоит полагаться из-за непереносимости кода!

Так есть ли какой-то способ в C++ выделить на стеке буфер произвольного размера? Да, есть. Можно воспользоваться функцией *alloca*.

```
void *alloca(size_t size);
```

Функция выделяет *size* байт на стеке. Память автоматически освобождается при выходе из функции, которая выделяла память с помощью *alloca*. Предупреждение: функция не возвращает *NULL*. Если в стеке недостаточно свободной памяти, то возникает неопределённое поведение.

Эту функцию можно использовать в C и C++ программах. Что удобно, память автоматически освобождается при выходе из функции.

```
void foo(size_t n)
{
    float *array = (float)alloca(sizeof(float) * n);
    // ....
}
```

Но опять есть нюанс. Эта функция не является частью стандарта C++. Как и в предыдущем случае, её использование не позволит написать переносимый C++ код. На самом деле, это вообще *intrinsic function*, вызов которой компилятор превращает в изменение указателя стека.

Возможно, вы начинаете подозревать некий «сговор разработчиков компиляторов», который мешает вам выделять на стеке буфер произвольного размера. Видимо, что-то такое есть.

И правильно! Дело в том, что такой код провоцирует возникновение в программах уязвимостей.

В стандарте MISRA C есть правило MISRA-C-18.8, указывающее не использовать VLA. Другие руководства, такие как SEI CERT C Coding Standard (ARR32-C) и Common Weakness Enumeration (CWE-129), не запрещают использовать эти массивы, но призывают проверять перед созданием их размер.

Если алгоритм обработки входных данных написан без единой ошибки, то неважно, используется буфер на стеке или в куче. Однако на практике люди допускают ошибки, которые можно эксплуатировать как уязвимости. И с этой точки зрения, порча стека намного более вредоносна и позволяет проводить больше вариантов атак, чем порча данных в куче.

Причём выход за границы массива — это не единственный способ нарушения работы программы. Например, некий алгоритм может выделять на стеке буфер достаточного размера и корректно обрабатывать данные, но при этом не учитывать размер этих данных. Подавая извне данные большого размера, можно спровоцировать исчерпание стека. Это вызовет неопределённое поведение, что с большой вероятностью приведёт к аварийному завершению работы приложения. Это вариант DoS-атаки уровня приложения.

В общем, лишний раз стоит задуматься, выделять ли буфер на стеке или в куче. Особенно если вы разрабатываете приложение, критическое с точки зрения надёжности и безопасности.

Вредный совет N26. Ничего «лишнего»

Не пользуйтесь системой контроля версий. Храните исходники прямо на сервере в виртуалке.

Думаю, никто из настоящих программистов не придерживается этого совета. Те, кто так поступал, или не становятся программистами, или учатся на своих ошибках, потеряв исходные коды.

Обычно это происходит ещё в университете, когда вдруг не читается флешка, на которой лежит сделанная программа/лабораторная работа/курсовая. И есть только достаточно старая копия, или её вообще нет.



Впрочем, встречаются отдельные заповедники динозавров. Мне рассказывал один человек, что, будучи студентом, он попал на практику в государственную организацию. Хотя на дворе был уже где-то 2010 год, они хранили различные версии исходных кодов, просто запаковав их в архив. Он высказал сомнение по эффективности такого подхода. А главное, что сложно понять, когда, кем и зачем менялся код. В ответ, они подвели его к огромному журналу, где ручкой каждый записывал, что он делал с кодом.

Вредный совет N27. Необычные конструкции

Все знают, что операторы обращения по индексу к указателю обладают коммутативностью. Так не будьте серыми, как все. Придайте коду оригинальность, используя конструкции вида `1[array] = 0`.

Профессионализм разработчика не в том, чтобы написать в стиле «смотрите, что я умею и как могу». Профессионализм — это написать код, который легко читать и сопровождать.

Наверное, все проходят через стадию, когда хочется максимально применить полученные знания. Кто-то читает книги про паттерны и начинает их использовать где надо и не надо, перегружая код бессмысленными классами, фабриками и т. д. Кто-то вдохновляется книгами типа «Современное проектирование на C++». Обобщённое программирование и прикладные шаблоны проектирования». И затем невозможно понять, как работает вся эта шаблонная магия, и главное — зачем она вообще нужна.

Полезно всё это изучить, переболеть и начать писать простой понятный код.

Новичок напишет обратный цикл так:

```
for (int i = n - 1; i >= 0; i--)
```

// Или так:

```
int i = n;  
while (--i >= 0)
```

Программист, подверженный звёздной болезни, напишет что-то типа этого:

```
int i = n;  
while (i-->0)
```

Прикольно, красиво, но не нужно. Такая конструкция заставляет задуматься или даже начать подозревать, а не придума-

ли ли в C++ новый оператор «-->».

На самом деле, это обыкновенный постфиксный декремент и оператор больше «>». Переменная *i* будет, как и в предыдущих примерах, принимать значения в диапазоне $[n - 1 .. 0]$. Немного подумайте и поймёте, как это работает.

Разобрались? Думаю, да. А теперь вопрос, а стоило ли оно того? Какой смысл был разбираться в этом хитром коде, который на самом деле выполняет очень простую вещь? Никакого.

Поэтому профессиональный программист напишет такой же код, что и новичок:

```
for (int i = n - 1; i >= 0; i--)
```

```
//Или так:
```

```
int i = n;
```

```
while (--i >= 0)
```

Важна простота и понятность. Тем более что скорость работы этого кода одинакова.

Примечание. Впрочем, есть ещё один нормальный вариант, как можно написать:

```
for (int i = n - 1; i >= 0; --i)
```

Я бы именно так и написал. Сложные префиксные итераторы для некоторых контейнеров работают быстрее, чем постфиксные. В данном случае префиксный декремент переменной типа *int* никакого выигрыша не даст. Но я пишу префиксный декремент везде для единообразия. Он точно будет работать точно так же, как постфиксный, или чуть быстрее.



У вас могут возникнуть сомнения, актуален ли в настоящее время этот стиль с точки зрения оптимизации. Ответ вы найдёте в статье [«Есть ли практический смысл использовать для итераторов префиксный оператор инкремента ++it, вместо постфиксного it++»](#).

Если читать её лень или нет возможности, то просто знайте: да, смысл есть.

Вредный совет N28. Всё в h-файлах

Побольше кода в заголовочных файлах, ведь так гораздо удобнее, а время компиляции возрастает очень незначительно.

В эпоху моды на header-only библиотеки этот совет не кажется таким уж и вредным.

Но одно дело — маленькие библиотеки, а другое — большой проект, в который вовлечены десятки людей и который развивается многие годы. В какой-то момент время компиляции вырастет с минут до часов, а сделать что-то с этим уже будет сложно. Не рефакторить же сотни и тысячи файлов, перенося реализацию функций из *.h в cpp-файлы? А если рефакторить, то не проще ли сразу было делать нормально?

Самым плохим следствием размещения реализации функций в заголовочных файлах является то, что минимальная правка приводит к необходимости перекомпиляции большого количества файлов в проекте. Есть важное отличие между кодом в header-only библиотеках и кодом вашего проекта. Код в библиотеках вы не трогаете, а свой код вы постоянно правите!

Здесь, кстати, уместно вспомнить про существование паттерна PImpl (pointer to implementation). Он используется, чтобы изменение реализации класса не приводило к необходимости перекомпиляции файлов, которые используют этот класс. Максимально упрощённое использование этого паттерна выглядит так:

```
// Заголовочный .h файл, в котором объявлен интерфейсный класс Foo
class Foo
{
public:
    int run(int value);
    Foo();
private:
    struct FooImpl;
```

```
std::unique_ptr<FooImpl> pImpl;
};

// .cpp файл. в котором реализована бизнес-логика класса Foo
struct Foo::FooImpl
{
    int a, b;
    FooImpl() : a(1), b(2) {}
    int run(int value) { return a + b + value; }
};

int Foo::run(int value)
{
    return pImpl->run(value);
}

Foo::Foo() : pImpl(new FooImpl)
{
}
```

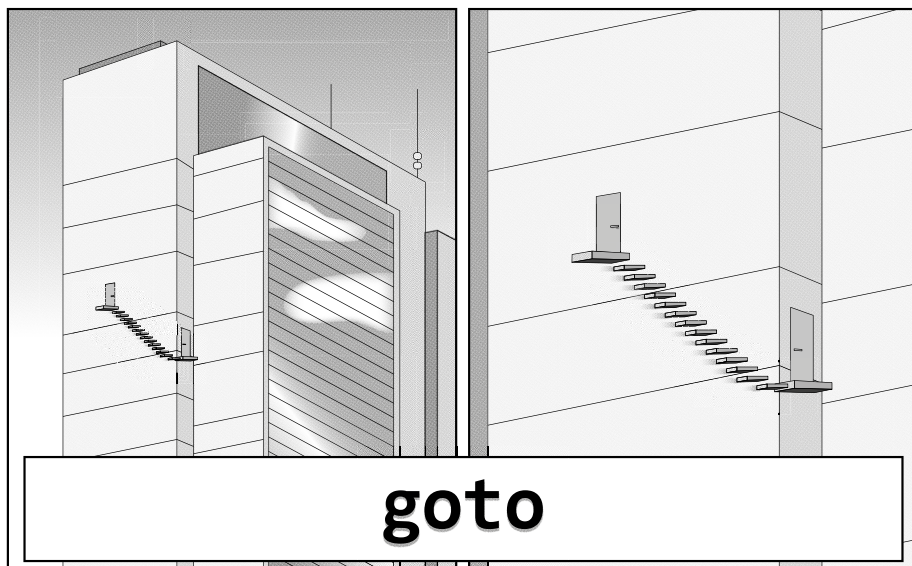
Можно без затруднений менять класс *FooImpl*, реализующий бизнес-логику. После добавления в класс новых членов, изменения логики функций и так далее потребуется перекомпиляция только одного сpp-файла, в котором находится реализация *FooImpl*.

Перекомпиляция большого числа файлов потребуется только в случае изменения интерфейса класса *Foo*. Но интерфейсы меняются реже, чем логика. У паттерна PImpl есть и недостаток — дополнительные накладные расходы при вызове функций и уменьшение возможностей компилятору применять различные оптимизации.

Всё на самом деле сложнее и интереснее, поэтому, если вам захотелось побольше познакомиться с паттерном PImpl, предлагаю загуглить это слово. В интернете много статей на эту тему.

Вредный совет N29. Оператор goto

Злые языки говорят, что `goto` считается вредным оператором, но это чушь. Этот оператор очень мощен и даже позволяет отказаться от `for`, `while`, `do`. Да здравствует `goto` и аскетизм!



Использование оператора `goto` провоцирует усложнение кода для понимания. Код, пронизанный операторами `goto`, сложно читать сверху вниз. Особенно если присутствуют переходы снизу вверх. Придётся «скакать» по меткам, чтобы понять, как устроена логика программы. Чем больше функция и чем больше в ней используется операторов `goto`, тем сложнее разобраться. Есть даже специальный термин: спагетти-код.

Спагетти-код — плохо спроектированная и слабо структурированная программа, содержащая много операторов `goto` (особенно переходов назад). Программа запутанна и поэтому трудна для понимания. Спагетти-код берёт своё название из аналогии, что ход выполнения программы похож на миску спагетти, то есть извилистый и запутанный.

Спагетти-код может выполнять свою задачу (работать правильно) и даже с высокой производительностью. Но он крайне сложен в сопровождении и развитии! Добавление новой функциональности в спагетти-код иногда несёт такой огромный риск внесения ошибок, что декомпозиция и рефакторинг становятся неизбежным шагом.

Естественно, виноват не сам по себе оператор `goto`, а его недуманное использование. Если он не виноват, почему существует рекомендация вообще его не использовать? Ведь в C++ многие конструкции опасны.

Дело в том, что без этого оператора можно вполне обойтись. Более того, без `goto` код обычно выглядит и читается лучше.

Вообще сложно даже привести какие-то примеры уместного использования `goto`. В голову приходит только паттерн с одной точкой выхода, который мы рассматривали в главе N17. Все `goto` осуществляют переход на одну метку, после которой следует освобождение памяти и возврат из функции кода ошибки. Такой код не создаёт сложности для понимания. Однако дальше мы рассмотрели, что ещё лучше использовать умные указатели. Получается, что `goto` опять-таки не нужен.

Вредный совет N30. Везде нужен некий константный экземпляр класса? Для удобства объявите его в заголовочном файле

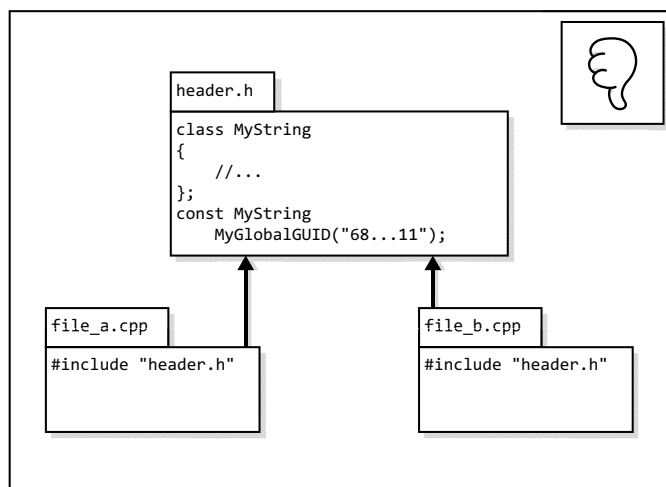
Бывает, что повсеместно нужен какой-то глобальный константный объект. Например, экземпляр самодельной строки. Для удобства разместите его в заголовочном файле, где объявлен ваш класс строки.

В итоге получается что-то такое:

```
// MyString.h
class MyString {
    char *m_buf;
    size_t m_size, m_capacity;
public:
    MyString(const char *str);
    ....
};
const MyString
    MyGlobalGUID("68648619-B3EA-4A5B-AB3A-4F6E2703DD11");
```

На самом деле, писать свой собственный класс строки — так себе идея, и мы поговорим про это в главе N42. Я просто сходу не придумал какой-то другой распространённый класс, который имеет тяжеловесный конструктор, выделяющий память.

Беда в том, что *MyGlobalGUID* вовсе не глобальный константный объект, существующий в одном экземпляре.



На рисунке код, который компилируется, но здесь нет объявления одной глобальной переменной.

При включении такого заголовочного файла через `#include` произойдёт создание множественных копий объекта. Это приведёт к пустой трате памяти и времени для создания множества строк.

В общем случае, если в классе есть конструктор, его код выполнится при каждом включении заголовочного файла, что может привести к нежелательным побочным эффектам.

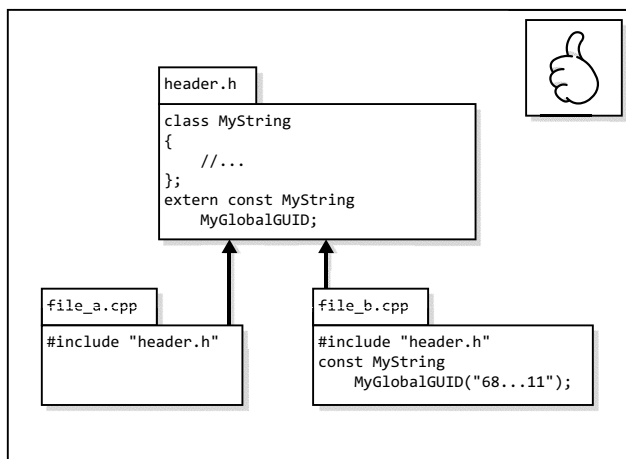
Иногда люди торопятся внести правки, добавляя `static`:

```
const static MyString
    MyGlobalGUID("68648619-B3EA-4A5B-AB3A-4F6E2703DD11");
```

Но `static` здесь не в тему. Видимо, возникает путаница со статическими членами класса. Действительно, статическая переменная в классе будет общей для всех экземпляров класса.

В данном же случае ничего не изменилось. В каждой единице трансляции по-прежнему будет создаваться своя константная переменная.

Чтобы избежать множественного создания объектов, можно объявить переменную как `extern`. В этом случае инициализация и вызов конструктора произойдут один раз.



Это классический старый способ. Единственный недостаток — если посмотреть `h`-файл, то непонятно, как инициализируется константа. Впрочем, рядом всегда можно оставить поясняющий комментарий. Сама глобальная переменная создаётся и инициализируется в одном из `cpp`-файлов.

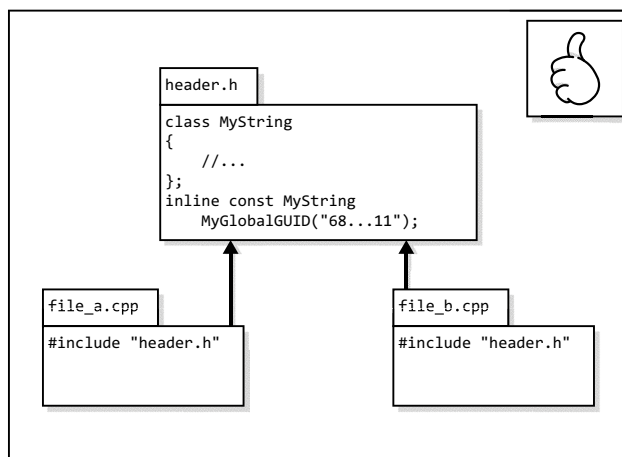
Стоит предупредить, что если мы говорим об обыкновенных численных константах, то, возможно, лучше их оставить в `h`-файле как есть и не добавлять *extern*.

```
// constants.h
const double g_golden_ratio(1.6180339887498948);
const size_t g_cache_size(1024);
```

Если объявить их *extern*, то такие константы больше не будут считаться константами типа *compile-time* и поэтому не смогут использоваться где-либо, где потребуется константа такого типа. Дополнительно это может негативно сказаться на возможностях оптимизации кода компилятором.

Начиная с C++17, можно поступить более изящно и объявить глобальную константную переменную как *inline*.

```
// MyString.h
....
inline const MyString
    MyGlobalGUID("68648619-B3EA-4A5B-AB3A-4F6E2703DD11");
```



Теперь сразу видно, каким значением инициализируется глобальная константа, и не требуется что-то писать в сpp-файлы.

Вредный совет N31. Объявление переменных в начале функции

Проявите немного уважения к программистам прошлого — объявляйте все переменные в начале функций. Это традиция!

От вредных традиций стоит отказываться. Переменную лучше всего объявлять как можно ближе к месту её использования. Ещё лучше, когда переменная сразу инициализируется при объявлении. Рассмотрим преимущества такого подхода.

Оптимизация

Если переменная «тяжёлая» и используется только при выполнении какого-то условия, то можно улучшить производительность, создавая её только в случае необходимости.

Рассмотрим инициализацию класса *Image*. Класс выдуманный, но я черпал вдохновение на основе реального кода. Обратите внимание, что конструктор работает медленно из-за вызовов операторов *new*. Выделение и освобождение памяти в куче — это всегда тяжёлые операции.

```
Image::Image(uint32_t imagetype) :  
    head(new BITMAPINFOHEADER { 0 }),  
    info(new IMAGEINFO { 0 })  
{  
    pDib = pSelection = pAlpha = NULL;  
    ppLayers = ppFrames = NULL;  
    info->dwType = imagetype;  
    info->fQuality = 90.0f;  
    info->nAlphaMax = 255;  
    info->nBkgndIndex = -1;  
    info->bEnabled = true;  
    info->nJpegScale = 1;  
    SetXDPI(IMAGE_DEFAULT_DPI);  
    SetYDPI(IMAGE_DEFAULT_DPI);  
}
```

Теперь представим, что некая функция использует этот класс следующим образом:

```
void foo(const bool a, const bool b, const bool c)
{
    Image imgA(IMAGE_FORMAT_PNG);
    Image imgB(IMAGE_FORMAT_JPG);
    Image imgC(IMAGE_FORMAT_GIF);
    // ....

    if (a)
    {
        // ....
        Use(imgA, 1);
    }

    if (b)
    {
        // ....
        Use(imgB, 2);
    }

    if (c)
    {
        // ....
        Use(imgC, 3);
    }
}
```

Независимо от того, понадобятся экземпляры класса или нет, под них резервируется память на стеке и происходит их начальная инициализация.

Чисто теоретически компилятор может сам оптимизировать изначальный код и создавать объекты только при необходимости. Но на практике компилятору будет очень сложно понять, можно ли так делать. Тут и выделение памяти, и конструкторы, и передача объектов во внешнюю функцию. Так что такая ручная подсказка компилятору (оптимизация) здесь крайне уместна.

Этот код лучше написать следующим образом:

```
void foo(const bool a, const bool b, const bool c)
{
    // ....

    if (a)
    {
        // ....
        Image img(IMAGE_FORMAT_PNG);
        Use(img, 1);
    }

    if (b)
    {
        // ....
        Image img(IMAGE_FORMAT_JPG);
        Use(img, 2);
    }

    if (c)
    {
        // ....
        Image img(IMAGE_FORMAT_GIF);
        Use(img, 3);
    }
}
```

С функциональной точки зрения ничего не изменилось. Зато не тратится время на инициализацию объектов, которые не потребуются!

Возможно, будет сэкономлена и стековая память: объекты могут поочерёдно создаваться в одном и том же фрагменте стека. Для этого класса это не критично, но для больших по размеру классов имеет смысл. Будет выполнена оптимизация использования стека или нет — зависит от компилятора и настроек.

Легче понимать код

Ещё раз взгляните на первый вариант функции *foo* в этой главе. Может, она и несложная для понимания. Но представьте, что код вам не знаком и вы смотрите на строчку:

```
Use(imgB, 2);
```

Что такое *imgB*? Как эта переменная инициализирована? Вам нужно идти куда-то вверх по тексту программы, чтобы узнать ответы на вопросы. Потом возвращаться обратно.

Когда переменная объявляется и инициализируется рядом с местом использования, перемещаться по коду не надо.

```
Image img(IMAGE_FORMAT_JPG);
```

```
Use(img, 2);
```

Вся необходимая информация уже перед глазами! Конечно, это идеальный случай. Однако в любом случае: чем ближе будет объявление переменной и использование, тем проще читать код.

Меньше вероятность опечатки

Чем больше код содержит похожих имён переменных, тем больше вероятность допустить опечатку. Представим, что программист находится в процессе написания уже рассмотренного нами кода.

```
void foo(const bool a, const bool b, const bool c)
{
    Image imgA(IMAGE_FORMAT_PNG);
    Image imgB(IMAGE_FORMAT_JPG);
    Image imgC(IMAGE_FORMAT_GIF);
    // ....

    if (a)
    {
        // ....
        Use(imgA, 1);
    }
    // <<<< Программист находится здесь
```

Нужно написать следующий блок кода, похожий на первый. С большой вероятностью программист не станет набирать его заново, а скопирует уже имеющийся блок и начнёт править в нём имена переменных.

Желание скопировать код для этого простого примера выглядит надуманным. В реальности кода, естественно, будет больше. Программист, написав блок кода один раз, будет его переиспользовать, делая копию с последующими правками. Это называется Copy-Paste программированием.

Здесь можно пофилософствовать о том, что однотипный код следует вынести в функцию. Но это выходит за рамки рассматриваемой темы. Сейчас важно то, что после копирования блока кода в нём можно забыть что-то поправить. Это очень частый паттерн ошибки. Те, кто читал мои статьи, посвящённые проверке открытых проектов с помощью PVS-Studio, наверняка видели десятки таких примеров.

Итак, программист копирует блок:

```
if (a)
{
    // ....
    Use(imgA, 1);
}
```

```
if (a)
{
    // ....
    Use(imgA, 1);
}
```

Правит его:

```
if (a)
{
    // ....
    Use(imgA, 1);
}
```

```
if (b)
{
    // ....
    Use(imgA, 2);
}
```

При этом забывает заменить *imgA*, *imgB*. Классическая ошибка. Код с объявлением схожих переменных в начале функции провоцирует появление таких ошибок.

Безопаснее работать со вторым вариантом кода:

```
if (a)
{
    // ....
    Image img(IMAGE_FORMAT_PNG);
    Use(img, 1);
}
```

Уменьшается вероятность забыть что-то поправить. Более того, такой код ещё больше хочется вынести в отдельную функцию, и это хорошее желание.

Важно не увлекаться и думать

Мы рассмотрели причины, почему стоит объявить переменную как можно ближе к точке её использования. Однако любой полезный паттерн не отменяет необходимость думать. Ситуации могут быть разные. Рассмотрим код с циклом, который лучше оставить как есть.

```
void foo()  
{  
    Image img(IMAGE_FORMAT_PNG);  
    for (auto &png : allPngFiles)  
    {  
        img.load(png);  
        Use(img);  
    }  
}
```

Объявлять переменную *img* внутри цикла — плохая идея. Это только замедлит скорость работы программы из-за многократного создания и уничтожения объектов класса *Image*.

Вредный совет N32. Создай свой h-квест

Пишите ваши .h-файлы так, чтобы они зависели от других заголовков, и при этом не включайте их в свой заголовочный файл. Пусть тот, кто инклюдит, догадается, какие заголовки нужно заранее заинклюдить перед использованием вашего файла. Развлеките коллег квестами!

Давайте поясню на примере, что имеется в виду. Пусть в файле `my_types.h` объявлены ваши различные типы:

```
// my_types.h
#ifndef MY_TYPES_H
#define MY_TYPES_H

typedef int result_type;
struct mystruct {
    int member;
};

#endif
```

В другом заголовочном файле `my_functions.h` описаны функции, использующие эти типы.

```
// my_functions.h
#ifndef MY_FUNCTIONS_H
#define MY_FUNCTIONS_H

result_type foo(mystruct &s);
result_type doo(mystruct &s);

#endif
```

Если в `cpp`-файле требуется использовать функции, описанные в `my_functions.h`, то перед ним потребуется включить и файл с описанием типов, иначе код не скомпилируется:

```
// my.cpp
#include "my_types.h"
#include "my_functions.h"
```

В таком простом синтетическом случае это не выглядит ужасным. Однако в реальном приложении подобные несамостоятельные заголовочные файлы станут проблемой.

Во-первых, раздражает выяснять, от каких файлов зависит тот или иной заголовочный файл и что нужно перед ним включить.

Во-вторых, такой подход порождает желание включать сразу побольше наиболее часто используемых заголовочных файлов. Как говорится, «про запас». В результате в начале срр-файлов возникают длинные списки заголовочных файлов, часть из которых на самом деле не нужна и только замедляет компиляцию.

Следует придерживаться принципа: **каждый заголовочный файл должен быть самодостаточным**. Для этого внутри заголовочного файла должны быть включены все другие заголовочные файлы, от которых он зависит.

Для рассмотренного нами примера следует изменить файлы следующим образом:

```
// my_types.h
#ifndef MY_TYPES_H
#define MY_TYPES_H

typedef int result_type;
struct mystruct {
    int member;
};
#endif

-----
// my_functions.h
#ifndef MY_FUNCTIONS_H
#define MY_FUNCTIONS_H
#include "my_types.h"

result_type foo(mystruct &s);
result_type doo(mystruct &s);
#endif

-----
// my.cpp
#include "my_functions.h"
```

Вредный совет N33. C-style cast

Зачем нужны все эти `*_cast`, если есть `reinterpret_cast`? А ещё лучше и короче старый добрый C-style cast: `(Type)(expr)`.

Дело в том, что приведения в стиле языка C более «жёсткие». Их использование позволяет выполнить изменение типа, которое не имеет смысла. В случае более мягких `*_cast` преобразований такой код не скомпилируется, благодаря чему можно обнаружить ошибку ещё на этапе написания кода.

Рассмотрим синтетический фрагмент кода Windows-приложения:

```
void ff(char *);
void ff(wchar_t *);

void foo(const TCHAR *str)
{
    ff((char *)(str));           // (1) компилируется
    ff(const_cast<char *>(str)); // (2) не компилируется
    ff(const_cast<TCHAR *>(str)); // (3) компилируется
}
```

Предположим, нужно снять константность с указателя на символы типа `TCHAR`. Тип `TCHAR` может раскрываться, в зависимости от режима компиляции, в `char` или в `wchar_t`. Пусть в нашем случае он раскрывается как раз в тип `wchar_t`.

Рассмотрим три разных приведения типа:

1. Если использовать C-style приведение типа, то не только снимается константность, но и меняется базовый тип указателя. В результате выбирается неправильная функция `ff`. Код содержит ошибку, но компилируется.
2. Использован оператор `const_cast`, который должен снять только константность. Поскольку базовый тип указателя не совпадает, то возникает ошибка компиляции. Отлично! Баг выявлен ещё на этапе компиляции.
3. Исправленный код.

Давайте рассмотрим ещё один пример. Пусть у нас есть два связанных наследованием класса и один, существующий сам по себе.

```
class CDerived : public CBase {...};  
class COther {...} ;
```

Два следующих явных приведения типа компилируются одинаково:

```
CMyBase *p = get();  
CDerived *pObject;
```

```
pObject = static_cast<CDerived*>(p); // Безопасно  
                                     // до тех пор, пока мы уверены,  
                                     // что get() возвращает указатель  
                                     // на объект производного класса.
```

```
pObject = (CDerived*)(p);           // То же самое, что static_cast<>
```

Однако давайте посмотрим на немного другой код:

```
CMyBase *p = get();  
COther *pObject;
```

```
pObject = static_cast<COther*>(p); // Ошибка компиляции:  
                                     // не удаётся преобразовать.
```

```
pObject = (COther*)(p);           // Некорректный код компилируется.  
                                     // То же, что и reinterpret_cast<>.
```

Использование *static_cast* позволило обнаружить ошибку несовместимости типов ещё на этапе компиляции. Приведение же в стиле языка C маскирует ошибку.

Более того, непонятно, насколько «безобидное» или «страшное» преобразование типа скрыто за приведением типа *(COther*)(p)*.

В этом смысле *reinterpret_cast* предпочтительнее, хоть и делает то же самое.

```
pOther = reinterpret_cast<COther*>(p); // Нет ошибки компиляции.  
    // Но наличие reinterpret_cast является аналогом  
    // таблички "опасно" на трансформаторной будке.
```

Явно написанный *reinterpret_cast* кричит, что здесь происходит что-то этакое. И читающему код следует проявить повышенное внимание к этому участку кода.

C++ программистам следует придерживаться следующего принципа:

- когда необходимо выполнить явное приведение типа, используйте *static_cast*;
- или *const_cast*. Но помните, *const_cast* немного пахнет. Прежде чем использовать, стоит подумать, нет ли какой-то ошибки проектирования в программе, раз понадобилось снять константность;
- в крайнем случае используйте *reinterpret_cast*;
- никогда не используйте приведения типов в стиле C: $T(p)$ или $T(p)$;
- ах да, ещё есть *dynamic_cast*. Но он стоит отдельно. Предназначен для работы с иерархиями классов. Он безопасный, но медленный. Используйте вдумчиво и понимая, зачем он вам нужен.

Вредный совет N34. Ты властелин указателей — делай что хочешь

Что может быть плохого в том, чтобы через указатель на переменную посмотреть в соседнюю переменную?

В своей практике я встречал код приблизительно следующего вида:

```
float rgb[3];  
float alphaChannel;  
....  
for (int i = 0; i < 4; i++)  
    rgb[i] = 0f;
```



Кому-то было лень отдельно записывать ноль в переменную для альфа-канала. Он совместил её инициализацию с инициализацией элементов массива.

Плохой и опасный способ по трём соображениям:

1. Такой код не имеет никаких преимуществ. Сэкономлена одна строка кода на явную инициализацию переменной *alphaChannel*, но у меня язык не поворачивается назвать это преимуществом.
2. Запись за границу массива — это неопределённое поведение. Так что дальше уже можно и не рассуждать. Просто так нельзя делать и всё.
3. Нет гарантии, что переменная будет располагаться в памяти сразу после массива.



Другой интересный случай. Давным-давно в 2011 году я написал статью про проверку проекта VirtualDub. Но вместо того, чтобы изменить код, где происходит доступ за границу массива, автор написал, что код работает правильно. И решил оставить всё как есть. Если заинтересуетесь, то, перейдя [по ссылке, вы сможете познакомиться с комментарием целиком](#), а здесь я приведу сокращённый вариант.

Автор комментария пишет, что я включил этот пример в статью, при этом не отметив, что код работает. Вот структура и код, о которых идет речь:

```
struct ConvoluteFilterData {
    long m[9];
    long bias;
    void *dyna_func;
    uint32 dyna_size;
    uint32 dyna_old_protect;
    bool fClip;
};

long rt0=cfd->m[9], gt0=cfd->m[9], bt0=cfd->m[9];
```

Автор пишет, что этот фрагмент кода был взят из convolution фильтра, который является одним из старейших фильтров в VirtualDub. Рассматриваемая здесь структура примечательна тем, что у полей фиксированное расположение, и на это завязано множество фрагментов кода. Некоторые из них написаны на языке ассемблера, другие — динамически генерируются. Поэтому известно, что элемент после массива коэффициентов (*m*) является показателем смещения. Соответственно, по мнению автора, этот код работает так, как задумано. Если бы кто-то исправил индекс массива на 8, посчитав это «ошибкой на единицу», он бы сломал код.

Автор сознаётся, что он писал код так давно, что уже и не помнит, почему он именно так сделал. Вероятно, это стало либо результатом переписывания процедуры сборки обратно в C/C++ — ещё с тех времен, когда он создавал прототипы непосредственно в ассемблере. Либо это произошло в результате рефакторинга структуры, целью которого была замена массива, состоящего из 10 элементов, массивом коэффициентов из 9 эле-

ментов и именованным полем смещения (*bias*).

В комментарии автор пишет, что код работает так, как он написан: поле находится по правильному адресу. Наиболее вероятным источником сбоя может стать компилятор, выполняющий агрессивную оптимизацию загрузки/сохранения отдельных полей структуры. Бывает так, что хранилище и загрузка находятся очень далеко друг от друга. Структура инициализируется на этапе запуска фильтра и считывается гораздо позже в цикле показового фильтра. Компилятор Visual C++, который я использую, ничего подобного не делает, поэтому сгенерированный код работает.

В данный момент мы имеем дело с распространённой проблемой, возникающей при работе с отчётами статического анализатора. Она заключается в том, что, внося изменения для исправления теоретической ошибки, мы в процессе рискуем получить реальную ошибку. Любые изменения в кодовой базе сопряжены с риском.



У меня на всё это реакция, похожая на изображённую на картинке... Не понимаю, почему просто не поправить код, чтобы значение бралось из переменной *bias*.

```
long rt0=cfd->bias, gt0=cfd->bias, bt0=cfd->bias;
```

Я согласен, что неправильно предложил в своей статье заменить *m[9]* на *m[8]*. Ведь я не знал, как работает этот код. Однако это не значит, что лучшее решение — оставить всё как есть. Вот как много слов понадобилось, чтобы объяснить логику работы кода. Код неочевиден, а значит, должен быть переписан.

Очевидный добрый совет. Не надо объяснять, почему код написан экзотически и как он работает. Надо писать код так, чтобы этих вопросов не возникало.

Вредный совет N35. `const` — лишняя сущность

Слово `const` только место в коде занимает. Если вы не хотите менять переменную, то просто не будете её менять.

Действительно, не хочешь — не меняй. Вот только беда в том, что человеку свойственно ошибаться. Квалификатор `const` позволяет писать более надёжный код. Он защищает от опечаток и других недоразумений, которые могут возникнуть в процессе написания или рефакторинга кода.

Пример ошибки, найденной мной в проекте Miranda NG:

```
CBaseTreeItem* CMsgTree::GetNextItem(...)\n{\n    ....\n    int Order = TreeCtrl->hItemToOrder(TreeView_GetNextItem(...));\n    if (Order == -1)\n        return NULL;\n    ....\n}
```

Рука программиста дрогнула, и в условии вместо проверки, чему равна переменная, ей присваивается новое значение. Предположим, что было бы использовано ключевое слово `const`:

```
const int Order = TreeCtrl->hItemToOrder(TreeView_GetNextItem(...));\nif (Order == -1)\n    return NULL;
```

Тогда такой код просто бы не скомпилировался.



А вот с точки зрения оптимизации кода от `const` мало пользы. В некоторых статьях можно встретить рассуждение: делая переменные константами, вы помогаете компилятору, и он сможет генерировать более оптимизированный код. Это завышенные ожидания. [Ссылка ведет на соответствующую статью: «Почему `const` не ускоряет код на C/C++?»](#).

Другое дело — `constexpr`. Это ключевое слово открывает интересные возможности для проведения многих вычислений ещё на этапе компиляции кода.

Вредный совет N36. Не инициализируй

Зачем инициализировать переменные, если там и так нули? Я вот недавно не инициализировал, и там ноль был. Всё работало.

Никто не воспримет совет использовать неинициализированные переменные всерьёз. Это ошибка новичков в программировании. Конечно, профессиональные разработчики тоже допускают такие ошибки, но это не массовое явление. По крайней мере, это более редкая ошибка, чем, например, разыменование нулевого указателя.

Почему эту ошибку допускают новички? Мне кажется, есть две причины, которые сбивают их с толку.

Во-первых, значение неинициализированной переменной может случайно оказаться равно нулю. Поскольку обычно именно нулём и нужно было инициализировать переменную, то создаётся впечатление, что программа работает правильно. Естественно, это просто везение. Использование неинициализированной переменной приводит к неопределённому поведению. Просто в данном случае получается, что неопределённое поведение программы совпадает с ожидаемым правильным поведением.

Во-вторых, есть разница между локальными и глобальными переменными. Локальные переменные не инициализируются значением по умолчанию. А вот глобальные инициализируются автоматически. Всё это дополнительно вносит путаницу, когда вы ещё только учитесь программировать.

Как я уже сказал, на практике такие ошибки встречаются не часто. Это очень хорошо. Мы должны быть благодарны обучающим книгам и курсам, в которых уделяется достаточно много внимания этой проблеме. Есть причины, почему крайне важно избегать таких ошибок:

1. Иногда очень сложно найти, где же в коде прячется та-

кой баг. Например, он может воспроизводиться только у клиентов. Поиск и устранение таких багов могут быть очень дорогим занятием. Я слышал историю, когда команда разработчиков летала на другую половину земного шара к одному из клиентов, чтобы наконец понять причину неприятных сбоев. Причиной оказался неинициализированный указатель. На машинах разработчиков он всегда случайно инициализировался нулём, и всё нормально работало.

2. Поведение приложения может отличаться от запуска к запуску. Или программа может давать неверный результат, но это сложно заметить. Такое происходит, если неинициализированная переменная участвует в математических вычислениях.

Сейчас мы посмотрим на пример такой ошибки. Фрагмент кода из проекта COVID-19 CovidSim Model:

```
void CalcLikelihood(int run, std::string const& DataFile,
                    std::string const& OutFileBase)
{
    ....
    double m = Data[row][col]; // numerator
    double N = Data[row][col + 1]; // denominator
    double ModelValue;
    // loop over all days of infection up to day of sample
    for (int k = offset; k < day; k++)
    {
        // add P1 to P2 to prevent degeneracy
        double prob_seroconvert = P.SeroConvMaxSens *
            (1.0 - 0.5 * ((exp(-((double)(_I64(day) - k)) * P.SeroConvP1) + 1.0) *
            exp(-((double)(_I64(day) - k)) * P.SeroConvP2))));
        ModelValue += c * TimeSeries[k - offset].incI * prob_seroconvert;
    }
    ModelValue += c * TimeSeries[day - offset].S * (1.0 - P.SeroConvSpec);
    ModelValue /= ((double)P.PopSize);
    // subtract saturated likelihood
```

```

LL += m * log((ModelValue + 1e-20) / (m / N + 1e-20)) +
      (N - m) * log((1.0 - ModelValue + 1e-20) / (1.0 - m / N + 1e-20));
....
}

```

Вот так сразу и не скажешь, что не так с этим кодом. Формулы, что-то считается. А ведь в этом коде прячется очень коварная ошибка. Она оставалась незамеченной, пока я не проверил проект с помощью PVS-Studio: V614 [CWE-457] Uninitialized variable 'ModelValue' used. CovidSim.cpp 5412

И действительно, посмотрим на код внимательнее:

```

double ModelValue;
for (int k = offset; k < day; k++)
{
    double probab_seroconvert = ....;
    ModelValue += c * TimeSeries[k - offset].incI * probab_seroconvert;
}

```

Этот алгоритм может насчитать всё что угодно. Если в *ModelValue* окажется 0, то в результате всё будет посчитано правильно. Если в *ModelValue* будет какое-то небольшое значение, то результат будет близок к истинному. При большом значении *ModelValue* результат не будет иметь никакого отношения к реальности. Формально это вообще неопределённое поведение, и нельзя угадать, как будет работать программа.

Обратите внимание: разработчику может казаться, что он проверил алгоритм на тестовых данных и функция выдаёт корректный результат. На самом же деле корректный результат — это везение. В будущем программа может выдавать случайные результаты и вводить пользователей в заблуждение. Этим и опасны такие ошибки.

Впрочем, не буду уж совсем сгущать краски. Неинициализированные переменные хорошо детектируются не только анализаторами кода, но и даже компиляторами. Наиболее сложные ошибки помогут обнаружить динамические анализаторы кода, например MemorySanitizer.

Вредный совет N37. Доверяй всем

Модификаторы доступа `private` и `protected` для параноиков. Кому они нужны, эти поля класса?

У некоторых даже возникает соблазн написать что-то типа:

```
#define private public  
#define protected public
```



Тогда «дурацкие ограничения» не будут мешать писать код как хочется и как угодно использовать сторонние классы.

Новичку в программировании может показаться, что удобно, когда все члены класса являются публичными. Ведь тогда можно поменять любую переменную в классе или вызвать любую функцию класса.

Платой за такой подход является сложность сопровождения, модификации и отладки кода. Поскольку данные в классах изменяются в разных частях программы, очень сложно потом разобраться, как всё это работает, а уж тем более вносить какие-то изменения в логику программы. В результате начальная простота быстрого написания кода в дальнейшем оборачивается большим количеством ошибок и сложностями развития программы.

Фактически нарушается один из базовых принципов ООП — инкапсуляция. Мы уже упоминали его в главе про глобальные переменные. У публичных членов класса и глобальных переменных есть общее: они провоцируют чтение/изменение из большего количества фрагментов кода, чем связывают эти фрагменты между собой. Это усложняет сопровождение кода и поиск ошибок.

Тема инкапсуляции и её пользы выходит за рамки этого текста. Поэтому опытные программисты со словами: «И так всё понятно», — могут продолжить чтение. А новичков я отсылаю к великолепной фундаментальной книге Гради Буча «Объектно-ориентированный анализ и проектирование с примерами приложений на C++».

Вредный совет N38. Тренируйся на работе для IOCCC

Пишите код так, как будто его будет читать председатель жюри IOCCC, который знает, где вы живёте (чтобы приехать и вручить вам приз).

Это отсылка к фразе «Пишите код так, как будто поддерживать его будет склонный к насилию психопат, который знает, где вы живёте». Фраза сказана Джоном Ф. Вудсом, но ошибочно приписывается Стивену Макконнеллу, так как он упоминает её в своей книге «Совершенный код».

Обыгрывая эту фразу, даётся вредный совет — писать как можно более необычный, странный и непонятный код, в духе конкурса IOCCC.



IOCCC (International Obfuscated C Code Contest) — конкурс программирования, в котором задачей участников является написание максимально запутанного кода на языке C с соблюдением ограничений на размер исходного кода. [По ссылке можно перейти в раздел победителей и посмотреть, что за код имеется в виду.](#)

Почему непонятный код — это плохо, кажется очевидным. Но всё-таки — почему? Программисты большую часть времени тратят не на написание кода, а на его чтение. Я не могу вспомнить источник, но там говорилось, что разработчики проводят за чтением более 80 % времени. По своему опыту могу подтвердить, что это так и есть.

Соответственно, если код тяжело прочесть и понять, это сильно замедляет разработку. Отсюда же берёт своё начало идея всем в команде оформлять код единообразным способом, чтобы он был привычен для восприятия.

Вредный совет N39. Развлекайся, оформляя код

Если в C++ переносы строк и отступы незначимые, то почему бы не оформить код в виде зайчика или белочки?

Странное развлечение, которое сделает код нечитаемым для тех, кто будет его поддерживать. Почему код должен быть понятен, мы разобрали в предыдущей главе.

Впрочем, не буду уж совсем снобом и ханжой. Я иногда писал творческие или юмористические комментарии, и не вижу в этом ничего плохого. Все мы люди и испытываем эмоции. Вполне естественно, если человек иногда выплёскивает их в комментариях. Все мы знаем, как сильны иногда эти самые эмоции при программировании.

Главное — не вредить самому коду, не перебарщивать, не оскорблять кого-то.

Встретив в коде дракона, я отнесусь к этому нормально. И буду благодарен, что автор обратил моё внимание на что-то таким необычным способом.

```
//      .==.      .==.
//      // ^\ \      // ^\ \
//      // ^ ^\(\_\/) ^ ^\ \
//      // ^ ^ ^/6 6\ ^ ^ ^ \ \
//      // ^ ^ ^/( .. ) ^ ^ ^ \ \
//      // ^ ^ ^/\ | v»»v |/\ ^ ^ ^\ \
//      // ^ ^\ \ /  `~~` \ \/\ ^ ^\ \
//      -----
```

Вредный совет N40. У каждого свой стиль

Выравнивание и единый стиль не дают раскрыться вашей индивидуальности и креативности. Это притеснение свободы личности и самовыражения. Каждый должен оформлять код так, как ему нравится.

Ладно-ладно, два предыдущих совета были эксцентричны. Что по поводу адекватного самовыражения в именовании сущностей и оформлении кода?

Этого не стоит делать по следующим причинам:

1. Собственный код, написанный в привычном индивидуальном стиле, проще читать и понимать. Вот только над проектами трудятся команды. В результате всем будет постоянно тяжело, так как придётся работать с кодом, написанным по-разному. Пустая трата времени и умственных усилий. Если выбрать один стиль, то вначале все тоже будут испытывать дискомфорт. Но постепенно к стилю привыкнут, и он станет удобным для чтения и написания. Повысится общая производительность команды.
2. Конфликт стилей. Часто нужно поправить только какой-то фрагмент кода. Если каждый делает это на свой лад, то код будет выглядеть как лоскутное одеяло. Получается, что вообще нет никакого стиля, сплошная анархия. Если же переделывать чужой код, с которым приходится работать, под себя, то это будет отнимать много времени. С точки зрения автора первоначального кода это вообще вредное действие. В общем, это почва для ненужных личностных конфликтов. Конечно, правя чужой код, можно подстраиваться под стиль, в котором он написан. Но, во-первых, это сложно, а во-вторых, вы всё равно точно не знаете, какой именно стиль у каждого из коллег.
3. То, что свой стиль нравится человеку, вовсе не значит, что он хороший. Есть множество приёмов, когда напи-

сание кода определённым образом помогает избежать ошибок. Примером может являться форматирование кода таблицей, которое мы рассматривали в главе N18. Поэтому опытным членам команды полезно совместно разработать и внедрить единый для всех стандарт кодирования.

Надеюсь, я был убедителен. Очень полезно в команде иметь стандарт кодирования. За основу можно взять, например, Google C++ Style Guide или любой другой. Затем адаптировать для себя.

Дополнительным источником для вдохновения могут стать:



- Уже не раз упомянутая здесь книга «Совершенный код» С. Макконнелла (ISBN 978-5-7502-0064-1).
- Ален И. Голуб. «Веревка достаточной длины, чтобы... выстрелить себе в ногу». М.: Пер. с англ. В. Зацепин, 2001. — 244 с. Мне лично она не очень понравилась, но я не могу про неё не вспомнить. Это классика.
- Рекомендую заглянуть по [ссылке на сайт Standard C++ Foundation в раздел «FAQ: Coding Standards»](#). Много полезных мыслей и ссылок.

Со стилем программирования и стандартом кодирования связана тема форматирования кода. Если то, как именуются сущности, можно проверять на обзорах кода, то с форматированием могут помочь различные утилиты. Главное — договориться, как ваша команда форматирует код, а дальше — дело техники. Наиболее подходящим на данный момент инструментом, пожалуй, является clang-format.

Вредный совет N41. Перегрузи всё

Перегрузите как можно больше операторов (в том числе и не арифметических) для как можно большего количества типов. Придавая операторам другой смысл, вы приближаетесь к созданию своего диалекта языка. Свой язык — это прикольно. А если ещё и макросы к делу подключить...

Перегрузка операторов — это более изящный вызов функций. Например, перегрузка позволяет использовать операторы `+` и `=` для строк. Это помогает писать изящный код конкатенации строк:

```
result = str1 + str2;
```

Согласитесь, это более красиво, чем:

```
result.assign(str1.strcat(str2));
```

Использовать перегрузку операторов следует только тогда, когда это упростит написание кода, как было показано в примере. Неуместное же использование, наоборот, затруднит чтение. Если действие перегруженных операторов интуитивно понятно, это хорошо. Если нет — это вредно.

Пусть A , B , C — это экземпляры класса матрицы. Тогда следующий код очевиден без пояснений:

```
A = B * C;
```

Другое дело:

```
A = B && C;
```

Этот код уже интуитивно непонятен. Конечно, можно посмотреть, что делает оператор `&&` с матрицами. Если это какая-то частая операция, действительно есть смысл использовать такую перегрузку для более компактного написания кода. Однако, я думаю, вы понимаете, что, чем больше неочевидных действий выполняют перегруженные операторы, тем сложнее понимать код. Перегрузка начинает в этом случае усложнять код, а не упрощать.

Вредный совет N42. Не верь в эффективность `std::string`

Универсальный `std::string` — это неэффективно. Можно ловчее и эффективнее использовать `realloc`, `strlen`, `strncat` и так далее.

То, что производительность программы можно существенно увеличить, отказавшись от класса `std::string`, является мифом. Но этот миф имеет под собой основание.

Дело в том, что раньше распространённые реализации `std::string` действительно оставляли желать лучшего. Так что, пожалуй, мы имеем дело даже не с мифом, а с устаревшей информацией.

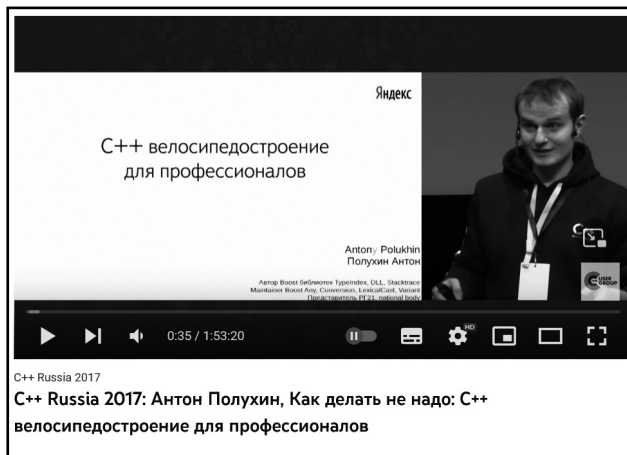
Поделюсь собственным опытом. Начиная с 2006 года мы разрабатываем статический анализатор PVS-Studio. Правда, в 2006 году он ещё назывался Viva64. Изначально в анализаторе мы широко использовали как раз стандартный класс `std::string`.

Шло время. Анализатор развивался, в нём появлялось всё больше диагностик, и с каждой версией он работал всё медленнее. Пришло время задуматься над оптимизацией кода. Одним из узких мест, на которое указал профилировщик, оказалась работа со строками. И тогда настало время цитаты: «В любом проекте рано или поздно появляется свой собственный класс строки». К сожалению, я не помню, ни откуда эта цитата, ни момент, когда именно это произошло. Кажется, это был 2008 или 2009 год.

Анализатор в процессе своей работы создаёт большое количество пустых или очень коротких строк. Мы написали свой собственный класс `vstring`, эффективно выделяющий память для таких строк. С точки зрения публичного интерфейса наш класс повторял `std::string`. Собственный класс строки повысил скорость работы анализатора приблизительно на 10 %. Крутое достижение!

Этот класс строки служил нам долгие годы, пока на конференции C++ Russia 2017 я не побывал на докладе Антона Полухина «Как делать не надо: C++ велосипедостроение для профессионалов».

В нём он рассказал, что уже много лет класс `std::string` хорошо оптимизирован. А те, кто используют свой собственный класс строки, являются ретроградами и динозаврами. Очень хороший интересный доклад. [Рекомендую посмотреть](#).



Антон разобрал в своём докладе, какие оптимизации сейчас используются в классе `std::string`. Например, конструктор перемещения. Меня же больше всего заинтересовала Small String Optimization [см. термин «SSO»].

Остаться динозавром не хотелось. Мы провели эксперимент по обратному переходу с самодельного класса `vstring` к `std::string`. Для начала мы просто закомментировали класс `vstring` и написали `using vstring = std::string;`. Благо после этого потребовались незначительные правки кода в других местах, так как интерфейсы нашего класса всё ещё почти полностью совпадали с `std::string`.

И как изменилось время работы? Никак! Это значит, что для нашего проекта универсальный `std::string` стал точно так же эффективен, как наш собственный специализированный класс, спроектированный нами с десятков лет назад. Здорово! Можно

убрать один велосипед из кодовой базы проекта.

Однако мы говорили о классах. А во вредном совете предлагается спуститься на уровень функций языка C. Вряд ли с этими функциями получится написать код, который будет быстрее и надёжнее кода, использующего класс строки.

Во-первых, обработка C-строк (нуль-терминированных строк) провоцирует частое вычисление их длины [см. термин «C-строки»]. Не храня отдельно размер строк, сложно написать высокопроизводительный код. А если длина хранится, то мы вновь приближаемся к аналогу строкового класса.

Во-вторых, сложно написать надёжный код с использованием таких функций, как *realloc*, *strncat* и так далее. По опыту обзора ошибок в различных проектах можно констатировать, что код, использующий эти функции, прямо-таки «притягивает» к себе ошибки. Чтобы не быть совсем голословным, приведу пару примеров.

Проект Wine:

```
section[strlen(section)] = '\\0'; /* remove last backslash */
```

Программист запутался в вычислениях и на самом деле записал терминальный ноль сам в себя. Т. е. строка не изменится. Согласитесь, при использовании *std::string* подобную ошибку сделать сложнее:

```
section.pop_back(); // remove last backslash
```

Работа с сырыми строками ещё и провоцирует возникновение в коде потенциальных уязвимостей. Наверняка вы слышали о проблемах переполнения буфера. Они могут возникать, например, при работе с функцией *strncat*. Многие программисты используют эту функцию вместо *strcat*, чтобы предотвратить выход за границу буфера, но делают это неправильно.

Проект ReactOS:

```
char CmdLine[MAX_PATH];  
....  
for (i=1; i<argc; i++)
```

```
{
    strncat(CmdLine, " ", MAX_PATH);
    strncat(CmdLine, argv[i], MAX_PATH);
}
```

Третий аргумент функции *strncat* задаёт не размер буфера, а максимальное количество символов, которые можно добавить. Поэтому нужно из размера буфера вычитать количество уже имеющихся в нём символов:

```
for (i=1; i<argc; i++)
{
    strncat(CmdLine, " ", MAX_PATH - strlen(CmdLine));
    strncat(CmdLine, argv[i], MAX_PATH - strlen(CmdLine));
}
```

Код стал более медленным. Но, возможно, хотя бы правильным? Нет. Я вас обманул, чтобы продемонстрировать, как сложно работать с C-строками. Код всё ещё неправильный.

Если вся строка уже заполнена, выражение *MAX_PATH - strlen(CmdLine)* вернёт единицу. В конце строки находится терминальный ноль, а значит, длина строки на один символ меньше размера буфера. В результате к строке может быть прибавлен ещё один символ, а терминальный ноль будет записан уже за границу буфера.

Правильный вариант:

```
for (i=1; i<argc; i++)
{
    strncat(CmdLine, " ", MAX_PATH - strlen(CmdLine) - 1);
    strncat(CmdLine, argv[i], MAX_PATH - strlen(CmdLine) - 1);
}
```

Так что если воспользоваться вредным советом, то вызовом для программиста будет не столько создание быстрого кода, сколько надёжная работа этого кода.

Вредный совет N43. Максимально откладывая использование нового C++ стандарта

Поддерживай максимально старый стандарт C++ для совместимости.

Существуют проекты, которые должны компилироваться под большое количество разнообразных платформ. Тогда действительно приходится ограничивать себя версией языка, которая поддерживается всеми используемыми компиляторами.

Но такое бывает редко, поэтому нет смысла откладывать переход на новую версию языка. Язык C++ с каждой версией позволяет писать всё более лаконичный, безопасный и эффективный код.

Некоторые программисты скажут, что они придерживаются старого стандарта на всякий случай. Вдруг их проект в дальнейшем понадобится для платформ, где компилятор будет не таким современным. Программисты вообще любят делать много лишнего про запас. Такой подход чаще приносит больше вреда (пустой траты времени), чем пользы.

Это «может быть» с большой вероятностью никогда не произойдёт. Если же это случится, то, скорее всего, основные проблемы будут вовсе не в том, что какие-то конструкции языка C++ не будут компилироваться. Проблемы будут с библиотеками, с необходимостью адаптировать графический интерфейс, с оптимизацией скорости работы и так далее. Будет большое количество сложностей, на фоне которых заботиться именно о версии стандарта языка имеет мало смысла.

Вредный совет N44. Переиспользование переменных

Максимально переиспользуйте переменные. Выделяйте память один раз, а затем используйте её повторно. Если вам нужно три целых числа в функции, создайте одно, назовите его чем-то универсальным, например *x*, а затем используйте *x* для всех трёх.

Очень вредный совет, усложняющий понимание программы и тем самым провоцирующий ошибки.

Во-первых, имя переменной в любом случае будет неудачным:

- если дать переменной осмысленное название, то оно будет логичным и понятным только в одном сценарии использования. При использовании этой переменной для других целей имя переменной будет только сбивать с толку. Если переменная называется *size*, то нормально в ней хранить размер массива, но странно потом её использовать для хранения температуры, полученной с датчика;
- если, как предлагается в совете, назвать переменную универсально (*a*, *t*, *x*, *foo*), то она не даст никакой информации при чтении программы, и код будет сложен для понимания.

Получается, что этот совет противоречит принципам самодокументируемой программы. Имена переменных лишаются смысла и не помогают понимать, как работает программа.

Примечание. Проблему можно обойти, создав различные ссылки на одну и ту же переменную. Однако в итоге так ещё проще запутаться. Случайно можно начать одновременно использовать одну и ту же переменную для записи разных по смыслу значений, перетирая другие, ещё нужные данные.

Во-вторых, использование одной переменной для разных

целей провоцирует появление ошибок. К счастью, так редко кто делает на практике, и у меня нет какой-то статистики, подтверждающей это утверждение. Я высказал мнение на основании своей многолетней экспертизы разбора ошибок в различных открытых проектах. Интуиция подсказывает, что, если так писать, плотность ошибок в коде существенно возрастёт.

В-третьих, совет противоречит общепризнанному принципу делать область видимости переменной минимальной:

- следует объявлять переменную как можно ближе к точке её использования. Идеально, если объявление переменной совмещается с её инициализацией;
- желательно, чтобы время существования переменной заканчивалось сразу после того, как она перестаёт быть нужна.

Всё это позволяет минимизировать количество ошибок, когда из-за опечаток используется не та переменная. Для максимальной локальности переменных, даже появилась возможность объявлять их в условии, чтобы их область видимости ограничивалась телом условного оператора:

```
if (auto p = dynamic_cast<Derived*>(bp))  
    p->df();
```

А как же экономия памяти?

Сомнительный способ экономить память. Если это переменные типа *int*, создаваемые на стеке, то какой-то существенной экономии не получится. Более того, если время жизни переменной кончилось, компилятор может разместить новую переменную в тех же ячейках памяти на стеке. Или компилятор решит использовать для хранения переменной не стек, а один из регистров процессора. В общем, он может самостоятельно проделать предлагаемую оптимизацию по экономии памяти.

Что с массивами, создаваемыми в куче? Это, конечно, более тяжёлые операции, и оптимизации компилятора, скорее

всего, здесь не помогут. Допускаю, что в целях оптимизации может иметь смысл не выделять новый массив, а переиспользовать существующий.

Однако это какой-то редкий случай. Я не смог вспомнить пример из своей практики программирования, когда у меня был соблазн переиспользовать массив для других целей. Просто как-то не нужно это было.

В любом случае не стоит прибегать к такому способу! Он крайне плохо сказывается на простоте понимания и сопровождения кода. Вред перевешивает эфемерный выигрыш.

Вредный совет N45. Отвечай в комментарии на вопрос «что?»

Пишите комментарии, чтобы точно объяснить, что делает строка кода. А почему это делается — настоящему программисту и так понятно.

Что делает строка кода, почти всегда и так понятно. Рассмотрим гиперболизированный пример:

```
i++; // add 1 to i
```

Ёлки-палки, любому человеку, знакомому с C++ даже на базовом уровне, и так понятно, что здесь происходит увеличение переменной на единицу. Написанный комментарий не несёт никакой полезной информации и даже вредит, так как загромождает код.

Комментарии должны отвечать не на вопрос «что делаем?», а на вопрос «зачем/почему делаем?».



Здесь очень уместна концепция «Золотого круга», предложенного Саймоном Синеком. Она не про комментарии, но очень хорошо к ним подходит. Саймон как раз предлагает по возможности начинать с вопроса «зачем?». [Ссылка](#) ведёт на видеозапись его выступления «Start with why — how great leaders inspire action» на конференции TED. Она даст пищу для ума: подумать не только над содержимым комментариев, но и над тем, как рассказывать другим о своей работе.

Все комментарии можно разделить на три группы по вопросу, на который они отвечают:

- «Что?» — худший вариант, который добавляет мало информации к написанному коду. Единственное осмысленное применение, если перед нами какая-то сложная конструкция и мы хотим пояснить, что она делает. По идее, в самодокументируемом коде таких комментариев вообще не должно быть;
- «Как?» — средний вариант. Такие комментарии могут

быть полезны, так как поясняют, как мы что-то делаем. Обычно в них описываются какие-то детали реализации. Например, как мы что-то вычисляем, используя численный метод;

- «Зачем/почему?» — наиболее полезные высокоуровневые комментарии. Они объясняют суть происходящего, не вдаваясь в детали реализации.

Давайте теперь рассмотрим реальный пример правильного полезного комментария.

Однажды анализатор PVS-Studio обратил моё внимание на этот фрагмент кода, выдав предупреждение: V575 [CWE-628] The 'memcpy' function doesn't copy the whole string. Use 'strcpy' / 'strcpy_s' function to preserve terminal null. qplaintestlogger.cpp 253.

Вот этот код:

```
const char *msgFiller = msg[0] ? " " : "";
QTestCharBuffer testIdentifier;
QTestPrivate::generateTestIdentifier(&testIdentifier);
QTest::qt_asprintf(&messagePrefix, "%s: %s%s%s\n",
                  type, testIdentifier.data(), msgFiller, msg,
                  failureLocation.data());

// In colored mode, printf above stripped our nonprintable
// control characters. Put them back.
memcpy(messagePrefix.data(), type, strlen(type));

outputMessage(messagePrefix.data());
```

Обратите внимание на вызов функции *memcpy*. Код вызывает два вопроса:

1. Зачем что-то записывается в буфер, содержимое которого было только что сформировано с помощью *printf*-подобной функции?
2. Точно не ошибка, что не копируется терминальный ноль? Это как раз и не нравится анализатору.

К счастью, комментарий сразу всё проясняет. Нужно восстановить некие непечатаемые символы.

Перед нами нужный и полезный текст. Он отвечает на вопрос, зачем что-то делается. Это отличный образец комментария, поясняющего неочевидный момент в коде. Можно приводить его как пример в обучающих статьях.

Для сравнения рассмотрим другой фрагмент кода из этого же файла:

```
char buf[1024];

if (result.setByMacro) {
    qsnprintf(buf, sizeof(buf), "%s%s%s%s%s\n", buf1, bufTag, fill,
               buf2, buf2_, buf3);
} else {
    qsnprintf(buf, sizeof(buf), "%s%s%s%s\n", buf1, bufTag, fill, buf2);
}

memcpy(buf, bmtag, strlen(bmtag));
outputMessage(buf);
```

Здесь забыли сделать аналогичный комментарий. И картина радикально меняется. Этот код способен ввести в замешательство нового члена команды, который будет его сопровождать или модифицировать. Совершенно непонятно, зачем нужен этот *memset*. Более того, непонятно, почему в начало строки печатается содержимое некоего буфера *buf1*, а затем в начало строки помещается содержимое буфера *bmtag*. Так много вопросов, и так мало ответов.

Вредный совет N46. Больше многопоточности

Многопоточность всегда эффективнее однопоточности.

Важно, что понимается под эффективностью. Если речь идёт о скорости работы программы в целом, то можно согласиться, что распараллеленный код обрабатывает данные быстрее, чем однопоточный.

Если говорить о суммарных вычислительных ресурсах, затраченных на решение задачи, то более эффективен код без распараллеливания. В нём отсутствует дополнительный функционал, необходимый для синхронизации потоков. Не теряется время, когда один поток что-то ждёт от другого потока. И так далее. В общем, код проще и быстрее.

В этот момент кажется, что утверждение — многопоточность лучше — верное. Пользователю неинтересно, сколько суммарно работают все потоки приложения. Интересно время, за которое будет решена задача.

Не всё так однозначно. Дело в том, что есть ещё такое понятие, как уровни распараллеливания. Чем крупнее «единицы распараллеливания», тем, скорее всего, эффективнее может работать приложение.

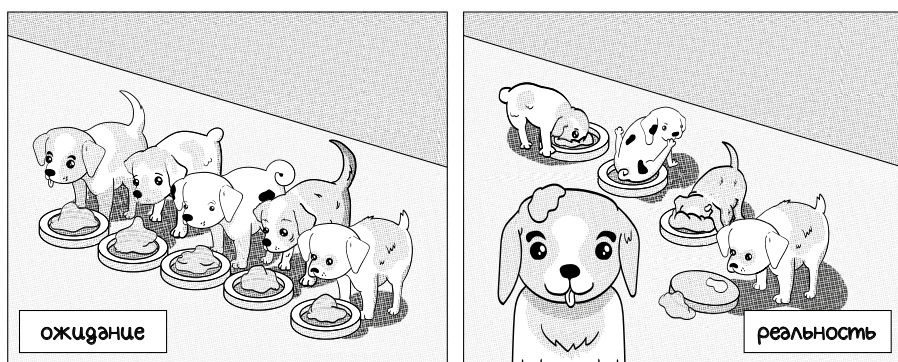
Можно привести следующий грубый пример. Пусть у нас есть многоядерный процессор. Стоит задача сконвертировать 1000000 изображений из jpeg формата, например, в png. В этом случае несколько параллельно работающих однопоточных программ справятся с задачей быстрее, чем программа, которая умеет распараллеливать внутри себя конвертацию картинок. Причина: однопоточные программы проще, и они не тратят время на синхронизацию.

Это ещё не всё. Реализовать высокоуровневый параллелизм легче. Согласитесь, нет ничего сложного, когда несколько эк-

земпляров запущенной программы берут из общего пула картинки и обрабатывают их. Несложно написать менеджер для всего этого.

Намного сложнее написать код, который будет делить изображение на части, параллельно их обрабатывать, а потом собирать обработанные части вместе. А где сложность, там больше вероятность допустить ошибку!

ПАРАЛЛЕЛЬНОЕ ПРОГРАММИРОВАНИЕ



Приведу ещё один пример разумного подхода. PVS-Studio распараллелен на уровне анализа C++ файлов. Т. е. для анализа каждого C++ файла запускается отдельный процесс, который обрабатывает данные в одном потоке. Это очень эффективно и просто. Сколько есть процессорных ядер, столько экземпляров анализатора и запускается. В общем-то так и параллельная компиляция кода работает.

Но подождите, а если нужно проверить только один файл? Распараллеленный анализатор справился бы с этим быстрее, чем однопоточный.

Да, но это не имеет практического смысла. Один файл и так проверяется быстро. Для пользователя не будет разницы с практической точки зрения. Можно даже грубо прикинуть числа.

PVS-Studio использует внешний препроцессор и никак не может повлиять на скорость его работы. Препроцессор работает последовательно. Предположим, препроцессирование выполняется за 2 секунды.

Анализатор делает разбор кода и поиск ошибок, предположим, тоже за 2 секунды.

Добавим 1 секунду на разные дополнительные операции. Требуется запросить данные о файле у среды Visual Studio. Запустить препроцессор, анализатор. Открыть окно в среде разработки и отобразить в нём предупреждения.

Итого, пусть анализ файла выполняется за 5 секунд.

Допустим, мы проведём колоссальную работу и распараллелим ядро C++ анализатора. Тогда анализ файла будет выполняться не за 2, а за 1 секунду. Вряд ли мы сможем получить большой выигрыш, сколько бы потоков ни запускали. Задача разбора на лексемы, построение синтаксического дерева, его обход и так далее очень плохо параллелятся. Ведь нельзя разбирать, скажем, вторую часть файла, не зная, какие типы объявлены в первой части.

В итоге получится 4 секунды вместо 5. С точки зрения пользователя, разница небольшая.

И это единственный сценарий, когда выигрыш заметен. В сценариях, когда обрабатывается много файлов, всё будет максимально быстро.

Зато за ускорение при анализе одного файла придётся заплатить колоссальным усложнением алгоритмов разбора и анализа кода. Стоит ли оно того? Нет.

Как видите, распараллеливание бывает разное, и не всегда оно одинаково полезно.

Вредный совет N47. Чтение книг уже неактуально

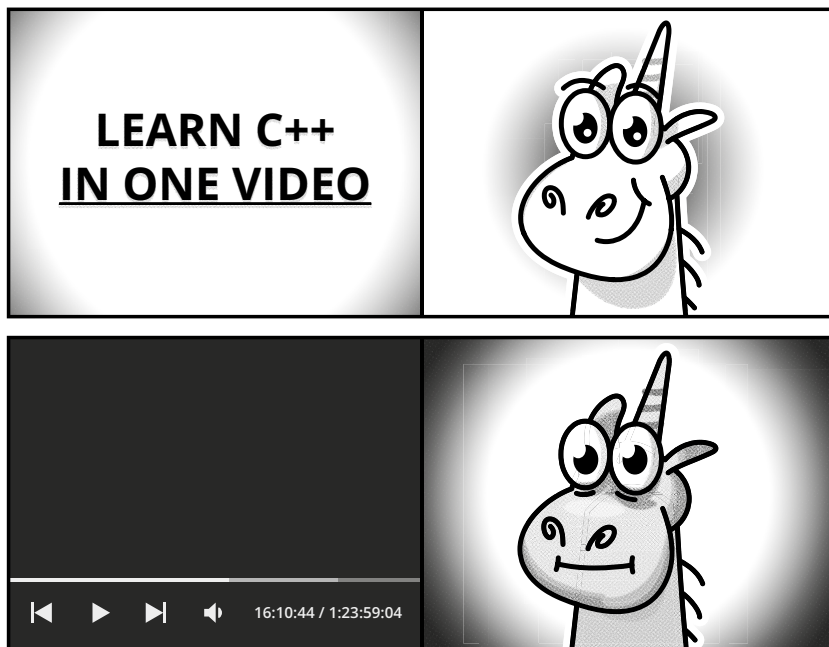
Чтение фундаментальных книг по C++ — пустая трата времени. Достаточно обучающих видео, во всём остальном поможет Google.

Я большой сторонник чтения книг по программированию. При этом я не отрицаю пользу от обучающих видео и других способов обучения. Я сам могу с удовольствием посмотреть какой-то доклад с C++ конференции.

Однако у обучающих видео есть два недостатка:

1. Они не дают остановиться и обдумать увиденное и услышанное. Можно поставить на паузу, отмотать, но всё это не так удобно и естественно, как при чтении книги. Изучая C++ в книгах, я нередко перечитывал абзацы или повторно медленно изучал код примеров. Делал паузу, чтобы обдумать. В общем, я о вдумчивом чтении. В случае видео всё это тоже можно делать, но как-то не так получается. Ты смотришь и смотришь дальше, даже если что-то было не совсем понятно. Иногда ещё и ускоренно.
2. Обучающие видео не дают такой же целостной картины, как книги. Я не представляю, как можно удобоваримо запихать в видео такой же объём знаний, которые, например, изложены на 1200 страницах книги Бьёрна Страуструпа «Язык программирования C++» (4-е издание).

Если подойти к видео с обстоятельностью книг, то будет как-то так...



Тяжёлый формат для восприятия. На мой взгляд, видео могут дополнять полученные знания. Они полезны, когда ложатся поверх фундамента знаний, полученных из книг. В противном случае видео формируют неглубокую разорванную картину знаний. Кажется, что человек разбирается в программировании, но такие «лоскутные» знания быстро выйдут боком и ему, и работодателю.

Я, кстати, встречал таких «программистов». Первое поверхностное впечатление — они очень крутые. Они могут быстро что-то создать на хакатоне, используя какую-то модную AI-библиотеку, или удивить какой-то немереной красотой, используя где-то игровой движок. Это кажется какой-то магией, особенно если ты сам не работал с этими библиотеками. Вот только всё это рано или поздно разбивается о незнание каких-то базовых алгоритмов или особенностей языка программирования.

А в чём проблема, если что-то человек не знает, но тем не менее в целом что-то делает? В неэффективности и ограниченности возможностей. Из-за незнания человек может использо-

вать странные и крайне неоптимальные подходы. Или может программировать методом поиска готовых решений в Google или на Stack Overflow. Но «наугадить» можно не всё, и рано или поздно человек просто не сможет решить стоящие перед ним задачи.

В общем, я подвожу вас к относительно свежей проблеме «Google-программистов». Доступность обучающих видео «Как сделать X» и готовых проектов/решений в интернете порождает программистов, которые не умеют решать задачи самостоятельно. А ведь рано или поздно они сталкиваются с такими задачами.



Интересное описание этой проблемы я встретил в [статье «Гугл-программисты»](#). Как идиот набрал на работу идиотов». Предупреждаю, это провокационная статья, которая собрала много критики. Однако, мне кажется, читатели зря цепляются к словам и грубому стилю повествования. Эта статья — гротеск! В ней проблема специально подаётся жёстко. А без этого, возможно, статья и не получила бы такого внимания аудитории и осталась незамеченной.

Кстати, я на собеседовании спрашиваю, какие книги по программированию на C++ читал соискатель. Это не значит, что мы не возьмём человека, если он ответит, что не читал книг, но это повод насторожиться.

Ещё мы просим написать на бумаге пару небольших функций, например «посчитать количество нулевых бит в переменной типа *unsigned int*». Это сразу очень многое говорит о кандидате. Впрочем, нас больше интересуют всесторонние теоретические знания языка C++ в силу специфики нашей работы по созданию анализатора кода.

P.S. Может показаться, что я против того, чтобы что-то поискать в интернете. Вовсе нет. Умение найти информацию — один из важнейших скиллов любого программиста. Просто умение поиска не заменяет систематизированные знания, умения писать и отлаживать сложный код.

Вредный совет N48. `printf(str);`

Нужно распечатать строку? Что тут думать! Берёшь и печатаешь её с помощью *printf*!

Когда нужно распечатать или, например, записать строку в файл, многие программисты, не задумываясь, пишут код следующего вида:

```
printf(str);  
fprintf(file, str);
```



Это очень опасные конструкции. Дело в том, что если внутри строки каким-то образом попадёт спецификатор форматирования, то это приведёт к непредсказуемым последствиям.

Допустим, мы хотим использовать следующий код для распечатки имени файла:

```
printf(getname().c_str());
```

Если у файла будет имя «file%s%i%s.txt», то программа может упасть или распечатать белиберду.

Но это только полбеды. Фактически такой вызов функции — это самая настоящая потенциальная уязвимость, используя которую, можно пытаться атаковать программы. Подготовив особенным образом строки, можно распечатать приватные данные, хранимые в памяти. Давайте разберём это подробнее.

Про странный код примера

Код, который будет представлен далее, покажется вам странным. Так оно и есть. Не так просто написать программу, чтобы потом её атаковать. Дело в оптимизации, которую производит компилятор. Получается, что если написать слишком простую программу, то компилятор создаёт такой код, что ломать там нечего. Он использует регистры, а не стек для хранения данных, встраивает функции и тому подобное.

Можно написать код с лишними действиями и циклами, чтобы компилятору не хватило свободных регистров и он начал помещать данные в стек. К сожалению, тогда пример получается слишком большой и запутанный.

Представленный далее текст программы является компромиссом между сложностью и необходимостью не дать компилятору «схлопнуть в ничто» слишком простой код. Признаюсь, немного я себе всё равно помог. Я отключил некоторые виды оптимизации, используемые в Visual C++. Во-первых, был отключен ключ /GL (Whole Program Optimization). Во-вторых, я использовал атрибут `__declspec(noinline)`.

Прошу прощения за длинное вступление. Хотелось пояснить неуклюжесть программного кода и то, что в нём смешан C и C++. Так что не тратьте силы на мысли, что код можно написать лучше. Я знаю, что можно. Но не получается сделать код одновременно и коротким, и достаточным для демонстрации уязвимости.

Демонстрационный код

Рассмотрим небезопасной код, использующий `printf(name)`.

```
const size_t MAX_NAME_LEN = 60;
enum ErrorStatus {
    E_ToShortName, E_ToShortPass, E_BigName, E_OK
};

void PrintNormalizedName(const char *raw_name)
{
    char name[MAX_NAME_LEN + 1];
    strcpy(name, raw_name);

    for (size_t i = 0; name[i] != '\0'; ++i)
        name[i] = tolower(name[i]);
    name[0] = toupper(name[0]);

    printf(name);
}
```

```
ErrorStatus IsCorrectPassword(
    const char *universalPassword,
    BOOL &retIsOkPass)
{
    string name, password;
    printf("Name: "); cin >> name;
    printf("Password: "); cin >> password;
    if (name.length() < 1) return E_ToShortName;
    if (name.length() > MAX_NAME_LEN) return E_BigName;
    if (password.length() < 1) return E_ToShortPass;

    retIsOkPass =
        universalPassword != NULL &&
        strcmp(password.c_str(), universalPassword) == 0;
    if (!retIsOkPass)
        retIsOkPass = name[0] == password[0];

    printf("Hello, ");
    PrintNormalizedName(name.c_str());

    return E_OK;
}

int _tmain(int, char *[])
{
    _set_printf_count_output(1);
    char universal[] = "_Universal_Pass_!";
    BOOL isOkPassword = FALSE;
    ErrorStatus status =
        IsCorrectPassword(universal, isOkPassword);
    if (status == E_OK && isOkPassword)
        printf("\nPassword: OK\n");
    else
        printf("\nPassword: ERROR\n");
    return 0;
}
```

Функция *_tmain* вызывает функцию *IsCorrectPassword*. Если пароль верен или совпадает с магическим словом «_Universal_Pass_!», то программа выводит строку «Password: OK». Цель атак — добиться, чтобы программа выводила именно эту строку.

Функция *IsCorrectPassword* запрашивает у пользователя имя и пароль. Пароль считается корректным, если он совпадает с переданным в функцию магическим словом. Также он корректен, если первая буква пароля совпадает с первой буквой имени.

Вне зависимости от того, введён правильный пароль или нет, программа приветствует пользователя. Для этого вызывается функция *PrintNormalizedName*.

В функции *PrintNormalizedName* — всё самое интересное. Именно в ней находится критикуемый *printf(name)*. Подумайте, как с помощью этой строчки можно обмануть программу. Если знаете как, то дальше можно не читать.

Что делает функция *PrintNormalizedName*? Она печатает имя, сделав первую букву заглавной, а остальные маленькими. Например, если ввести имя «andREy2023», то она распечатает «Andrey2023».

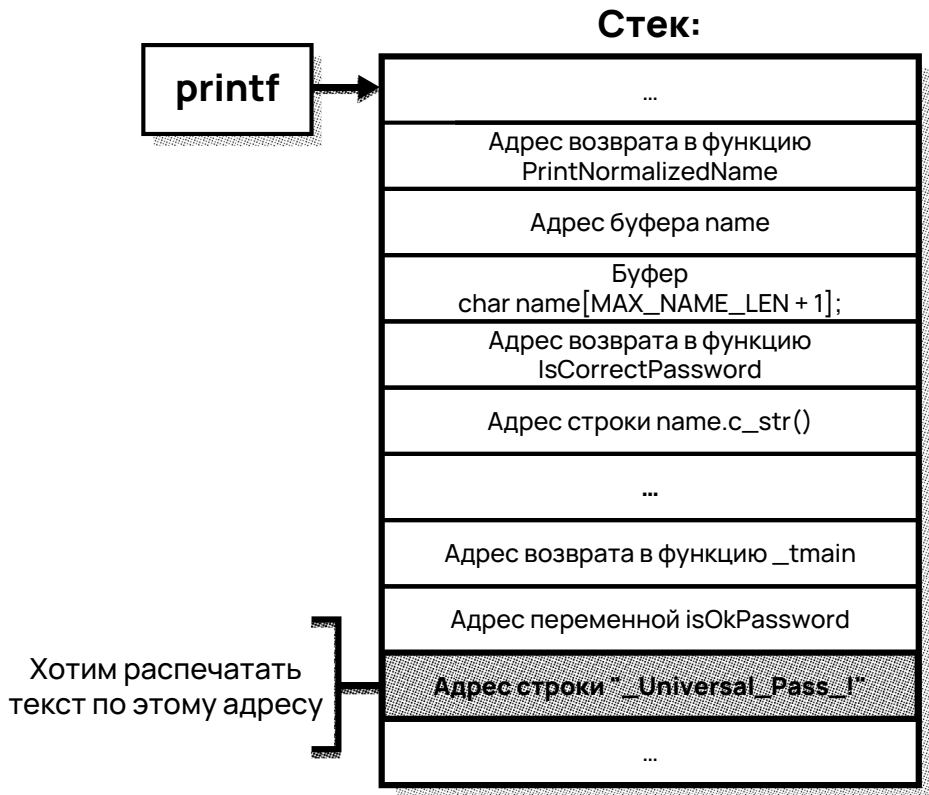
Первая атака

Предположим, мы не знаем правильный пароль. Но знаем, что где-то есть некий магический пароль. Попробуем его поискать, используя *printf*. Если адрес этого пароля есть где-то в стеке, то у нас есть шанс на успех.

Функция *printf* относится к семейству функций с переменным количеством аргументов. Работают такие функции так. В стек записывается произвольное количество данных. Функция *printf* не знает, сколько данных записано в стек и какой у них тип. Она руководствуется исключительно строкой форматирования. Если написано *%d%s*, значит из стека следует извлечь одно значение типа *int* и один указатель. Так как функ-

ция *printf* не знает, сколько аргументов ей передали, то она может заглянуть глубже в стек и распечатать данные, которые никакого к ней отношения не имеют. Из-за этого возникнет неопределённое поведение. На практике с высокой вероятностью это приведёт к access violation или к распечатке мусора. Этим распечатанным мусором можно воспользоваться.

Рассмотрим, как может выглядеть стек в момент, когда мы вызываем функцию *printf*:



Вызов функции *printf(name)* имеет только один аргумент, который является строкой форматирования. Это значит, что если вместо имени мы введём *%d*, то распечатаем данные, которые лежат в стеке до адреса возврата в функцию *PrintNormalizedName*. Попробуем:

Name: %d

Password: 1

Результат:



Попробуем эту строчку в качестве волшебного пароля:

Name: Aaa

Password: Universal_Pass_!

Hello, Aaa

Password: OK

Ура! Мы смогли найти и вывести на экран приватные данные, к которым программа не планировала давать нам доступ. Причём, обратите внимание, для этого нет необходимости иметь доступ к самому двоичному коду программы. Достаточно усердия и настойчивости.

Выводы по первой атаке

Подобный способ получения приватных данных следует обдумать более широко. При разработке программ, содержащих функции с переменным количеством аргументов, подумайте, существуют ли ситуации, когда через них могут утечь данные во внешний мир. Это может быть лог-файл, пакет, передаваем-

мый по сети, и так далее.

В рассмотренном случае атака стала возможна из-за того, что на вход функции *printf* поступает строка, которая может содержать управляющие команды. Чтобы этого избежать, было достаточно написать так:

```
printf("%s", name);
```

Вторая атака

Вы знаете, что функция *printf* может модифицировать память? Возможно, вы про это читали в книге, но забыли. Речь идет о спецификаторе *%n*. Он позволяет записать по указанному адресу количество символов, которые уже распечатала функция *printf*.

Если честно, атака, основанная на спецификаторе *%n*, носит исключительно исторический характер. Например, начиная с Visual Studio 2005, возможность использования *%n* по умолчанию отключена. Чтобы провести эту атаку, мне пришлось явно разрешить этот спецификатор. Вот это магическое действие:

```
_set_printf_count_output(1);
```

Чтобы стало понятнее, приведу пример использования *%n*:

```
int i;  
printf("12345%n6789\n", &i);  
printf("i = %d\n", i );
```

Вывод программы:

```
123456789
```

```
i = 5
```

Как добраться до нужного указателя, находящегося в стеке, мы уже узнали. А теперь у нас в руках есть инструмент, который позволяет модифицировать память по этому указателю.

Конечно, пользоваться этим неудобно. Во-первых, мы можем записать только 4 байта (размер типа *int*). Если нам нужно большое число, то вначале функция *printf* должна будет выве-

сти очень много символов. Чтобы этого не делать, может помочь спецификатор `%00i`. Спецификатор влияет на значение текущего количества выведенных байт. Подробнее вникать в тонкости не будем.

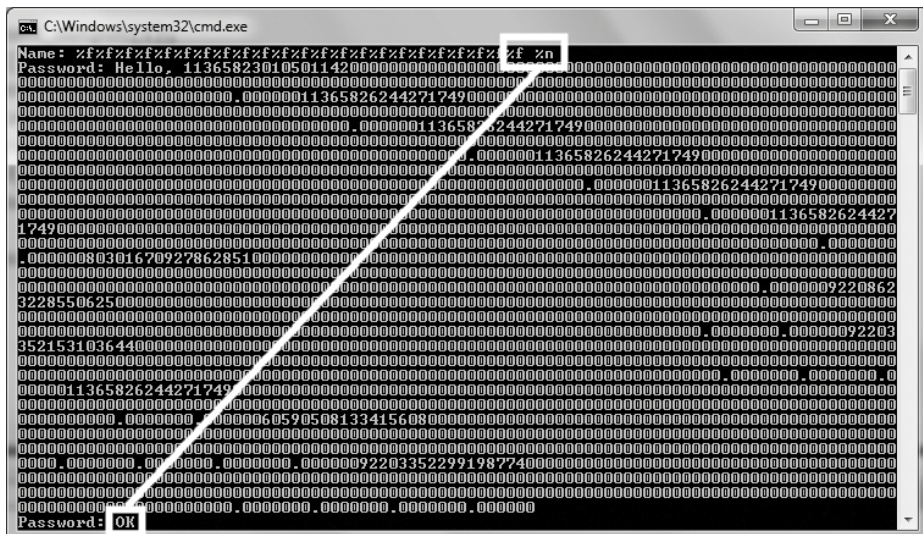
В нашем случае всё проще. Нам достаточно записать в переменную *isOkPassword* любое значение, не равное 0. Адрес этой переменной передаётся в функцию *IsCorrectPassword*, а значит, находится где-то в стеке. Пусть вас не смущает, что переменная передается как ссылка. На низком уровне ссылка является указателем.

Вот строка, которая позволит нам модифицировать переменную *IsCorrectPassword*:

```
%f%f%f%f%f%f%f%f%f%f%f%f%f%f%f%f%f%f%f%f%f%f %n
```

Спецификатор `%n` не учитывает количество символов, выведенных с помощью таких спецификаторов, как `%f`. Поэтому перед `%n` поставим один пробел, чтобы записать в `isOkPassword` значение 1.

Пробуем:



Впечатляет? Но это ещё далеко не всё. Можно произвести запись почти по произвольному адресу. Если выводимая строка

находится в стеке, то мы можем дойти до нужных символов и использовать их как адрес.

Например, мы можем написать строку, содержащую подряд символы с кодами `xF8`, `x32`, `x01`, `x7F`. Получается, что в строке есть жестко закодированное число, которое эквивалентно значению `0x7F0132F8`. В конце мы поставим спецификатор `%n`. Используя `%x` или другие спецификаторы, мы можем добраться до закодированного числа `0x7F0132F8` и записать количество выведенных символов по этому адресу. У такого способа есть ограничения. Но он всё равно очень любопытен, пусть хоть только с исторической точки зрения.

Выводы по второй атаке

Можно сказать, что атака второго рода сейчас вряд ли возможна. Как видите, поддержка спецификатора `%n` в современных библиотеках по умолчанию выключена. Однако можно создать свой самодельный механизм, который будет предрасположен данному виду уязвимости. Будьте аккуратны, когда введённые извне данные управляют тем, что и куда записать в память.

Конкретно в этом случае проблемы опять не возникнет, если написать так:

```
printf("%s", name);
```

Заключение

Здесь рассмотрено только два простых примера уязвимости. Конечно, их существует намного больше. Я не пытаюсь описать или перечислить их. Вместо этого хотел продемонстрировать, что опасность может представлять даже такая простая конструкция, как:

```
printf(getname().c_str());
```

Всегда используйте безопасный вариант кода:

```
printf("%s", getname().c_str());
```

Вообще, такие функции, как *printf*, *sprintf*, *fprintf*, в целом опас-

ны. Лучше их не использовать, а воспользоваться чем-то более современным. Например, вам могут пригодиться *boost::format*, *std::stringstream* или *std::format*.

Вредный совет N49. Виртуальные функции в конструкторах и деструкторах

Стандарт C++ чётко определяет, как работает вызов виртуальной функции в конструкторе и деструкторе. Значит, смело можно делать такие вызовы.

В идеальном мире совершенных программистов — это действительно так. Вот только на практике неправильное использование виртуальных функций — это классическая ошибка при разработке на языке C++. Причин две:

1. Можно пользоваться виртуальными функциями и не знать об особенностях их работы в конструкторах/деструкторах. Или забыть.
2. В разных языках программирования поведение виртуальных функций отличается. Это добавляет путаницы.

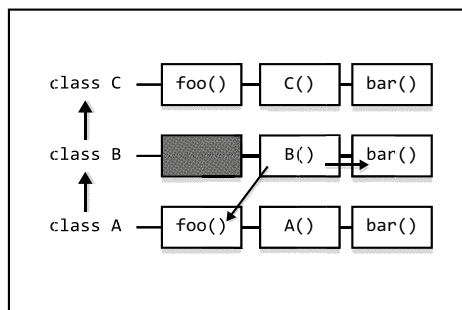
Давайте как следует разберёмся в этой теме. Глава полезна всем, кто сейчас изучает программирование на языке C++. Впрочем, даже бывалым программистам может оказаться полезным освежить познания в этой области. Особенно тем, кто пишет код сразу на нескольких языках.

Теория

Когда в конструкторе вызывается виртуальная функция, она работает только в пределах базовых или создаваемого в данный момент классов. Конструкторы в классах-наследниках ещё не вызывались, и поэтому реализованные в них виртуальные функции не будут вызваны.

Для начала поясню это рисунком:

- от класса *A* наследуется класс *B*;
- от класса *B* наследуется класс *C*;
- функции *foo* и *bar* являются виртуальными;
- у функции *foo* нет реализации в классе *B*.



Создадим объект класса *C* и рассмотрим, что произойдёт, если мы вызовем эти две функции в конструкторе класса *B*.

- **Функция *foo*.** Класс *C* ещё не создан, а в классе *B* нет функции *foo*. Поэтому будет вызвана реализация функции из класса *A*.
- **Функция *bar*.** Класс *C* ещё не создан. Поэтому вызывается функция, относящаяся к текущему классу *B*.

Теперь продемонстрирую то же самое кодом.

```
#include <iostream>
```

```
class A
{
public:
    A()                { std::cout << "A()\n";        };
    virtual void foo() { std::cout << "A::foo()\n"; };
    virtual void bar() { std::cout << "A::bar()\n"; };
};
```

```
class B : public A
{
public:
    B()
    {
        std::cout << "B()\n";
        foo();
        bar();
    };
    void bar() { std::cout << "B::bar()\n"; };
};
```

```
class C : public B
```

```
{  
public:  
    C()          { std::cout << "C()\n"; };  
    void foo() { std::cout << "C::foo()\n"; };  
    void bar() { std::cout << "C::bar()\n"; };  
};
```

```
int main()  
{  
    C x;  
    return 0;  
}
```

Если скомпилировать и запустить этот код, то он распечатает:

```
A()  
B()  
A::foo()  
B::bar()  
C()
```

При вызове виртуальных методов в деструкторах всё работает точно так же.

Казалось бы, в чём проблема? Всё это описано в книгах по программированию на языке C++.

Проблема в том, что про это легко забыть! И считать, что функции *foo* и *bar* будут вызваны из крайнего наследника, т. е. из класса *C*.

Думаю, теперь понятно, почему в подобном коде легко допустить ошибку. Особенно легко запутаться, если довелось программировать на других языках, где поведение отличается. Рассмотрим следующую программу на языке C#:

```
class Program
{
    class Base
    {
        public Base()
        {
            Test();
        }
        protected virtual void Test()
        {
            Console.WriteLine("From base");
        }
    }

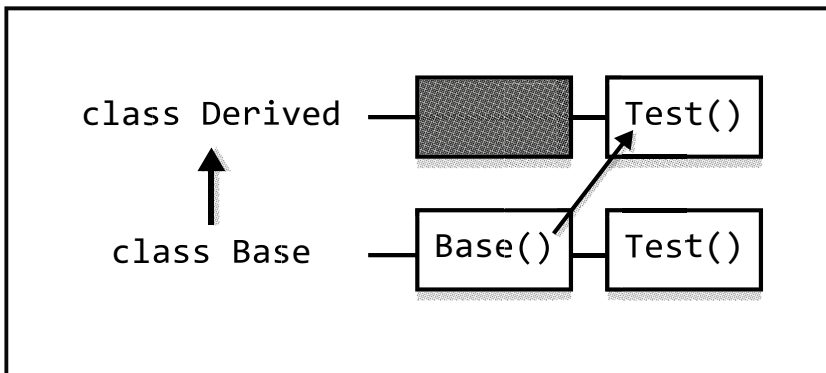
    class Derived : Base
    {
        protected override void Test()
        {
            Console.WriteLine("From derived");
        }
    }

    static void Main(string[] args)
    {
        var obj = new Derived();
    }
}
```

Если её запустить, то будет распечатано:

From derived

Соответствующая визуальная схема для C#:



Из конструктора базового класса вызывается метод наследника!

При вызове виртуального метода из конструктора учитывается тип времени выполнения создаваемого экземпляра. Исходя из этого типа и происходит виртуальный вызов. Несмотря на то, что вызов метода происходит в конструкторе базового типа, фактический тип создаваемого экземпляра — *Derived*, что и определяет выбор метода.

Стоит отметить, что такое поведение тоже может быть чревато ошибками. Например, проблемы могут возникнуть, если виртуальный метод работает с членами производного типа, которые ещё не были проинициализированы в его конструкторе.

Рассмотрим пример:

```
class Base
{
    public Base()
    {
        Test();
    }

    protected virtual void Test() { }
}

class Derived : Base
{
    public String MyStr { get; set; }

    public Derived(String myStr)
    {
        MyStr = myStr;
    }

    protected override void Test()
        => Console.WriteLine($"Length of {nameof(MyStr)}: {MyStr.Length}");
}
```

При попытке создания экземпляра типа *Derived* возникнет исключение типа *NullReferenceException*, даже если в качестве аргумента в конструктор передаётся значение, отличное от *null*: *new Derived(«Hello there»)*

При исполнении тела конструктора типа *Base* будет вызвана реализация метода *Test* из типа *Derived*. Этот метод обращается к свойству *MyStr*, которое в текущий момент проинициализировано значением по умолчанию (*null*), а не параметром, переданным в конструктор (*myStr*).

Как правильно написать код

Нет причины использовать механизм виртуальных функций в конструкторах/деструкторах. Это провоцирует ошибки и усложняет понимание кода. Я не одинок во мнении.

Запрет на вызов виртуальных функций в конструкторах/деструкторах нашёл своё отражение в различных стандартах кодирования. Например, в SEI CERT C++ Coding Standard есть правило: “OOP50-CPP. Do not invoke virtual functions from constructors or destructors”. Соответствующее диагностическое правило реализуют многие анализаторы кода: PVS-Studio, Parasoft C/C++test, Polyspace Bug Finder, PRQA QA-C++, SonarQube C/C++ Plugin и т. д.

Читатель может удивиться. Как же быть, если действительно нужно вызвать какую-то функцию в конструкторе/деструкторе, которая обязана быть виртуальной? Понятно, что можно ошибиться. Но ведь всё может быть написано правильно и работать именно так, как должно!

Для этого вам всё равно не нужен механизм виртуальных функций. Если вы знаете, какую функцию из данного или из базового класса вызывать, то явно её и вызывайте.

Явно укажите имя класса, из которого требуется вызвать функцию:

```
B() {  
    std::cout << "B()\n";  
    A::foo();  
    B::bar();  
};
```

Такой код будет однозначно правильно понят вашими коллегами. Статические анализаторы, в свою очередь, тоже всё поймут и промолчат.

Вредный совет N50. Нет времени думать, копируй код!

Копирование кода с последующей правкой — самый быстрый способ его написания.

Да здравствует `copy-paste` программирование!

Действительно, так можно быстро написать много кода. Это особенно удобно, если ваша работа оценивается по количеству строк кода. Впрочем, надеюсь, вы не работаете в такой компании.

Если мы говорим о разработке качественного приложения, а не о том, как создать больше «говнокода», то у `copy-paste` подхода есть три больших недостатка:

1. **Избыточный код.** Кода больше, чем необходимо для решения задачи. А чем больше кода, тем больше сил тратится на его обзоры, сопровождение, правку, отладку. Например, программист нашёл и исправил ошибку в коде, но забыл, что такой же фрагмент кода он использовал в другой функции. В результате вторая ошибка останется в коде и, когда она проявит себя, программисту придётся повторно её искать.
2. **Необходимость множественных правок.** При использовании методики `copy-paste` программа будет содержать схожие блоки кода, выполняющие приблизительно одни и те же действия. Когда нужно изменить логику поведения программы, высока вероятность, что придётся модифицировать все эти похожие места. Это отнимает время, и есть вероятность где-то забыть внести правку.
3. **Ошибки.** Копируя и затем модифицируя код, очень легко забыть исправить какое-то имя или числовой литерал. Возникают ошибки, которые сложно заметить на обзоре кода, так как блоки текста похожи и тяжело их проверять, не теряя внимание. В блоге PVS-Studio мы регулярно описываем подобные ошибки, которые находим в открытых проектах.

Исправить ситуацию можно, не копируя код, а вынося его в функции и затем используя из различных частей программы. Это устраняет все три проблемы:

1. Нет избыточного кода.
2. Логика работы правится только в одном месте.
3. Уменьшается вероятность допустить ошибку.

Рассмотрим, как такой рефакторинг может выглядеть на практике. Показанная ниже ошибка была найдена нами в проекте Blend2D.

```
static BLResult BL_CDECL bl_convert_argb32_from_prgb_any(
    const BLPixelConverterCore* self,
    uint8_t* dstData, intptr_t dstStride,
    const uint8_t* srcData, intptr_t srcStride,
    uint32_t w, uint32_t h, const BLPixelConverterOptions* options) noexcept {

    if (!options)
        options = &blPixelConverterDefaultOptions;

    const size_t gap = options->gap;
    dstStride -= intptr_t(w) * 4 + gap;
    srcStride -= intptr_t(w) * PixelAccess::kSize;

    const BLPixelConverterData::NativeFromForeign& d =
        blPixelConverterGetData(self)->nativeFromForeign;
    uint32_t rMask = d.masks[0];
    uint32_t gMask = d.masks[1];
    uint32_t bMask = d.masks[2];
    uint32_t aMask = d.masks[3];

    uint32_t rShift = d.shifts[0];
    uint32_t gShift = d.shifts[1];
    uint32_t bShift = d.shifts[2];
    uint32_t aShift = d.shifts[3];

    uint32_t rScale = d.scale[0];
    uint32_t gScale = d.scale[1];
    uint32_t bScale = d.scale[2];
    uint32_t aScale = d.scale[3];

    for (uint32_t y = h; y != 0; y--) {
```

```

if (!AlwaysUnaligned && blIsAligned(srcData, PixelAccess::kSize)) {
    for (uint32_t i = w; i != 0; i--) {
        uint32_t pix = PixelAccess::fetchA(srcData);
        uint32_t r = (((pix >> rShift) & rMask) * rScale) >> 16;
        uint32_t g = (((pix >> gShift) & gMask) * gScale) >> 8;
        uint32_t b = (((pix >> bShift) & bMask) * bScale) >> 8;
        uint32_t a = (((pix >> aShift) & aMask) * aScale) >> 24;

        BLPixelOps::unpremultiply_rgb_8bit(r, g, b, a);
        blMemWriteU32a(dstData, (a << 24) | (r << 16) | (g << 8) | b);

        dstData += 4;
        srcData += PixelAccess::kSize;
    }
}
else {
    for (uint32_t i = w; i != 0; i--) {
        uint32_t pix = PixelAccess::fetchA(srcData);
        uint32_t r = (((pix >> rShift) & rMask) * rScale) >> 16;
        uint32_t g = (((pix >> gShift) & gMask) * gScale) >> 8;
        uint32_t b = (((pix >> bShift) & bMask) * bScale) >> 8;
        uint32_t a = (((pix >> aShift) & aMask) * aScale) >> 24;

        BLPixelOps::unpremultiply_rgb_8bit(r, g, b, a);
        blMemWriteU32a(dstData, (a << 24) | (r << 16) | (g << 8) | b);

        dstData += 4;
        srcData += PixelAccess::kSize;
    }
}

dstData = blPixelConverterFillGap(dstData, gap);
dstData += dstStride;
srcData += srcStride;
}

return BL_SUCCESS;
}

```

В данном фрагменте кода `then` и `else`-ветки полностью совпадают. Этот код явно писался методом `copy-paste`. Программист скопировал внутренний цикл, а затем отвлёкся и забыл заменить во втором цикле вызов функции `PixelAccess::fetchA` на `PixelAccess::fetchU`.

К счастью, статический анализатор PVS-Studio заметил аномалию и выдал предупреждение: V523 The 'then' statement is equivalent to the 'else' statement. pixelconverter.cpp 1215

Эта классическая copy-paste ошибка появилась из-за того, что программист немного поленился при написании кода. Есть как минимум 3 способа избежать дублирования кода и заодно снизить вероятность рассмотренной ошибки.

Первый вариант — можно вынести внутренний цикл в отдельную функцию и передавать ей дополнительный флажок, который будет указывать, следует вызывать *PixelAccess::fetchA* или *PixelAccess::fetchU*.

Этот способ мне не нравится, так как этой новой функции придётся передавать очень много аргументов. Поэтому не будем его подробнее рассматривать. Отмечу только, что даже такой вариант снижает вероятность ошибки. Сложно забыть использовать аргумент функции (флажок). Тем более даже компилятор предупредит, что аргумент не используется.

Второй вариант — вынести общий код в лямбда-функцию. Тогда код может выглядеть приблизительно так:

```
static BLResult BL_CDECL bl_convert_argb32_from_prgb_any(
    const BLPixelConverterCore* self,
    uint8_t* dstData, intptr_t dstStride,
    const uint8_t* srcData, intptr_t srcStride,
    uint32_t w, uint32_t h, const BLPixelConverterOptions* options) noexcept {

    if (!options)
        options = &blPixelConverterDefaultOptions;

    const size_t gap = options->gap;
    dstStride -= uintptr_t(w) * 4 + gap;
    srcStride -= uintptr_t(w) * PixelAccess::kSize;

    const BLPixelConverterData::NativeFromForeign& d =
        blPixelConverterGetData(self)->nativeFromForeign;
    uint32_t rMask = d.masks[0];
    uint32_t gMask = d.masks[1];
    uint32_t bMask = d.masks[2];
    uint32_t aMask = d.masks[3];
```

```

uint32_t rShift = d.shifts[0];
uint32_t gShift = d.shifts[1];
uint32_t bShift = d.shifts[2];
uint32_t aShift = d.shifts[3];

uint32_t rScale = d.scale[0];
uint32_t gScale = d.scale[1];
uint32_t bScale = d.scale[2];
uint32_t aScale = d.scale[3];

auto LoopImpl = [&](const bool fetch_a)
{
    for (uint32_t i = w; i != 0; i--) {
        uint32_t pix = fetch_a ? PixelAccess::fetchA(srcData) :
                                PixelAccess::fetchU(srcData);
        uint32_t r = (((pix >> rShift) & rMask) * rScale) >> 16;
        uint32_t g = (((pix >> gShift) & gMask) * gScale) >> 8;
        uint32_t b = (((pix >> bShift) & bMask) * bScale) >> 8;
        uint32_t a = (((pix >> aShift) & aMask) * aScale) >> 24;

        BLPixelOps::unpremultiply_rgb_8bit(r, g, b, a);
        blMemWriteU32a(dstData, (a << 24) | (r << 16) | (g << 8) | b);

        dstData += 4;
        srcData += PixelAccess::kSize;
    }
};

for (uint32_t y = h; y != 0; y--) {
    LoopImpl(!AlwaysUnaligned && blIsAligned(srcData, PixelAccess::kSize));
    dstData = blPixelConverterFillGap(dstData, gap);
    dstData += dstStride;
    srcData += srcStride;
}

return BL_SUCCESS;
}

```

На самом деле, для рассматриваемого примера выносить что-то в функцию вообще нет необходимости. Но мне хотелось показать, как подход с функцией может сокращать код и уменьшать вероятность ошибки.

Третий вариант — самый простой и естественный — делать проверку внутри цикла.

```
static BLResult BL_CDECL bl_convert_argb32_from_prgb_any(
    const BLPixelConverterCore* self,
    uint8_t* dstData, intptr_t dstStride,
    const uint8_t* srcData, intptr_t srcStride,
    uint32_t w, uint32_t h, const BLPixelConverterOptions* options) noexcept {

    if (!options)
        options = &blPixelConverterDefaultOptions;

    const size_t gap = options->gap;
    dstStride -= uintptr_t(w) * 4 + gap;
    srcStride -= uintptr_t(w) * PixelAccess::kSize;

    const BLPixelConverterData::NativeFromForeign& d =
        blPixelConverterGetData(self)->nativeFromForeign;
    uint32_t rMask = d.masks[0];
    uint32_t gMask = d.masks[1];
    uint32_t bMask = d.masks[2];
    uint32_t aMask = d.masks[3];

    uint32_t rShift = d.shifts[0];
    uint32_t gShift = d.shifts[1];
    uint32_t bShift = d.shifts[2];
    uint32_t aShift = d.shifts[3];

    uint32_t rScale = d.scale[0];
    uint32_t gScale = d.scale[1];
    uint32_t bScale = d.scale[2];
    uint32_t aScale = d.scale[3];

    for (uint32_t y = h; y != 0; y--) {
        const bool fetch_a = !AlwaysUnaligned &&
            blIsAligned(srcData, PixelAccess::kSize);
        for (uint32_t i = w; i != 0; i--) {
            uint32_t pix = fetch_a ? PixelAccess::fetchA(srcData) :
                PixelAccess::fetchU(srcData);
            uint32_t r = (((pix >> rShift) & rMask) * rScale) >> 16;
            uint32_t g = (((pix >> gShift) & gMask) * gScale) >> 8;
            uint32_t b = (((pix >> bShift) & bMask) * bScale) >> 8;
            uint32_t a = (((pix >> aShift) & aMask) * aScale) >> 24;

            BLPixelOps::unpremultiply_rgb_8bit(r, g, b, a);
            blMemWriteU32a(dstData, (a << 24) | (r << 16) | (g << 8) | b);
        }
    }
}
```

```

        dstData += 4;
        srcData += PixelAccess::kSize;
    }

    dstData = blPixelConverterFillGap(dstData, gap);
    dstData += dstStride;
    srcData += srcStride;
}

return BL_SUCCESS;
}

```

Мы рассмотрели красивый пример, где можно избавиться от дублирования кода. Однако так бывает не всегда. Иногда явно виден *copy-paste* и вызванная этим ошибка, но непонятно, как можно было этого избежать. Рассмотрим такой случай на примере бага, найденного мною в LLVM 15.0:

```

static std::unordered_multimap<char32_t, std::string>
loadDataFiles(const std::string &NamesFile, const std::string &AliasesFile) {
    ....
    auto FirstSemiPos = Line.find(';');
    if (FirstSemiPos == std::string::npos)                // <=
        continue;
    auto SecondSemiPos = Line.find(';', FirstSemiPos + 1);
    if (FirstSemiPos == std::string::npos)                // <=
        continue;
    ....
}

```

Код писался с помощью *Copy-Paste*. Был размножен этот фрагмент:

```

auto FirstSemiPos = Line.find(';');
if (FirstSemiPos == std::string::npos)
    continue;

```

Затем поменяли вызов функции *find*. *First* заменили на *Second*, но не везде. В результате эта строчка:

```

if (FirstSemiPos == std::string::npos)

```

Осталась без изменений. Так как это условие уже проверялось выше, анализатор PVS-Studio сообщает, что условие всегда ложно: V547 Expression 'FirstSemiPos == std::string::npos' is always false. UnicodeNameMappingGenerator.cpp 46

В данном случае нечего обобщать и выносить в функцию. И непонятно, как по-другому написать этот код, чтобы избежать ошибки.

Кто-то может сказать: надо было не копировать код, а написать его заново. Тогда бы и ошибки не было. А если хочется что-то скопировать — это повод остановиться, подумать и сделать функцию или по-другому написать код.

Звучит разумно, но я не верю в такой подход. Что бы там ни говорили, но иногда быстро, удобно и эффективно именно скопировать фрагмент кода. В общем, я не буду столь радикален, чтобы предлагать никогда не использовать `copy-paste`.

Более того, я вообще сомневаюсь в искренности людей, которые призывают никогда не использовать копирование кода. Небось оставили умный комментарий, чтобы казаться идеальней других, и пошли дальше `copy-paste` в своём проекте делать.

В общем, все копировали код и будут продолжать это делать. Да, количество таких случаев можно и нужно сокращать. Однако надо признать, что всё равно какие-то фрагменты будут копироваться, и это будет приводить к ошибкам. Что же делать?

Советы всё те же:

- писать простой понятный код. В нём будет легче заметить ошибку;
- использовать форматирование кода таблицей (см. главу N18);
- использовать статические анализаторы кода.

Вредный совет N51. Можно заглянуть за пределы массива

Раз допустимо ссылаться на следующий элемент за пределами массива, то, значит, можно к этому элементу и обращаться.

Ой, это же уже 51-я глава, а в содержании их только 50. Это пасхалка. В коллекции вредных советов обязан быть выход за границу массива!

Выход за границу массива приводит к неопределённому поведению. Однако есть один момент, который может смутить неопытного программиста.

В C++ допустимо ссылаться на элемент, лежащий за последним элементом массива. Например, допустим следующий код:

```
int array[5] = { 0, 1, 2, 3, 4 };  
int *ptr = array + 5;
```

Однако значение указателя *ptr* можно только сравнивать с другими значениями, но не разыменовывать.

Подобное допущение позволяет построить изящную концепцию итераторов. В классах для работы с массивами функция *end* возвращает итератор, указывающий на условный элемент, находящийся за последним элементом контейнера. Итератор *end* можно сравнивать с другими итераторами, но нельзя разыменовывать.

Помимо этого, программисты просто по невнимательности допускают ошибку, выходя на 1 элемент за пределы массива. У такой ошибки даже есть название — *off-by-one error*. Причина в том, что элементы в массиве нумеруются с 0, и это иногда сбивает с толку, особенно при поспешном написании кода.

Чаще всего ошибка возникает из-за неправильной проверки индекса. Проверяют, что индекс не больше количества элементов в массиве. Но это неправильно, так как если индекс равен

количеству элементов, то он уже ссылается на элемент за пределами массива. Поясним это практическим примером.

Следующая ошибка была найдена мною с помощью PVS-Studio в проекте LLVM 11. Так что, как видите, подобные ляпы допускают не только новички.

```
std::vector<Decl *> DeclsLoaded;

SourceLocation ASTReader::getSourceLocationForDeclID(GlobalDeclID ID) {
    ....
    unsigned Index = ID - NUM_PREDEF_DECL_IDS;

    if (Index > DeclsLoaded.size()) {
        Error("declaration ID out-of-range for AST file");
        return SourceLocation();
    }

    if (Decl *D = DeclsLoaded[Index])
        return D->getLocation();
    ....
}
```

Правильная проверка должна быть такой:

```
if (Index >= DeclsLoaded.size()) {
```

Какой-то особенный совет, как избегать подобных ошибок, дать сложно. Просто желаю вам быть всегда внимательными: как при написании кода, так и при обзоре кода коллег. Удачи.

Заклучение

Спасибо, что прочитали книгу. Надеюсь, вам было интересно, а иногда даже немножко страшно от вида фрагментов кода. Если попробовать дать какую-то общую полезную рекомендацию, то, пожалуй, я сформулирую её так:

Пишите простой и красивый код: его будет легче понять, а ошибки в нём будут заметнее.

Совет неидеален, как и всё в мире. Но многие вредные паттерны связаны как раз с нарушением сказанного. Кто-то стремится срезать угол, кто-то стремится показать в коде свою крутизну, кто-то торопится.

Пишите код с любовью. Желаю вам продуктивного творчества и безбажного кода.

С уважением, Андрей Карпов

DevRel, PVS-Studio

Благодарности

Я благодарен солнечному городу Ташкент, в котором писал книгу. Когда я только начинал писать книгу, то думал, что напишу так: тёплому и солнечному. Чтобы быть честным, слово «тёплый» пришлось исключить. Я застал самую холодную за 50 лет зиму в Узбекистане. Впрочем, книге это пошло на пользу, поскольку несколько недель нечем было заниматься, кроме как писать. В любом случае Ташкент радушный, зелёный, быстро развивающийся город. Рекомендую.

Я благодарен своей жене Анне, которая помогает в моих начинаниях. Она очень позитивно поддержала идею написать эту книгу.

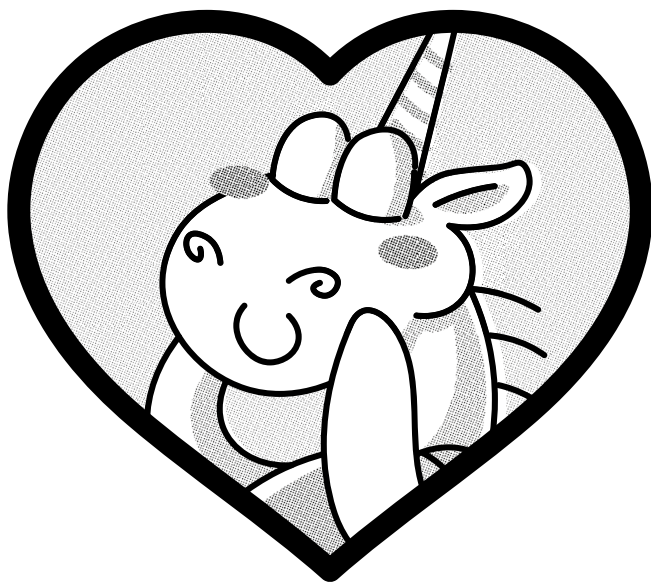
Спасибо Филиппу Хандельянцу за внимательную вычитку книги и консультации по некоторым аспектам языка C++. Благодарю Александра Куренева и Григория Семенчева в помощи с подготовкой терминологического раздела. Спасибо Инне Пристягиной, которая напоминала мне про идею написания книги, а потом помогла вычитать текст и подготовить его к публикации. Спасибо Сергею Васильеву, Евгении Елтуковой и Ирине Борисовой за стилистические правки.

Ярославу Павлову-Брейчеру я признателен за иллюстрации и помощь в оформлении созданного материала в виде книги.

Доброе слово Петру Козлёнкову, который подключался удалённо и помогал восстанавливать работоспособность моей «печатной машинки». В самый разгар творческих изысканий у меня перестал работать Office. Тот случай, когда все мои знания в программировании, тестировании и отладке программ оказались бессильны. Не помогла даже переустановка. Пётр выявил конфликт Office и антивируса после какого-то неудачного апдейта. Славься тайная магия админов!

В какой-то момент текст книги будет зафиксирован и, возможно, не удастся в самый последний момент поблагодарить

кого-то ещё. Поэтому на всякий случай ещё раз спасибо всем, кто мотивировал меня, помогал в написании книги и участвовал в подготовке её к печати и распространению. Да пребудет с вами Сила.



Терминология

Capacity

Вместимость контейнера, например, `std::vector`. Количество элементов, под которые в контейнере уже выделена память. Вместимость всегда больше или равна количеству элементов в контейнере, поскольку контейнер не может содержать элементы, для которых у него нет места.

В большинстве случаев при добавлении элемента вместимость будет больше, чем количество элементов. Это не что иное, как оптимизация, призванная сократить количество операций выделения памяти. Выделение памяти — очень медленная операция.

Другими словами, при добавлении элемента, например, в `vector` с помощью функции `push_back`, новый элемент попадёт в уже заранее выделенную под него память. Только когда элементы полностью заполнят выделенный буфер, контейнер выделит больший блок памяти (буфер) и переместит в него все существующие элементы.

Вы можете расширить вместимость некоторых контейнеров с помощью функции `reserve`. Выбирая вместимость с большим и точным значением, можно быть уверенным, что расширение контейнера не будет происходить до тех пор, пока количество элементов меньше вместимости.

DoS-атака

DoS-атака (аббревиатура от Denial of Service) — это атака с целью остановить или замедлить обработку запросов клиентов. DoS-атаки не имеют цели украсть или повредить данные клиентов. Сутью атаки является создание проблем с доступностью вычислительных ресурсов, из-за которых процессы не выполняются за приемлемое время. Вычислительными ресурсами обычно являются:

- процессорное время;
- объём оперативной памяти;
- объём дисковой памяти;
- пропускная способность сетевого канала;
- пропускная способность канала ввода-вывода памяти.

Атаки могут быть разделены на две группы: DDoS (distributed denial-of-service) и DoS-атаки уровня приложения.

DDoS (distributed denial-of-service)

DDoS-атака достигается созданием огромного количества обычных клиентских запросов (входных данных) к API системы. Атака производится множеством устройств, поэтому и называется распределённой. Обычно для системы эти устройства и их запросы не имеют явных проблем для процессов её внутренней логики. Запросы не содержат вредоносных или использующих уязвимостей данных. Проблемы возникают, когда сетевой канал сервера не имеет достаточной пропускной способности для обработки всех запросов или вычислительные ресурсы машины, на которой установлен сервер, исчерпываются.

DoS-атаки уровня приложения

DoS-атаки уровня приложения производятся через особенности и проблемы архитектур программ и оборудования. Атаки также нацелены на достижение состояния исчерпания ресурсов хост-машины. Атаки производятся малым количеством устройств, но с использованием уязвимостей или архитектурных проблем системы. Для использования этих уязвимостей и проблем подбираются специальные данные, обработка которых приводит к нарушению процессов.

При неправильной архитектуре системы случайная DoS-атака может произойти даже без злого умысла. Ошибки в архитектуре системы могут быть причиной аварийного останова программы или большого расхода ресурсов даже при работе с обычными данными. Например:

- отсутствие обработчика некоторых ошибок;
- использование неэффективных алгоритмов с вычислительной сложностью больше $O(n^2)$;
- ошибки в условных конструкциях, приводящие к бесконечному циклу;
- утечки памяти;
- выход за границу буфера (access violation);
- создание неэффективных регулярных выражений и отсутствие ограничений по времени поиска текста.

MISRA, AUTOSAR

MISRA C и MISRA C++ — это стандарты разработки программного обеспечения, созданные организацией MISRA (Motor Industry Software Reliability Association). На момент написания книги (начало 2023 года) эти стандарты устарели. Последняя версия MISRA C датируется 2012 годом, а MISRA C++ — 2008. Однако известно, что сейчас идёт работа над реновацией этих стандартов. Скоро можно ожидать выход их обновлённых версий, учитывающих изменения, произошедшие в языках C и C++.

AUTOSAR C++14 — это набор руководств для написания кода на языке C++14, который предназначен для работы в системах, где важны безопасность и отказоустойчивость. Этот документ является обновлением стандарта MISRA C++:2008, а также основан на других ведущих стандартах написания кода и исследований. Основной сферой применения данного документа является автомобильная промышленность, но он также может быть использован и в других отраслях, занимающихся разработкой встраиваемых систем.

Стандарты родственны и служат одной цели: улучшить безопасность, переносимость и надёжность программ для встраиваемых систем.

В стандартах много правил, которым стоит следовать всегда, независимо от того, для чего предназначен ваш проект. По большей части они призваны устранить неопределённое/не-

уточнённое/зависимое от реализации поведение. Например:

- не используйте значение неинициализированной переменной;
- не используйте указатель на *FILE* после закрытия файла;
- все non-void функции должны возвращать значение.

Есть менее строгие правила. В целом они полезны, но их нарушение в обыкновенной программе не ведёт к серьёзным последствиям:

- каждый *switch* должен заканчиваться *default*;
- суффиксы численных литералов должны быть написаны в верхнем регистре: *100i* — неправильно, *100U* — правильно;
- не используйте функции с переменным числом аргументов.

У людей, которые впервые сталкиваются с MISRA, складывается впечатление, что философия стандарта — запретить вон то и запретить вот это. На самом деле, это так, но лишь отчасти.

Стандарт действительно имеет много подобных правил, но его цель — не запретить всё возможное, а перечислить способы, которые снижают надёжность кода. Для большинства правил вы сами выбираете, стоит им следовать или нет. В MISRA C правила делятся на три основные категории:

1. **Mandatory** — это правила, которые нельзя нарушать ни под каким предлогом. Например, к этому разделу относится правило «не используйте значение неинициализированной переменной».
2. **Required**-правила менее строги: они допускают возможность отклонения, но только если эти отклонения тщательно документируются и письменно обосновываются.
3. **Advisory** — это правила, которым следовать не обязательно.

В MISRA C++ немного по-другому. Там отсутствует категория

Mandatory, и большинство правил принадлежит к категории Required. По сути, вы имеете право нарушить любое правило — только не забывайте документировать отклонения.

Overcommit

Стратегия выделения памяти, когда операционная система разрешает приложениям занимать больше виртуальной памяти, чем доступно в системе. Можно перечислить несколько стратегий работы операционной системы:

- ядро всегда удовлетворяет любые запросы на выделение памяти. Реально страницы памяти будут выделяться при первом обращении к ним;
- эвристический подход к распределению памяти. Используется по умолчанию, так как подходит для большинства задач. Система будет отвергать только запросы, которые в принципе не могут быть удовлетворены, остальные — удовлетворять вне зависимости от наличия свободной памяти;
- работа без overcommit. Система будет выделять память, только если она подкреплена реальными страницами в ОЗУ или свопе.

ptrdiff_t

ptrdiff_t — это специальный знаковый целочисленный тип, определённый в стандартных библиотеках языков C и C++. Является типом результата вычитания указателей. Поведение типа схоже с *size_t*: на 32-битной системе размер *ptrdiff_t* будет 32 бита, на 64-битной — 64 бита.

При работе с контейнерами стандартной библиотеки результат вычитания двух итераторов имеет тип *difference_type* используемого контейнера, который, в зависимости от стандартной библиотеки, часто равен типу *ptrdiff_t*.

Для массивов меньших, чем *PTRDIFF_MAX*, *ptrdiff_t* ведёт себя как аналог *size_t*: может хранить размер массива любого типа и на большинстве платформ является синонимом *intptr_t*. Од-

нако если массив достаточно большой (больше *PTRDIFF_MAX*, но меньше *SIZE_MAX*) и разница его указателей не может быть представлена в виде *ptrdiff_t*, то результат вычитания таких указателей не определён.

В языке С тип *ptrdiff_t* объявлен в заголовочном файле `<stddef.h>`. В языке С++ его декларация находится в `<cstddef>` и размещается в глобальном пространстве имён и в *std*. Стандартные заголовочные файлы языка С для обеспечения обратной совместимости также могут включаться в С++ программы.

size_t

size_t — это специальный беззнаковый целочисленный тип, определённый в стандартных библиотеках языков С и С++. Является типом результата, возвращаемого оператором *sizeof* и *alignof*.

Размер *size_t* выбирается таким образом, чтобы в него можно было записать максимальный размер теоретически возможного массива или объекта. Другими словами, количество бит в *size_t* равно количеству бит, которое требуется для хранения максимального адреса в памяти машины. Например, на 32-битной системе *size_t* будет занимать 32-бита, на 64-битной — 64 бита.. В свою очередь это означает, что в тип *size_t* может быть безопасно помещён указатель (исключение составляют платформы с сегментной адресацией и указатели на функции-члены классов).

Данный тип перекладывает заботу о возможном разном поведении целочисленных переменных при смене платформы с плеч программиста на реализацию стандартной библиотеки. Поэтому использование типа *size_t* безопаснее и эффективнее, чем использование обычных беззнаковых целочисленных типов:

1. Данный тип позволяет писать циклы и счётчики, не беспокоясь о возможном переполнении при смене платформы. Например, когда количество необходимых ите-

- раций превышает `UINT_MAX`.
2. Поскольку стандартом гарантируется, что `size_t` может вместить в себя размер максимально возможного объекта в системе, этот тип используют для хранения размеров объектов.
 3. Из-за этого же свойства `size_t` используют для индексации массивов. В отличие от обычных базовых целочисленных типов, `size_t` гарантирует, что значение индекса не может быть больше `SIZE_MAX`.
 4. Так как в `size_t` обычно может быть безопасно помещён указатель, его используют для адресной арифметики. Однако для этих целей лучше подходит другой беззнаковый целочисленный тип — `uintptr_t`, само название которого отражает его предназначение.
 5. Компилятор может построить более простой и, следовательно, более быстрый код, в котором будут отсутствовать лишние преобразования 32-битных и 64-битных данных.

В языке C тип `size_t` объявлен в заголовочных файлах `<stddef.h>`, `<stdlib.h>`, `<string.h>`, `<wchar.h>`, `<uchar.h>`, `<time.h>` и `<stdio.h>`. В языке C++ его декларация находится в файлах `<cstddef>`, `<cstdlib>`, `<cstring>`, `<cwchar>`, `<cuchar>`, `<ctime>` и `<cstdio>`. Тип `size_t` размещён в глобальном пространстве имён и в `std`. Стандартные заголовочные файлы языка C для обеспечения обратной совместимости также могут включаться в C++ программы.

SSO (Small String Optimization)

Small String Optimization (или Short String Optimization) — это оптимизация, применяемая в шаблоне класса `std::basic_string` и его аналогах. Позволяет избегать дополнительных аллокаций динамической памяти для строк небольшого размера и размещать их внутри самого объекта.

Для пояснения принципа работы оптимизации рассмотрим пример простейшей реализации строки, которая оперирует символами типа `char`:

```
class string
{
    size_t capacity;
    char *buf;
    size_t size;
    // ....
};
```

Эта неэффективная реализация строки хранит три поля: указатель на динамический буфер (*buf*), реальный размер строки (*size*) и реальный размер буфера (*capacity*).

Основная проблема заключается в том, что в такой реализации даже для пустых строк будет необходимо выделить буфер и записать в его начало терминальный ноль. Это нужно для того, чтобы такую строку можно было без проблем передавать в функции, ожидающие нуль-терминированные строки (например, *strlen*).

Для устранения накладных расходов можно разместить символы прямо внутри объекта строки. Для этого применим SSO и немного модифицируем класс:

```
class string
{
    size_t capacity;

    union
    {
        struct
        {
            char *ptr;
            size_t size;
        } heapbuf;

        char stackbuf[sizeof(heapbuf)];
    };
};
```

В новой реализации можно расположить строки небольшой длины внутри объекта в *stackbuf*, не аллоцируя буфер на куче. Количество хранимых символов внутри объекта зависит от размера указателя и *size_t*. Например, на 64-битной платформе размер *heapbuf* будет составлять 16 байт, следовательно,

внутри объекта можно хранить до 15 символов и терминальный ноль. На основе нестатического поля *capacity* определяется, где хранятся символы:

- если *capacity* превышает размер *stackbuf*, то объект управляет буфером на куче и хранит символы там;
- иначе символы хранятся внутри объекта.

Многие реализации строк используют SSO, чтобы ускорить работу с короткими строками. Например, SSO используется в следующих библиотеках: Microsoft STL, libstdc++, libc++, Boost, Folly.

VLA (Variable Length Arrays)

Массив переменной длины — это массив, длина которого определяется на этапе исполнения программы. Несмотря на сбивающее с толку название, длина такого массива не может измениться во время исполнения программы после его декларации. Массив, длина которого может измениться во время исполнения программы, называется массивом с динамической длиной.

Массивы переменной длины опционально доступны в языке C, начиная со стандарта C99, и обязательны к реализации, начиная с C23. В C++ массивы переменной длины доступны только в качестве нестандартных расширений.

Массив переменной длины декларируется следующим образом:

```
void f()
{
    size_t n = get();
    int32_t array[n];
    ....
}
```

Массив переменной длины также может быть объявлен как параметр функции. Вместо размера может использоваться символ '*'. Такое разрешено только при декларации формальных

параметров и только в прототипе функции. В этом случае массив имеет неуточнённый размер:

```
int sum(size_t n, int arr[*]);
```

При передаче массива переменной длины он так же, как и обычные массивы, передаётся через указатель. Поэтому `sizeof(arr) == sizeof(int *)`.

Рассмотрим пример использования массива переменной длины. Допустим, нам нужно вычислить сумму произвольного количества чисел. Поскольку на этапе компиляции количество чисел неизвестно, используем массив переменной длины:

```
int32_t Sum(size_t, int32_t arr[*]); // Декларация
```

```
int32_t Sum(size_t n, int32_t arr[n]) // Реализация
```

```
{
    int32_t sum = 0;
    for (size_t i = 0; i < n; ++i)
    {
        sum += arr[i];
    }
    return sum;
}
```

```
int32_t foo(size_t n)
{
    int32_t rnd[n]; // VLA массив
    for (size_t i = 0; i < n; ++i)
    {
        rnd[i] = rand();
    }
    return Sum(n, rnd);
}
```

В этом примере аргумент `int32_t arr[n]` эквивалентен `int32_t *arr`. Такую форму записи указателя используем, чтобы человеку, читающему код, было ясно, что в качестве второго аргумента функции ожидается массив переменной длины, а не простой указатель.

Обзор кода

Обзор кода (инспекции кода, code review) — один из самых ста-

рых и полезных методов выявления дефектов. Он основан на простой идее, используемой во множестве других сфер человеческой деятельности, — дублировании. Если проблему будут решать и обсуждать несколько человек, то они смогут предложить лучшее решение и избежать множества ошибок. Когда человек работает один, то он может не замечать, что допускает явную ошибку или реализует что-то неоптимальным способом.

Обзор заключается в совместном внимательном чтении исходного кода и высказывании рекомендаций по его улучшению. В процессе чтения кода выявляются ошибки или участки кода, которые могут стать ошибочными в будущем. Также считается, что автор кода во время обзора не должен давать объяснений, как работает та или иная часть программы. Алгоритм работы должен быть понятен непосредственно из текста программы и комментариев. Если это условие не выполняется, то код должен быть доработан.

Обзоры кода работают, так как свежим взглядом бывает проще заметить ошибку в чужом коде, чем в собственном. Также обзор кода носит обучающую составляющую. Участники обсуждения узнают новые приёмы программирования и новые паттерны, учатся красиво оформлять код.

Недостатком методологии совместного обзора кода является стоимость её использования. Необходимо регулярно собирать нескольких программистов для обзора нового кода или повторного обзора кода после внесения изменений. Это отвлекает программистов от собственных задач и требует их настроиться на новую работу. При этом программисты должны регулярно делать перерывы для отдыха. Если пытаться просматривать сразу большие фрагменты кода, то внимание быстро притупляется и польза от обзора кода быстро уменьшается.

Есть даже шутка. Покажи коллегам 10 новых строчек кода, и они сделают 10 замечаний. Покажи им тысячу строк, и они скажут «всё ок».

С-строки (ноль-терминированные строки)

Способ представления строки в памяти, представляющий собой последовательность символов, заканчивающуюся специальным ноль-символом (NUL в ASCII таблице, со значением 0).

Фактически С-строки представляют собой указатель на первый символ строки в буфере памяти. Строка заканчивается там, где будет встречен первый ноль-терминальный символ.

Пример С-строки:

```
char str[10];  
strcpy(str, "Firefly");
```

Слово «Firefly» расположено в массиве из 10 байт следующим образом:

F	i	r	e	f	l	y	NUL		
0x46	0x69	0x72	0x65	0x66	0x6C	0x79	0x00	0x??	0x??

Строка занимает 8 байт. В первых 7 байтах хранятся символы слова, а в восьмом байте — терминальный ноль. Последние 2 байта в массиве остаются неинициализированными и могут содержать любые значения.

При использовании однобайтных кодировок (ASCII) объём памяти, требуемый для представления строки из N символов, равен $N + 1$ байт.

В том случае, когда для кодирования символов применяется Юникод, для хранения одного символа строки требуется 2 (кодировка UTF-16) или более байт. Соответственно, терминальный ноль также будет представлен несколькими нулевыми байтами.

Теория разбитых окон

Криминологическая теория, которая интерпретирует мелкие

правонарушения не только как индикатор уровня преступности, но и как фактор, влияющий на уровень этой преступности.

Сформулирована социологами Джеймсом Уилсоном и Джорджем Келлингом. Название берёт своё начало из приводимого ими примера: «Если в здании разбито одно стекло и никто его не заменяет, то через некоторое время в этом здании не останется ни одного целого окна».

Теория говорит, что игнорирование обществом мелких правонарушений (вандализм, публичное пьянство, выбрасывание мусора мимо урны) провоцирует людей к совершению подобных и более серьёзных правонарушений. С точки зрения психологии, это работает по принципу «если можно им, можно и мне». Если общество не прикладывает усилий к наведению порядка, то уровень «допустимости» постепенно повышается, что со временем приведёт к более частым и серьёзным преступлениям.

Теория подвержена критике, но хорошо согласуется с наблюдением из мира программирования: чем больше предупреждений компилятора/анализатора, тем меньше на них обращают внимания.

Уязвимость нулевого дня (0-day vulnerability)

Термин обозначает ещё не устранённые уязвимости. В программном обеспечении могут существовать бреши и уязвимости, которые были допущены разработчиками, но при этом ещё не были обнаружены ими. До тех пор, пока уязвимость не будет устранена, она может использоваться для доступа к сетям, удалённого управления компьютером, манипуляций с данными и т. п.

Название термина отражает, что у разработчиков нет ни дня на исправление дефекта, так как они о нём пока не знают.

Множество уязвимостей связано не с какими-то просчётами в системах безопасности или высокоуровневыми ошибками в логике, а с обыкновенными ошибками в коде приложений.

Важной составляющей выявления уязвимостей нулевого дня яв-

ляется использование статических анализаторов кода. Здесь следует отметить, что статические анализаторы — это достаточно общее понятие. Чтобы подчеркнуть, что анализатор ориентирован предотвращать уязвимости, его классифицируют как средство статического тестирования защищённости приложений (static application security testing — SAST). К таким инструментам относится и уже не раз упомянутый анализатор PVS-Studio.

Неправильно говорить, что SAST-решения непосредственно выявляют уязвимости нулевого дня. Анализаторы выявляют потенциальные уязвимости. Другими словами, они указывают на странные/аномальные фрагменты кода, которые могут являться ошибками и дефектами безопасности.

Только небольшая часть выявленных ошибок на самом деле представляет угрозу. Возьмём, например, ошибку — переполнение буфера. Она классифицируется как опасная с точки зрения безопасности. Но только малую часть таких ошибок можно эксплуатировать. Большинство из них приводят к неприятным, но не фатальным последствиям. Например, переполнение буфера может приводить к порче изображения.

Хотя только часть ошибок может стать причинами уязвимостей, программистам нет смысла пытаться их как-то разделять. Рационально исправлять все участки кода, на которые выданы предупреждения (кроме случаев явно ложных срабатываний анализатора). Даже если исправляется не потенциальная уязвимость, а просто ошибка, это всё равно полезно. Тем более что часто очень непросто понять, представляет ли та или иная ошибка угрозу с точки зрения безопасности. Лучше не рисковать, а исправить её.

Введение.....	4
Вредный совет N1. Только C++	6
Вредный совет N2. Табуляция в строковых литералах	8
Вредный совет N3. Вложенные макросы.....	11
Вредный совет N4. Выключить предупреждения.....	19
Вредный совет N5. Чем короче имя переменной, тем лучше.....	22
Вредный совет N6. Магические числа	24
Вредный совет N7. Везде int.....	28
Вредный совет N8. sizeof(int) == sizeof(void *).....	32
Вредный совет N9. Глобальные переменные.....	34
Вредный совет N10. abort в библиотеках	36
Вредный совет N11. Во всём виноват компилятор.....	37
Вредный совет N12. double == double	41
Вредный совет N13. Undefined behavior — просто страшилка	45
Вредный совет N14. memmove — лишняя функция.....	54
Вредный совет N15. Не проверяй, что вернула функция malloc	57
Вредный совет N16. Расширяй пространство std	76
Вредный совет N17. Старая школа	81
Вредный совет N18. Компактный код	90
Вредный совет N19. Анализаторы нужны только студентам	99
Вредный совет N20. Нет разницы между delete и delete[]	102

Вредный совет N21. Просьба помощь на форуме, делай так, чтобы из тебя вытягивали информацию. Так всем будет интереснее	111
Вредный совет N22. Ромбовидное наследование	119
Вредный совет N23. Напишу всё сам	125
Вредный совет N24. Удалите stdafx.h	128
Вредный совет N25. Массив на стеке — лучшее решение ...	139
Вредный совет N26. Ничего «лишнего»	144
Вредный совет N27. Необычные конструкции	145
Вредный совет N28. Всё в h-файлах	147
Вредный совет N29. Оператор goto	149
Вредный совет N30. Везде нужен некий константный экземпляр класса? Для удобства объявите его в заголовочном файле	151
Вредный совет N31. Объявление переменных в начале функции	155
Вредный совет N32. Создай свой h-квест	162
Вредный совет N33. C-style cast	164
Вредный совет N34. Ты властелин указателей — делай что хочешь	167
Вредный совет N35. const — лишняя сущность	170
Вредный совет N36. Не инициализируй	172
Вредный совет N37. Доверяй всем	175
Вредный совет N38. Тренируйся на работе для IOCCC	176
Вредный совет N39. Развлекайся, оформляя код	177
Вредный совет N40. У каждого свой стиль	178

Вредный совет N41. Перегрузи всё	180
Вредный совет N42. Не верь в эффективность std::string	181
Вредный совет N43. Максимально откладывая использова- ние нового C++ стандарта	185
Вредный совет N44. Переиспользование переменных	186
Вредный совет N45. Отвечай в комментарии на вопрос «что?»	189
Вредный совет N46. Больше многопоточности	192
Вредный совет N47. Чтение книг уже неактуально	195
Вредный совет N48. printf(str);	198
Вредный совет N49. Виртуальные функции в конструкторах и деструкторах	209
Вредный совет N50. Нет времени думать, копируй код!	215
Заключение	225
Благодарности	226
Терминология	228



Карпов Андрей Николаевич к.ф.-м.н.

Более 15 лет занимается темой статического анализа кода и качества программного обеспечения. Автор большого количества статей, посвящённых написанию качественного кода на языке C++. С 2011 по 2021 год удостоивался награды Microsoft MVP в номинации Developer Technologies. Один из основателей проекта PVS-Studio. Долгое время являлся СТО компании и занимался разработкой C++ ядра анализатора.

Основная деятельность на данный момент — управление командами, обучение сотрудников и DevRel активности.